

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-104376-117218

NÁVRH IMPLEMENTÁCIE VYBRANÝCH ALGORITMOV RIADENIA

Diplomová práca

Študijný program:	Robotika a kybernetika
Študijný odbor:	Kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Školiteľ:	Ing. Marián Tárník, PhD.

Bratislava 2026

Ing. Andrii Stoliar



ZADANIE DIPLOMOVEJ PRÁCE

Študent: **Bc. Andrii Stoliar**
ID študenta: 117218
Študijný program: robotika a kybernetika
Študijný odbor: kybernetika
Vedúci práce: Ing. Marián Tárník, PhD.
Vedúci pracoviska: prof. Ing. František Duchoň, PhD.
Miesto vypracovania: Ústav robotiky a kybernetiky

Názov práce: **Návrh implementácie vybraných algoritmov riadenia**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Práca sa zameriava na samotnú implementáciu algoritmov riadenia pričom sa predpokladá uplatnenie poznatkov z oblasti pokročilých metód automatického riadenia v rámci príslušného študijného programu. Hlavným cieľom práce je zostavenie riadiaceho systému pre vybraný laboratórny dynamický systém vrátane vlastného softvérového návrhu pre dostupný laboratórny hardvér (mikrokontroléry, PLC a procesory pre distribuovaný riadiaci systém). Pre splnenie cieľa práce je tiež potrebné adekvátne uplatňovať postupy z oblasti identifikácie systémov, numerickej simulácie a vyhodnocovania kvality riadenia systémov.

Úlohy:

1. Oboznámte sa s dostupným laboratórnym hardvérom a vypracujte stručnú technickú dokumentáciu vybraných laboratórnych dynamických systémov.
2. Vypracujte krátky prehľad riadiacich algoritmov vybraných pre následnú implementáciu.
3. Navrhnite a realizujte predmetné riadiace systémy a analyzujte kvalitu riadenia.
4. Vyhodnoťte dosiahnuté výsledky.

Termín odovzdania diplomovej práce: 15. 05. 2026
Dátum schválenia zadania diplomovej práce: 06. 11. 2025
Zadanie diplomovej práce schválil: prof. Ing. Jarmila Pavlovičová, PhD. – garantka študijného programu

Podakovanie

Na tomto mieste by som rád podakoval školiteľovi Ing. Mariánovi Tárnikovi, PhD., za odborné vedenie, cenné rady a trpezlivosť počas vypracovania tejto diplomovej práce. Podakovanie patrí aj doc. Ing. Ladislavovi Körösimu, PhD., za pomoc a podporu pri práci s programovateľným logickým automatom.

Zároveň ďakujem celému Ústavu robotiky a kybernetiky za odovzdanie cenných vedomostí a skúseností, ktoré mi pomohli rozvíjať sa v oblasti riadenia a automatizácie.

Veľká vďaka patrí aj mojim priateľom, starým rodičom, mame, otcovi, sestre a mojej partnerke za ich obrovskú podporu, povzbudenie a pochopenie počas celého štúdia.

Andrii Stoliar
Bratislava, 2026

Anotácia diplomovej práce

Slovenská technická univerzita v Bratislave
Fakulta elektrotechniky a informatiky

Študijný odbor:	kybernetika
Študijný program:	Robotika a kybernetika
Autor:	Bc. Andrii Stoliar
Diplomová práca:	Návrh implementácie vybraných algoritmov riadenia
Školiteľ:	Ing. Marián Tárník, PhD.
Mesiac, rok odovzdania:	Máj, 2026

Kľúčové slová: implementácia algoritmu riadenia, laboratórny dynamický systém, identifikácia systémov, PID regulátor, adaptívne riadenie, MATLAB/Simulink, Arduino UNO, C++, programovateľný logický automat, PLC

Diplomová práca prezentuje návrh implementácie riadiaceho systému pre laboratórne zariadenie predstavujúce reálny dynamický riadený systém. Implementované sú tri algoritmy riadenia: PI regulátor, PI regulátor s prepínaním pracovných bodov a adaptívny regulátor typu MRAC. Riadeným systémom je tepelný systém, kde riadenou veličinou je teplota vzduchu vháňaného do trubice. K riadenému systému je vypracovaná dokumentácia jeho komponentov a komunikačného rozhrania. Zrealizovaná je tiež potrebná identifikácia matematických modelov daného systému z hľadiska jeho statických aj dynamických vlastností. Samotná implementácia algoritmov riadenia je realizovaná na troch rôznych platformách: v prostredí MATLAB/Simulink, v jazyku C++ na vývojovej doske Arduino UNO a na priemyselnom PLC systéme. Ku každému prípadu je prezentovaný teoretický rozbor algoritmu, jeho sumarizácia vo forme pseudo-kódu a technická dokumentácia výslednej implementácie. Súčasťou je aj overenie každého z implementovaných algoritmov riadenia na reálnom laboratórnom zariadení a vzájomné porovnanie výsledkov pre daný algoritmus implementovaný na rôznych platformách. Výsledky práce ukazujú praktické možnosti implementácie zvolených algoritmov na rôznych platformách a zároveň poukazujú na vplyv merania, časovania, spracovania signálov a vlastností hardvéru na výsledné správanie regulačného systému.

Master Thesis Abstract

Slovak University of Technology in Bratislava
Faculty of Electrical Engineering and Information Technology

Branch of Study: Cybernetics
Study Programme: Robotics and Cybernetics
Author: Bc. Andrii Stoliar
Thesis Title: Implementation Design of Selected Control Algorithms
Supervisor: Ing. Marián Tárník, PhD.
Month, Year: May, 2026

Keywords: control algorithm implementation, laboratory dynamic system, system identification, PID controller, adaptive control, MATLAB/Simulink, Arduino UNO, C++, programmable logic controller, PLC

This master's thesis presents the design and implementation of a control system for a laboratory device representing a real dynamic controlled system. Three control algorithms are implemented: a PI controller, a PI controller with operating point switching, and an adaptive controller of the MRAC type. The controlled system is a thermal system in which the controlled variable is the temperature of air forced into a tube. Documentation of the system components and the communication interface is provided for the controlled system. The necessary identification of mathematical models of the system, considering both its static and dynamic properties, is also carried out. The implementation of the control algorithms is realized on three different platforms: in the MATLAB/Simulink software, using the C++ programming language on an Arduino UNO development board, and by means of an industrial PLC system. For each case, a theoretical analysis of the algorithm, its summary in the form of pseudocode, and technical documentation of the resulting implementation are presented. The thesis also includes verification of each implemented control algorithm on a real laboratory device, together with a comparative evaluation of the results obtained for the same algorithm implemented on different platforms. The results demonstrate the practical possibilities of implementing the selected algorithms on various platforms while also highlighting the influence of measurement, timing, signal processing, and hardware characteristics on the resulting behavior of the control system.

Obsah

Úvod	8
1 Podstata návrhu implementácie algoritmu riadenia	9
1.1 Analytické spracovanie riadiaceho systému	9
1.2 Zákon riadenia pre implementáciu	10
1.2.1 Pseudokód implementácie zákona riadenia	11
2 Riadený systém	12
2.1 Opis riadeného systému	12
2.1.1 Technický opis komponentov	12
2.1.2 Statické vlastnosti	13
2.2 Identifikácia modelu dynamického systému	13
2.2.1 Meranie a spracovanie dát	13
2.2.2 Identifikácia v zmysle metódy najmenších štvorcov	17
2.2.3 Identifikácia pomocou optimalizácie parametrov simulovaného dynamického systému	18
2.2.4 Výsledky identifikácie	20
2.2.5 Redukcia a zjednodušenie identifikovaných modelov	20
2.3 Komunikačné rozhranie riadeného systému	21
2.3.1 Pôvodná komunikácia v prostredí MATLAB/Simulink	22
2.3.2 Vlastná komunikácia v jazyku C++	23
2.3.3 Komunikácia s programovateľným logickým automatom	27
3 PI regulátor pre riadenie v okolí jedného pracovného bodu	32
3.1 Rozbor algoritmu riadenia	32
3.2 Nastavenie parametrov PI regulátora	33
3.3 Implementácia: MATLAB/Simulink	35
3.3.1 Overenie na reálnom systéme	37
3.4 Implementácia: mikrokontrolér a jazyk C++	38
3.4.1 Overenie na reálnom systéme	39
3.5 Implementácia: programovateľný logický automat (PLC)	40
3.5.1 Overenie na reálnom systéme	42
3.6 Porovnanie výsledkov z jednotlivých implementácií	43
4 Riadiaci systém s prepínaním pre celý rozsah výstupnej veličiny riadeného systému	44
4.1 Syntéza riadiaceho systému	44
4.1.1 Globálna regulácia po určitý čas	47
4.1.2 Prepínanie na základe detekcie ustálenia v okolí želaného pracovného bodu	49
4.2 Realizácia riadiaceho systému v prostredí MATLAB/Simulink	51
4.2.1 Simulačné overenie	53
4.2.2 Overenie na reálnom systéme	54
4.3 Realizácia riadiaceho systému v jazyku C++ pre mikrokontrolér	54
4.3.1 Overenie na reálnom systéme	61
4.4 Implementácia pre programovateľný logický kontrolér (PLC)	61
4.4.1 Overenie na reálnom systéme	66
4.5 Porovnanie výsledkov z jednotlivých implementácií	66
5 Adaptívne riadenie s referenčným modelom	71
5.1 Rozbor algoritmu riadenia	71
5.1.1 Prepis vstupno-výstupného opisu systému do stavového opisu	73
5.1.2 Stavový opis súčastí MRAC algoritmu	74
5.1.3 Pseudokód adaptívneho riadiaceho systému	76
5.2 Implementácia: MATLAB/Simulink	78

5.2.1	Overenie na reálnom systéme	82
5.3	Implementácia: mikrokontrolér a jazyk C++	83
5.3.1	Overenie na reálnom systéme	87
5.3.2	Heuristická modifikácia na potlačenie driftu adaptovaných parametrov	89
5.4	Implementácia: programovateľný logický automat (PLC)	93
5.4.1	Overenie na reálnom systéme	97
5.5	Porovnanie výsledkov z jednotlivých implementácií adaptívneho riadiaceho systému	97
Záver		100
Literatúra		101
Prílohy		102
P.1	Hlavný program pre mikrokontrolér platformy Arduino UNO	102
P.2	Externý skript pre logovanie dát	105

Úvod

Riadenie dynamických systémov patrí medzi základné úlohy automatizácie a kybernetiky. V technickej praxi však samotný návrh regulačného algoritmu nepredstavuje celý problém. Aby bolo možné algoritmus použiť na reálnom zariadení, je potrebné zabezpečiť jeho softvérovú implementáciu, komunikáciu s meracím a akčným hardvérom, správne časovanie výpočtu, spracovanie vstupných a výstupných signálov a rešpektovanie fyzikálnych obmedzení riadeného systému.

Táto diplomová práca sa zameriava najmä na praktickú implementáciu algoritmov riadenia pre vybraný laboratórny dynamický systém. V súlade so zadaním práce je hlavným cieľom zostavenie riadiaceho systému pre dostupný laboratórny hardvér vrátane vlastného softvérového návrhu. Súčasťou riešenia je aj stručná technická dokumentácia použitého laboratórneho zariadenia, keďže poznanie jeho vstupov, výstupov, fyzikálnych vlastností a spôsobu pripojenia je nevyhnutné pre návrh a realizáciu riadiacich algoritmov.

Práca zároveň využíva postupy z oblasti identifikácie systémov, numerickej simulácie a vyhodnocovania kvality riadenia. Pred samotným návrhom regulátorov je preto potrebné získať model správania riadeného systému na základe experimentálne nameraných údajov. Identifikovaný model následne slúži ako podklad pre numerickú simuláciu, návrh parametrov regulátorov a počiatočné overenie regulačných štruktúr pred ich nasadením na reálne zariadenie.

Riadiace algoritmy sú realizované na viacerých hardvérových a softvérových platformách. Uvažované platformy sa odlišujú spôsobom práce so stavovými premennými, spracovaním analógových signálov, časovaním výpočtu aj možnosťami záznamu údajov. Práve tieto rozdiely sú z praktického hľadiska dôležité pri prenose algoritmu z návrhového alebo simulačného prostredia na reálny hardvér.

Výber riadiacich algoritmov bol zvolený tak, aby pokrýval viacero úrovní náročnosti návrhu a implementácie. Prvým algoritmom je PI regulátor, ktorý predstavuje klasický a v praxi široko používaný prístup k riadeniu pomalších technologických sústav. Tento regulátor tvorí základný referenčný prípad, na ktorom je možné ukázať postup návrhu, diskretnej realizácie a implementácie na rôznych platformách.

Druhým algoritmom je PI regulácia s prepínaním pracovných bodov. Ide o vlastne navrhnutú riadiacu štruktúru, ktorej cieľom je rozšíriť použiteľný pracovný rozsah základného PI riadenia. Algoritmus kombinuje globálnu regulačnú vetvu, lokálnu regulačnú vetvu a prepínanie logiku založenú na vyhodnotení ustálenia systému. Tým vzniká prechod medzi jednoduchým lokálnym regulátorom a zložitejšou štruktúrou, ktorá zohľadňuje zmenu pracovného bodu.

Tretím algoritmom je adaptívna regulácia typu MRAC (*Model Reference Adaptive Control*) založená na gradientnej metóde. Tento prístup bol zvolený ako reprezentant pokročilejšej regulačnej metódy, pri ktorej sa parametre regulátora menia počas prevádzky. Zároveň ide o algoritmus citlivejší na praktické podmienky merania a implementácie, napríklad na merací šum, numerickú deriváciu, voľbu adaptačných koeficientov a vlastností konkrétnej hardvérovej platformy.

Štruktúra práce zodpovedá jednotlivým úlohám zadania. Najskôr je uvedený úvodný príklad návrhu regulácie a opis použitého laboratórneho systému. Následne je spracovaná identifikácia systému a prehľad vybraných riadiacich algoritmov. Ďalšie kapitoly sa venujú návrhu, realizácii a experimentálnemu overeniu jednotlivých riadiacich systémov na dostupných platformách. V závere práce sú vyhodnotené dosiahnuté výsledky a zhrnuté praktické poznatky získané pri implementácii algoritmov na reálnom laboratórnom zariadení.

1 Podstata návrhu implementácie algoritmu riadenia

V tejto časti je uvedený stručný ilustračný príklad návrhu implementácie regulátora. Zvolený je štandardný a všeobecne známy PID (Proporcionálny, Integrovaný, Derivačný) regulátor. Predpokladá sa, že máme k dispozícii identifikovaný model riadeného systému, prípadne uvažujeme, že je k dispozícii analytický návrh regulátora pre daný reálny riadený systém. V oboch prípadoch budeme riadený systém ďalej označovať ako $G_s(s)$. Cieľom je navrhnúť, teda presne stanoviť, ako bude daný regulátor implementovaný, ako konkrétne bude vypočítaný akčný zásah.

1.1 Analytické spracovanie riadiaceho systému

Samotná implementácia algoritmu riadenia často musí zohľadňovať rôzne aspekty vyplývajúce z jeho teoretického návrhu a syntézy. Pre ilustráciu v tejto časti uvádzame náčrt niektorých teoretických poznatkov, ktoré sú relevantné pre návrh implementácie v tomto prípade PID regulátora.

Prenosová funkcia PID regulátora, označme $C(s)$, v paralelnej forme je definovaná ako

$$C(s) = K_p + \frac{K_i}{s} + K_d s, \quad (1.1)$$

kde K_p predstavuje proporcionálne zosilnenie, K_i integračné zosilnenie a K_d derivačné zosilnenie regulátora.

Na reguláciu budeme uvažovať štandardnú zápornú spätnú väzbu. Regulačná odchýlka vzniká ako rozdiel medzi požadovanou hodnotou a výstupom systému,

$$e(t) = w(t) - y(t), \quad (1.2)$$

pričom $w(t)$ je žiadaná hodnota (setpoint) a $y(t)$ je výstup regulovanej sústavy. Výstup regulátora predstavuje akčný zásah $u(t)$, ktorý vstupuje do systému $G_s(s)$.

Pre diskretnú implementáciu s periódou vzorkovania T_s budeme používať zápis

$$e(k) = w(k) - y(k), \quad (1.3)$$

kde index k označuje diskretný časový krok.

Uzavretý regulačný obvod

Bloková schéma uzavretého regulačného obvodu je znázornená na obrázku 1.

Pre ilustráciu predpokladajme, že riadený systém má prenosovú funkciu s čitateľom nultého rádu a menovateľom druhého rádu,

$$G_s(s) = \frac{b_0}{s^2 + a_1 s + a_0}. \quad (1.4)$$

Prenosová funkcia otvoreného regulačného obvodu (ORO) je daná súčinom systému a regulátora,

$$G_{\text{ORO}}(s) = G_s(s) C(s). \quad (1.5)$$

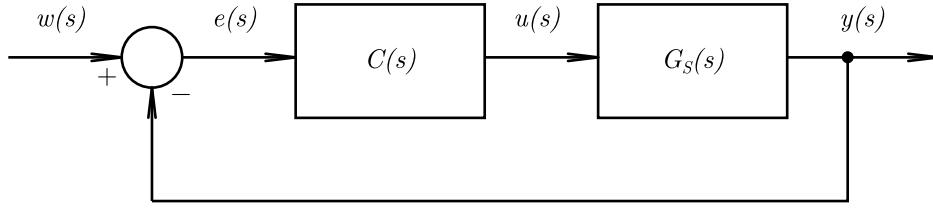
Po dosadení (1.1) a (1.4) dostaneme

$$\begin{aligned} G_{\text{ORO}}(s) &= \frac{b_0}{s^2 + a_1 s + a_0} \left(K_p + \frac{K_i}{s} + K_d s \right) \\ &= \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0)}. \end{aligned} \quad (1.6)$$

Prenosová funkcia uzavretého regulačného obvodu (URO) pri jednotkovej spätnej väzbe je

$$G_{\text{URO}}(s) = \frac{G_{\text{ORO}}(s)}{1 + G_{\text{ORO}}(s)}. \quad (1.7)$$

Uvedený tvar vychádza zo štandardného opisu uzavretej spätnej väzby v klasickej teórii riadenia [8].



Obr. 1: Bloková schéma uzavretého regulačného obvodu so zápornou spätnou väzbou a PID regulátorom

Po dosadení (1.6) platí

$$G_{\text{URO}}(s) = \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i)}. \quad (1.8)$$

Poloha pólov a stabilita

Póly ORO sú dané koreňmi menovateľa prenosu $G_{\text{ORO}}(s)$, t. j.

$$s (s^2 + a_1 s + a_0) = 0, \quad (1.9)$$

odkiaľ priamo vyplýva jeden pól v $s_1 = 0$ a zvyšné dva póly sú koreňmi kvadratickej rovnice

$$s^2 + a_1 s + a_0 = 0. \quad (1.10)$$

Póly URO sú určené koreňmi charakteristického polynómu (menovateľa) uzavretého obvodu,

$$s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i) = 0, \quad (1.11)$$

pričom po roznásobení dostaneme explicitný tvar tretieho rádu

$$s^3 + (a_1 + b_0 K_d) s^2 + (a_0 + b_0 K_p) s + b_0 K_i = 0. \quad (1.12)$$

Poloha pólov je v oboch prípadoch kľúčová, avšak význam sa líši. V prípade ORO sa od polohy pólov odvídzajú následné vlastnosti uzavretej regulačnej slučky, najmä z hľadiska robustnosti a bezpečnosti návrhu regulátora. V prípade URO je stabilita nevyhnutná podmienka správnej funkcie regulácie. Pre spojitý systém stabilita znamená, že všetky póly $G_{\text{URO}}(s)$ majú zápornú reálnu časť [7]. V opačnom prípade výstup $y(t)$ diverguje, regulačná odchýlka neklesá a systém môže byť v reálnej prevádzke nebezpečný.

1.2 Zákon riadenia pre implementáciu

V časovej oblasti je PID zákon riadenia možné zapísať ako

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (1.13)$$

Pre praktickú implementáciu v diskretnom čase budeme uvažovať základnú aproximáciu:

$$I(k) = I(k-1) + T_s e(k), \quad (1.14)$$

$$D(k) = \frac{e(k) - e(k-1)}{T_s}, \quad (1.15)$$

$$u(k) = K_p e(k) + K_i I(k) + K_d D(k). \quad (1.16)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu sa v praxi uplatňuje saturácia

$$u(k) = \min(\max(u(k), u_{\min}), u_{\max}). \quad (1.17)$$

1.2.1 Pseudokód implementácie zákona riadenia

Zákon riadenia uvedený v časti 1.2 je možné implementovať v rôznych programovacích jazykoch a prostrediach. Nižšie (výpis kódu 1) je uvedený ilustračný pseudokód výpočtu PID regulátora v diskretnom čase s vnútornými stavmi integrátora a pamäťou predchádzajúcej regulačnej odchýlky.

```
% Inputs: w (setpoint), y (output), Ts
% States: I (integrator state), e_prev (previous error)
% Params: Kp, Ki, Kd, u_min, u_max

e = w - y;

% Integrator update
I = I + Ts * e;

% Derivative term
D = (e - e_prev) / Ts;

% PID law
u = Kp * e + Ki * I + Kd * D;

% Saturation
u = min(max(u, u_min), u_max);

% Memory update
e_prev = e;
```

Výpis kódu 1: Pseudokód diskkrétnej implementácie PID regulátora (ilustračný príklad)

2 Riadený systém

V tejto kapitole sa budeme zaoberať laboratórnym modelom, ktorý bol poskytnutý pre potreby tejto diplomovej práce – tepelným systémom TS05. Cieľom kapitoly je podrobne opísať túto experimentálnu zostavu, vykonať jej identifikáciu a navrhnúť viaceré riešenia riadenia, a to ako klasické, tak aj pokročilejšie metódy.

Súčasťou práce je tiež implementácia vybraných riadiacich algoritmov na rôznych hardvérových platformách. Táto časť má umožniť porovnanie prístupov k implementácii riadenia v simulačnom prostredí MATLAB/Simulink a na vybranom mikrokontroléri, čím sa vytvorí priestor pre analýzu rozdielov v správaní systému a v samotnej realizácii algoritmov.

2.1 Opis riadeného systému

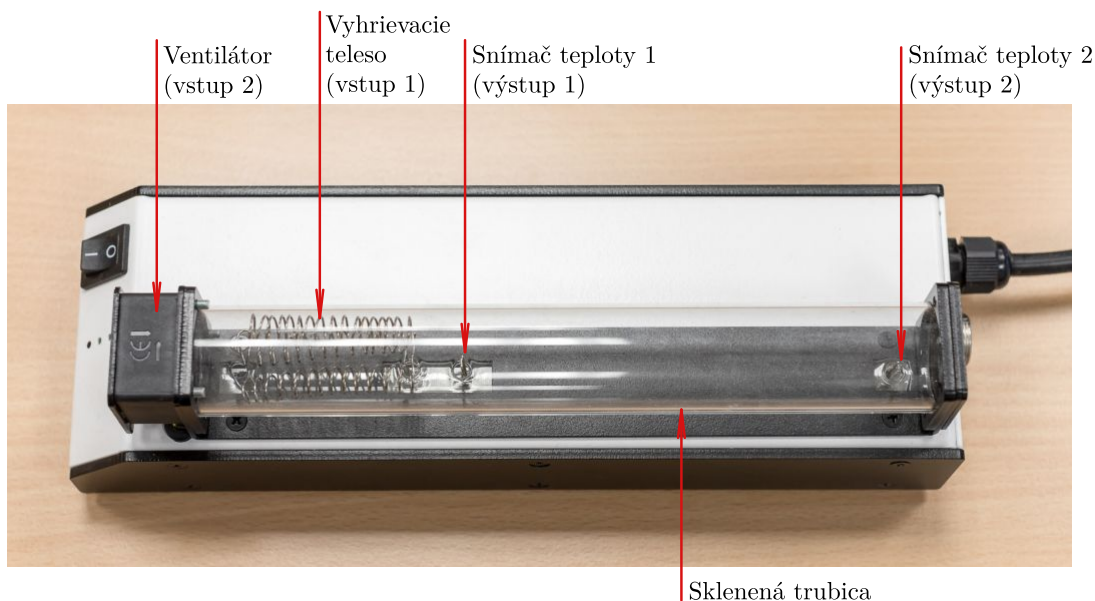
Táto časť sa zaoberá popisom laboratórneho modelu TS05 a jeho základnými statickými vlastnosťami. Model predstavuje fyzický systém určený na výučbu a experimentálne overovanie princípov riadenia v oblasti automatizácie a mechatroniky.

2.1.1 Technický opis komponentov

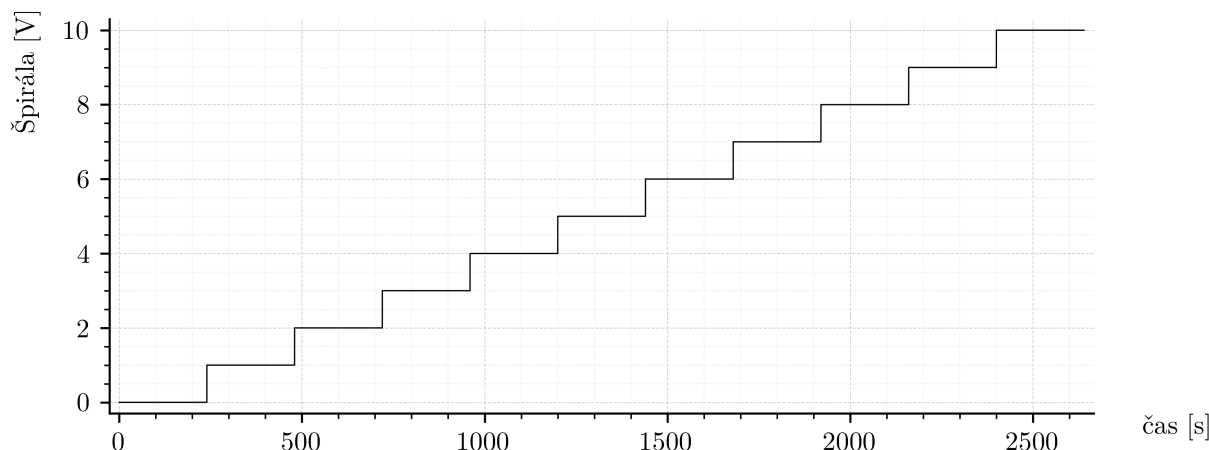
Laboratórny model TS05 predstavuje MIMO systém (Multiple Input – Multiple Output) so dvomi vstupmi a dvomi výstupmi, ktorého zapojenie je uvedené v tabuľke 1. Vstupné veličiny tvoria napätia riadiace ventilátor a vykurovaciu špirálu (v rozsahu 0–10 V), zatiaľ čo výstupy sú merané pomocou Senzora 1 a Senzora 2 (tiež v rozsahu 0–10 V). Fotografia zariadenia je na obrázku 2.

Tabuľka 1: Vstupy a výstupy systému TS05

Typ signálu	Označenie	Rozsah
Vstup	Ventilátor	0–10 V
Vstup	Výkon špirály	0–10 V
Výstup	Senzor 1 (teplota 1)	0–10 V
Výstup	Senzor 2 (teplota 2)	0–10 V



Obr. 2: Laboratórny model TS05 – reálna zostava [2]



Obr. 3: Pribeh napätia na špirále počas všetkých troch experimentov

V ďalšom texte tejto sekcie bude systém TS05 v prostredí MATLAB/Simulink používaný prostredníctvom komunikačného rozhrania s meracou kartou. Podrobnejší popis tejto komunikácie a použitých Simulinkových schém je uvedený v podkapitole 2.3.1.

2.1.2 Statické vlastnosti

Na lepšie pochopenie správania systému boli experimentálne namerané jeho statické charakteristiky. Počas meraní boli skúmané tri pracovné body ventilátora: 3 V, 6 V a 9 V, pričom v každom prípade bola hodnota napätia ventilátora udržiavaná konštantná. Napätie na vykurovacej špirále sa menilo lineárne od 0 V do 10 V počas celého merania, keďže špirála má výraznejší vplyv na teplotu ako ventilátor. Pribeh signálu špirály počas experimentov je zobrazený na obrázku 3.

Výsledkom meraní je šesť grafov (obrázky 4–6), ktoré zobrazujú odozvu systému na zmenu vstupných veličín. Každý graf obsahuje výstupy zo Senzora 1 a Senzora 2 ako funkcie času pre jednotlivé pracovné body ventilátora.

Z tých istých meraní boli zostavené aj prevodové (statické) charakteristiky, ktoré opisujú závislosť výstupných veličín od vstupného napätia špirály pre jednotlivé pracovné body ventilátora. Sú znázornené na obrázkoch 7–9.

2.2 Identifikácia modelu dynamického systému

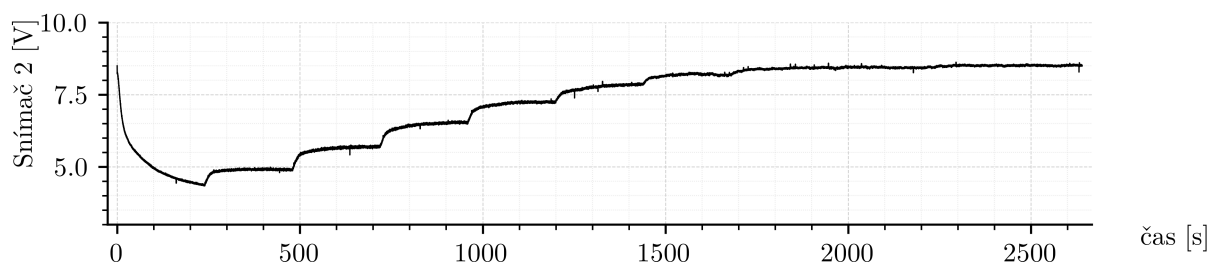
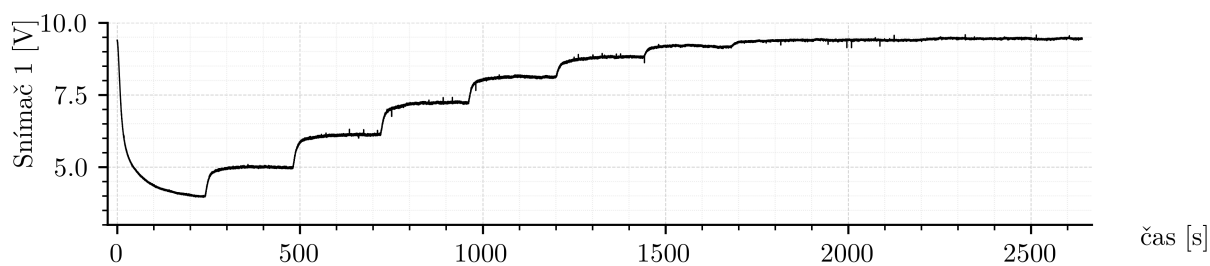
V tejto časti sa pokúsime identifikovať tepelný systém TS05 rôznymi metódami a porovnať ich výsledky. Cieľom je určiť, ktoré prístupy poskytujú presnejšie modely a na aké faktory je potrebné dávať pozor počas procesu identifikácie.

2.2.1 Meranie a spracovanie dát

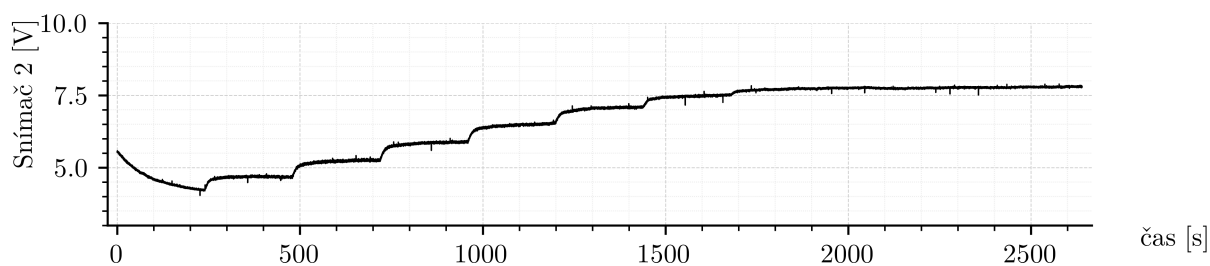
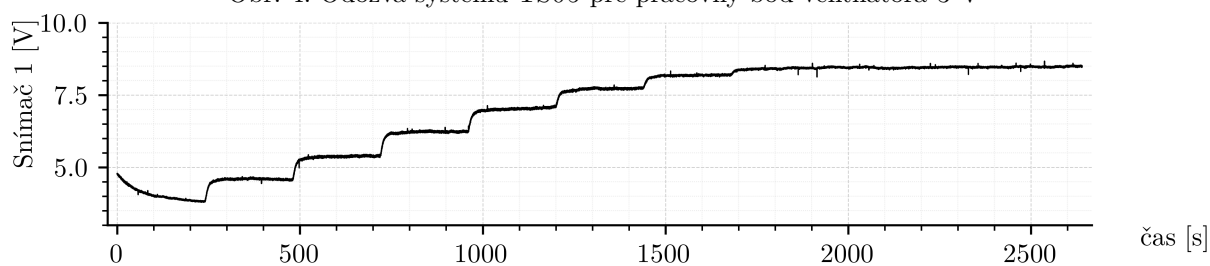
Na účely identifikácie systému pomocou nami zvolených *off-line* metód je najprv potrebné získať reakciu systému na jednotkové skoky v okolí pracovného bodu. V našom prípade bol pracovný bod zvolený ako vstupné napätia špirály 4 V a ventilátora 6 V.

Na obrázku 10 je zobrazený vstupný signál špirály a odpoveď systému na výstupe zo senzora 1.

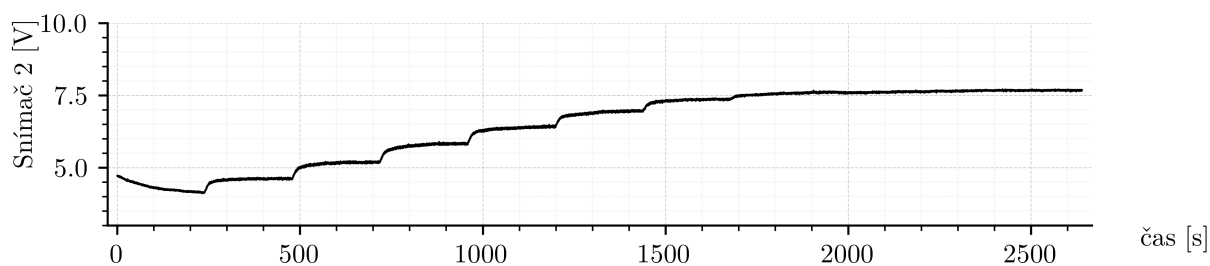
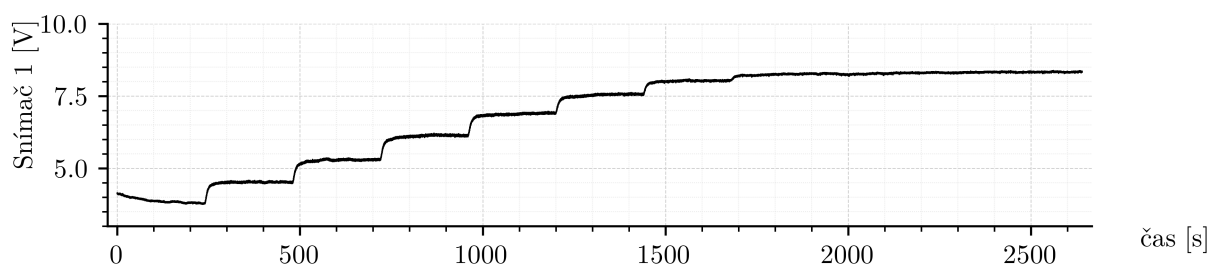
Z grafu možno pozorovať zreteľný lineárny trend, teda postupné zvyšovanie výstupnej teploty počas celého merania. Tento jav je spôsobený pomalým nahrievaním špirály a tiež prirodzenými zmenami teploty v miestnosti, ktoré neboli v rámci experimentu kontrolované. Teplota prostredia sa však počas meraní pohybovala v rozsahu bežnej izbovej teploty.



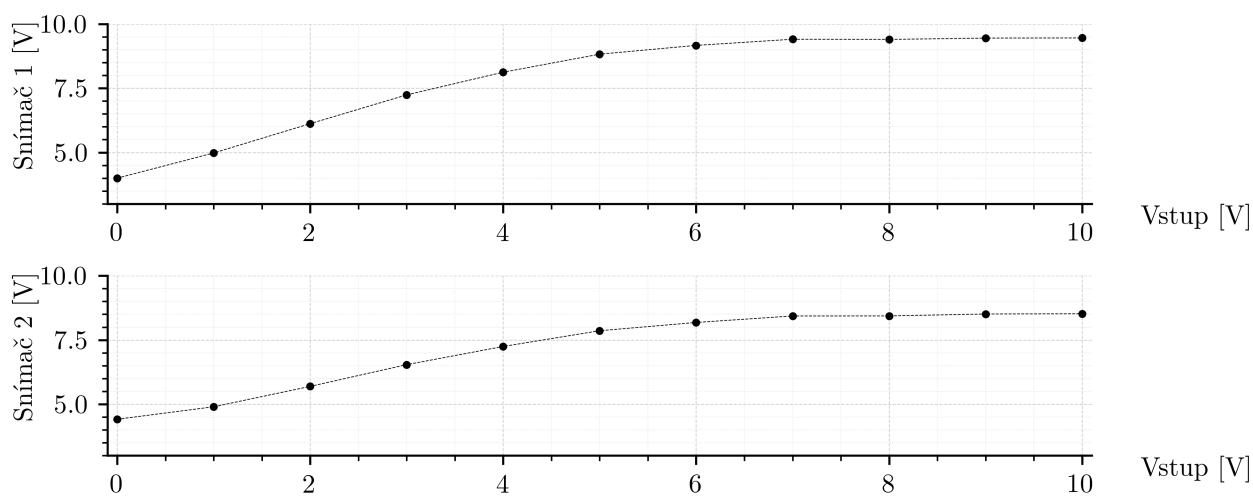
Obr. 4: Odozva systému TS05 pre pracovný bod ventilátora 3 V



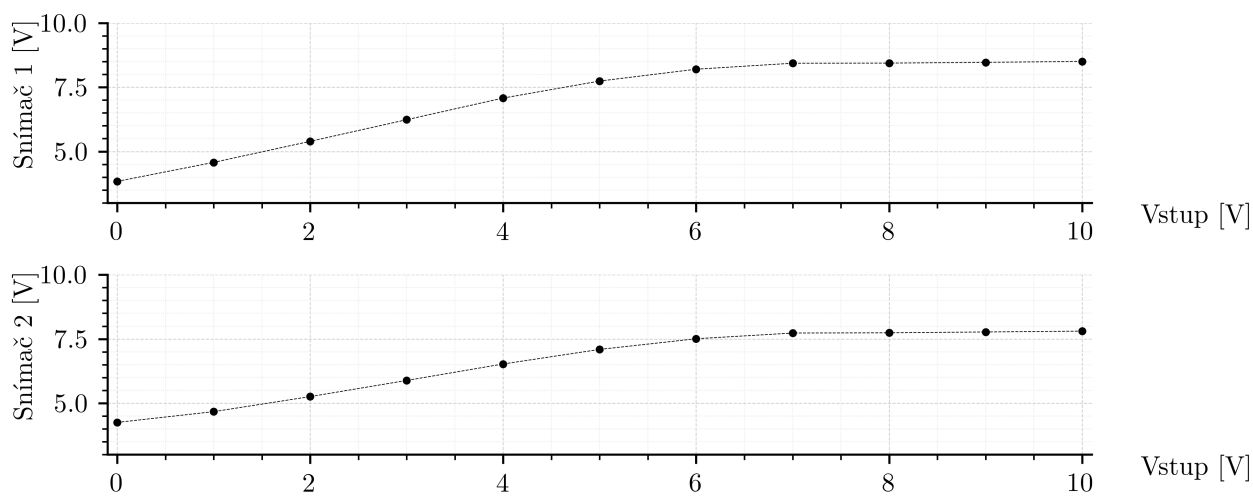
Obr. 5: Odozva systému TS05 pre pracovný bod ventilátora 6 V



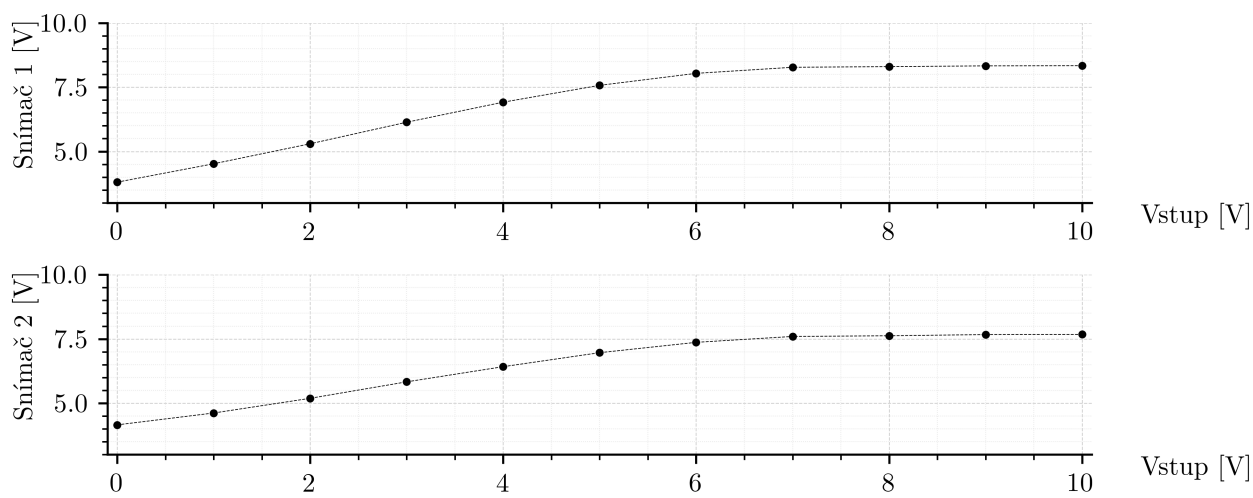
Obr. 6: Odozva systému TS05 pre pracovný bod ventilátora 9 V



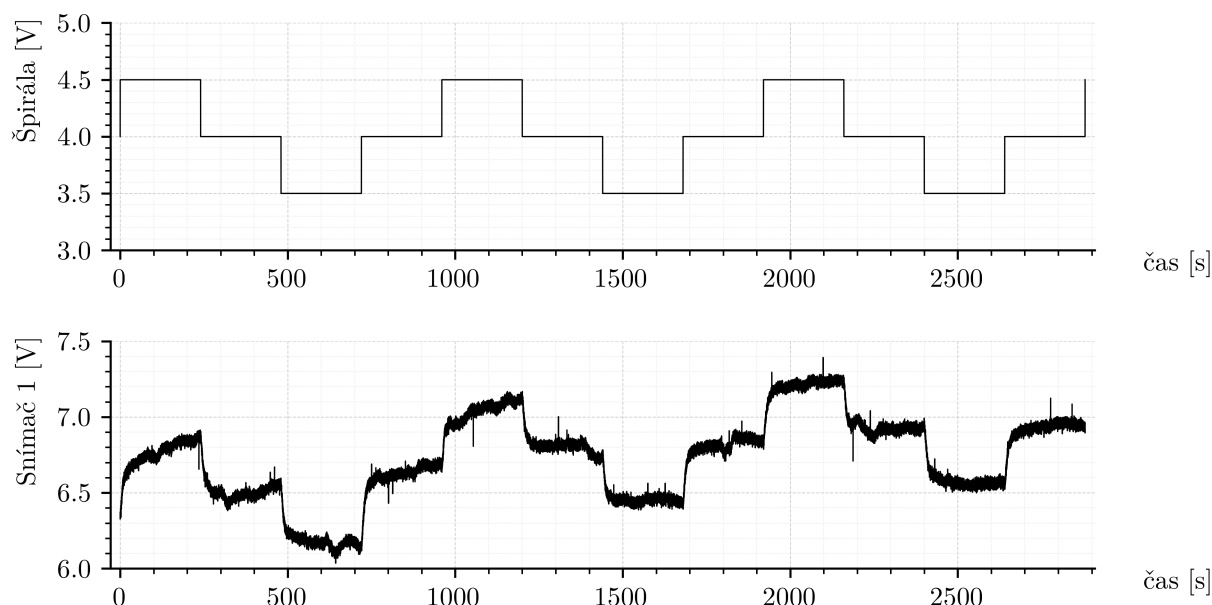
Obr. 7: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 3 V



Obr. 8: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 6 V



Obr. 9: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 9 V



Obr. 10: Vstupný signál špirály a výstup senzora 1 pri skokovej zmene

Aby bolo možné získať z nameraných údajov vhodné parametre pre identifikáciu, bolo potrebné odstrániť tento trend aplikovaním odstránenia lineárneho trendu. Metodika výpočtu je založená na lineárnej regresii, kde sa určuje priamka trendu v tvare

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x \quad (2.1)$$

pričom koeficienty sa počítajú podľa vzťahov [3]:

$$\hat{\beta}_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (2.2)$$

Takto určený trend je následne odčítaný od pôvodného signálu:

$$y_{\text{detr}} = y - (\hat{\beta}_0 + \hat{\beta}_1 x) \quad (2.3)$$

Uvedený postup bol implementovaný v prostredí MATLAB pomocou vlastnej funkcie:

```

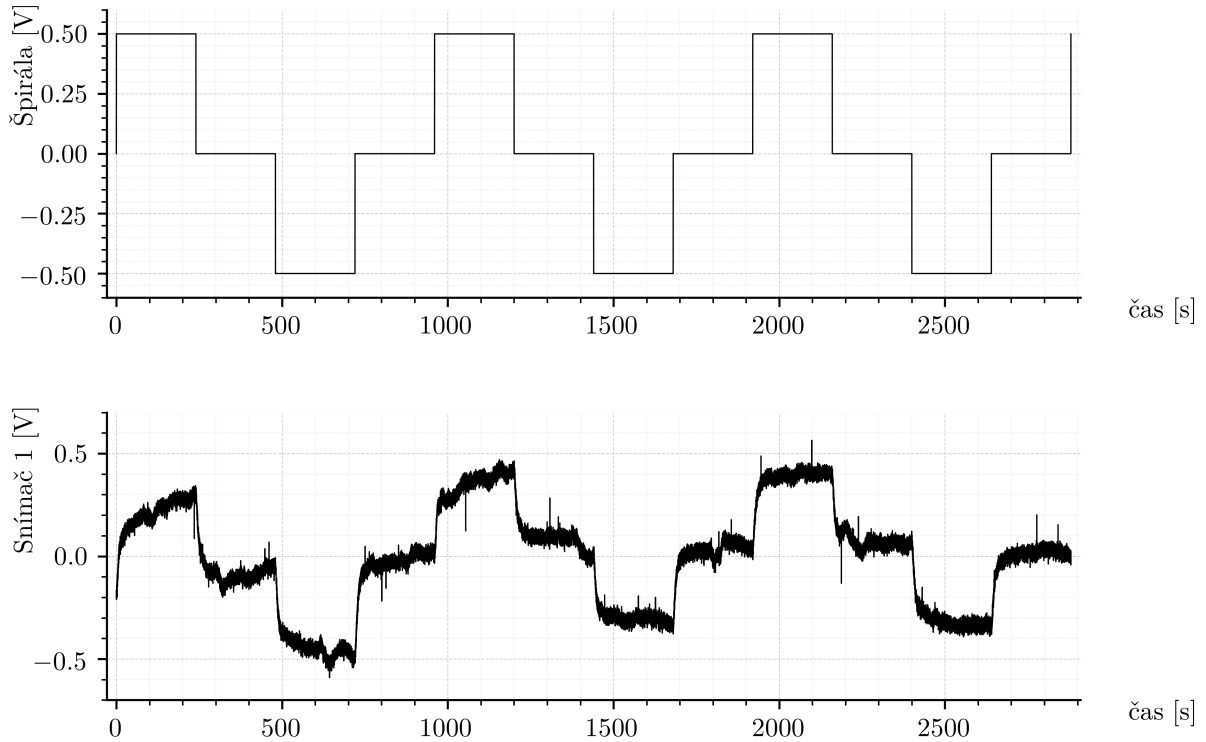
1 function y_detr = my_detrrend(y, x)
2     y = y(:);
3     x = x(:);
4     n = length(y);
5
6     Sx = sum(x);
7     Sy = sum(y);
8     Sxx = sum(x.^2);
9     Sxy = sum(x .* y);
10
11     b1 = (n*Sxy - Sx*Sy) / (n*Sxx - Sx^2);
12     b0 = mean(y) - b1 * mean(x);
13
14     y_detr = y - (b0 + b1*x);
15 end

```

Výpis kódu 2: MATLAB implementácia detrendácie signálu pomocou lineárnej regresie

Po odstránení lineárneho trendu bol signál zároveň normalizovaný pre potreby ďalšej identifikácie. Výsledné priebehy normalizovaných a detrendovaných údajov sú zobrazené na obrázku 11.

Z grafov možno vidieť, že jednotlivé skokové odozvy sa mierne líšia svojím priebehom, čo poukazuje na citlivosť systému TS05 na vonkajšie podmienky



Obr. 11: Normalizované a detrendované priebehy vstupného a výstupného signálu

a fluktuácie prostredia. Preto boli vykonané tri nezávislé experimenty s rovnakým vstupným signálom, ktorých priebehy sú zobrazené na obrázku 11. Po detrendácii a normalizácii boli tieto dáta spriemerované bod po bode, aby sa eliminoval vplyv náhodných rušení a získala reprezentatívna odozva systému.

Výsledný stredný priebeh, znázornený na obrázku 12, predstavuje typickú reakciu systému TS05 v okolí pracovného bodu 4 V s rozsahom $\pm 0,5$ V a slúži ako podklad pre ďalšiu identifikáciu modelu.

2.2.2 Identifikácia v zmysle metódy najmenších štvorcov

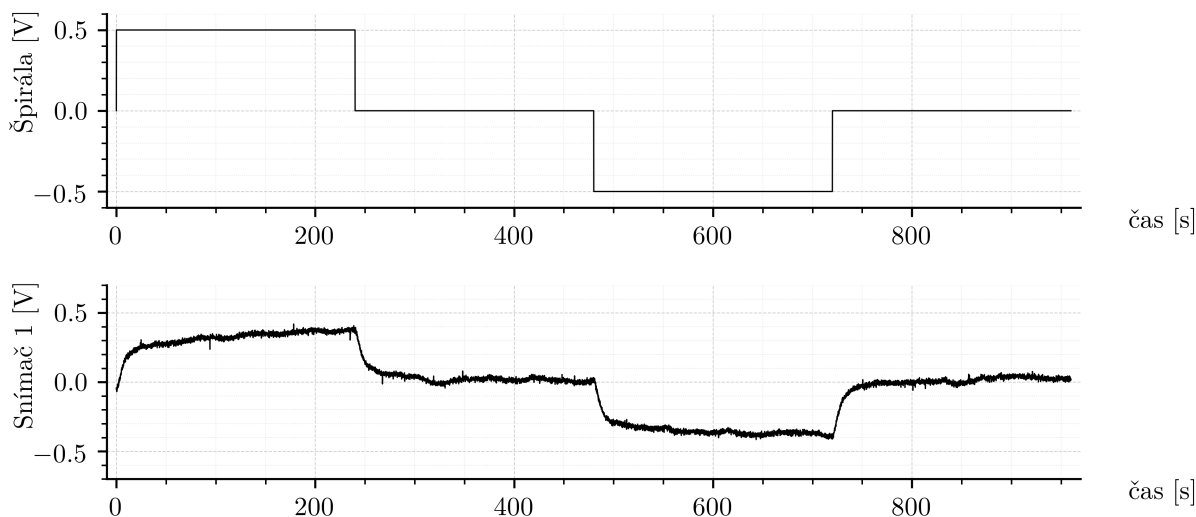
V tejto časti bola uskutočnená identifikácia systému TS05 na základe priemernej odozvy získanej z troch experimentálnych meraní popísaných v predchádzajúcej kapitole. Tento priemerný priebeh reprezentuje typickú reakciu systému v okolí pracovného bodu 4 V a slúži ako základ pre odhad parametrov modelu. Cieľom bolo získať lineárny model druhého rádu, ktorý dostatočne presne opisuje dynamické správanie systému v tejto oblasti.

Identifikácia bola realizovaná pomocou metódy najmenších štvorcov [1], ktorá umožňuje odhad parametrov diskrétného modelu systému na základe meraní vstupov a výstupov. Pre odhad parametrov bola vytvorená regresná matica $H(k, :)$ a vektor výstupov Y , pričom výsledný odhad parametrov $\hat{\theta}$ bol určený podľa rovníc:

$$H(k, :) = [-y(k-1), -y(k-2), u(k-1), u(k-2)] \quad (2.4)$$

$$\hat{\theta} = (H^T H)^{-1} H^T Y \quad (2.5)$$

Na základe týchto odhadov boli vypočítané koeficienty diskrétného modelu druhého rádu, ktorý následne predstavuje prenosovú funkciu $G_d(z)$. Pre účely simulácie a ďalšieho návrhu riadenia bol tento model prevedený na spojitú formu $G(s)$ pomocou Tustinovej aproximácie.



Obr. 12: Priemerná odozva systému TS05 v pracovnom bode 4 V

V prostredí MATLAB bola implementovaná funkcia, ktorá realizuje uvedené výpočty a vytvára identifikovaný model systému. Implementácia tejto funkcie je uvedená nižšie:

```

1 function G = ls_identify_basic(y, u, Ts)
2
3     y = y(:);
4     u = u(:);
5     N = length(y);
6     na = 2; nb = 2;
7
8     H = zeros(N-2, na+nb);
9     for k = 3:N
10        H(k-2,:) = [-y(k-1), -y(k-2), u(k-1), u(k-2)];
11     end
12     Y = y(3:N);
13
14     theta = pinv(H) * Y;
15
16     a1 = theta(1);
17     a2 = theta(2);
18     b1 = theta(3);
19     b2 = theta(4);
20
21     num = [b1 b2];
22     den = [1 a1 a2];
23     Gd = tf(num, den, Ts);
24
25     G = d2c(Gd, 'tustin');
26
27 end

```

Výpis kódu 3: MATLAB implementácia identifikácie metódou najmenších štvorcov

2.2.3 Identifikácia pomocou optimalizácie parametrov simulovaného dynamického systému

V tejto časti bola na identifikáciu systému TS05 zvolená numerická optimalizačná metóda. Namiesto priameho odhadu parametrov modelu metódou najmenších štvorcov bola použitá optimalizácia parametrov prenosovej funkcie tak, aby sa minimalizovala chyba medzi simulovanou odozvou modelu a reálnou priemernou odozvou systému, získanou z experimentov.

Ako nástroj optimalizácie bola použitá funkcia `fminsearch` z prostredia MATLAB, ktorá implementuje Nelderovu–Meadovu simplexovú metódu [9]. Ide o bezderivačnú lokálnu optimalizačnú metódu, ktorá na základe hodnotenia účelovej funkcie

v niekoľkých bodoch parametrov iteratívne posúva simplex (mnohouholník v priestore parametrov) tak, aby sa znižovala hodnota účelovej funkcie. V našom prípade je účelová funkcia definovaná ako koreň zo strednej kvadratickej chyby (RMSE) medzi nameranou odozvou systému a simulovaným výstupom modelu:

$$J(\mathbf{p}) = \sqrt{\frac{1}{N} \sum_{k=1}^N (y(k) - y_{\text{sim}}(k, \mathbf{p}))^2} \quad (2.6)$$

kde $y(k)$ je nameraný výstup, $y_{\text{sim}}(k, \mathbf{p})$ je výstup simulovaný modelom s parametrami $\mathbf{p}$ a N je počet vzoriek merania.

Optimalizácia prebieha nad parametrami diskrétného modelu druhého rádu, ktoré sú následne prevedené na spojitú prenosovú funkciu pomocou Tustinovej aproximácie. Implementácia použitej funkcie v prostredí MATLAB je uvedená nižšie:

```

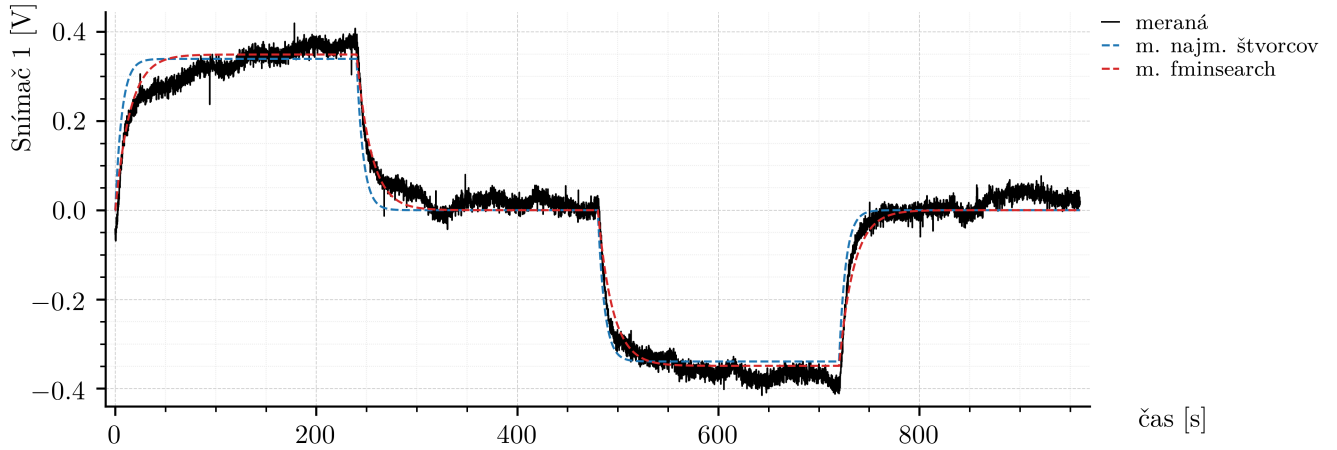
1 function G = tf_identify_fmin(y, u, time, Ts)
2
3     y = y(:);
4     u = u(:);
5     time = time(:);
6
7     p0 = [0 0 0 0];
8
9     cost_fun = @(p) rmse_cost_stable(p, u, y, time, Ts);
10
11     opts = optimset('Display', 'off', 'TolX', 1e-8, 'TolFun', 1e-8);
12     p_opt = fminsearch(cost_fun, p0, opts);
13
14     p_opt = 50 * tanh(p_opt);
15     a1 = p_opt(1); a2 = p_opt(2); b1 = p_opt(3); b2 = p_opt(4);
16
17     Gd = tf([b1 b2], [1 a1 a2], Ts);
18     G = d2c(Gd, 'tustin');
19
20 end
21
22 function err = rmse_cost_stable(p, u, y, t, Ts)
23
24     p = 50 * tanh(p);
25     a1 = p(1); a2 = p(2); b1 = p(3); b2 = p(4);
26
27     try
28
29         Gd = tf([b1 b2], [1 a1 a2], Ts);
30
31         poles = pole(Gd);
32         if any(abs(poles) >= 1)
33             err = 1e6;
34             return;
35         end
36
37         G = d2c(Gd, 'tustin');
38         y_sim = lsim(G, u, t);
39
40         err = sqrt(mean((y - y_sim).^2));
41
42     catch
43         err = 1e6;
44     end
45 end

```

Výpis kódu 4: MATLAB implementácia identifikácie pomocou optimalizácie prenosovej funkcie

Pri tomto prístupe je dôležité upozorniť na jeden podstatný problém. Ak nie sú parametre prenosovej funkcie počas optimalizácie vhodne obmedzené, algoritmus môže nájsť také hodnoty parametrov, ktoré síce veľmi dobre aproximujú konkrétnu meranú odozvu, ale vedú k nerealistickému alebo nestabilnému správaniu modelu pri iných vstupoch. V predchádzajúcich pokusoch viedlo práve toto k prenosovým funkciám s numericky nevhodnými parametrami, ktoré boli použiteľné iba pre konkrétny fitovaný experiment. Z tohto dôvodu sú v uvedenej implementácii parametre počas optimalizácie mapované pomocou transformácie

$$p_i^* = 50 \tanh(p_i) \quad (2.7)$$



Obr. 13: Porovnanie skokovej odozvy reálneho systému TS05 a identifikovaných modelov

čím sú efektívne udržiavané v konečnom rozsahu a znižuje sa riziko vzniku numericky extrémnych alebo fyzikálne nezmyselných modelov. Okrem toho bola do optimalizácie pridaná aj dodatočná penalizácia, ktorá výrazne zvyšuje hodnotu účelovej funkcie v prípade, že identifikovaná prenosová funkcia vykazuje známky nestability. Týmto spôsobom algoritmus uprednostňuje iba stabilné riešenia a zabráňuje vzniku modelov s numericky nevhodnými pólmi.

2.2.4 Výsledky identifikácie

Po identifikácii systému v pracovnom bode pomocou dvoch rôznych metód (metódy najmenších štvorcov a optimalizácie prenosovej funkcie) boli získané výsledky zobrazené na obrázku 13.

Porovnanie priemernej skokovej odozvy reálneho systému a odoziev oboch identifikovaných modelov je zobrazené na obrázku 13.

Identifikovaný model získaný metódou najmenších štvorcov má tvar

$$G_{s,LS}(s) = \frac{0.00171 s^2 - 0.3379 s + 6.074}{s^2 + 59.01 s + 8.95} \quad (2.8)$$

a model identifikovaný pomocou optimalizácie parametrov prenosovej funkcie metódou *fminsearch* je

$$G_{s,fmin}(s) = \frac{0.0007181 s^2 - 0.05942 s + 0.9012}{s^2 + 18.94 s + 1.291} \quad (2.9)$$

Miera zhody medzi modelmi a experimentálnymi dátami bola hodnotená pomocou ukazovateľa RMSE:

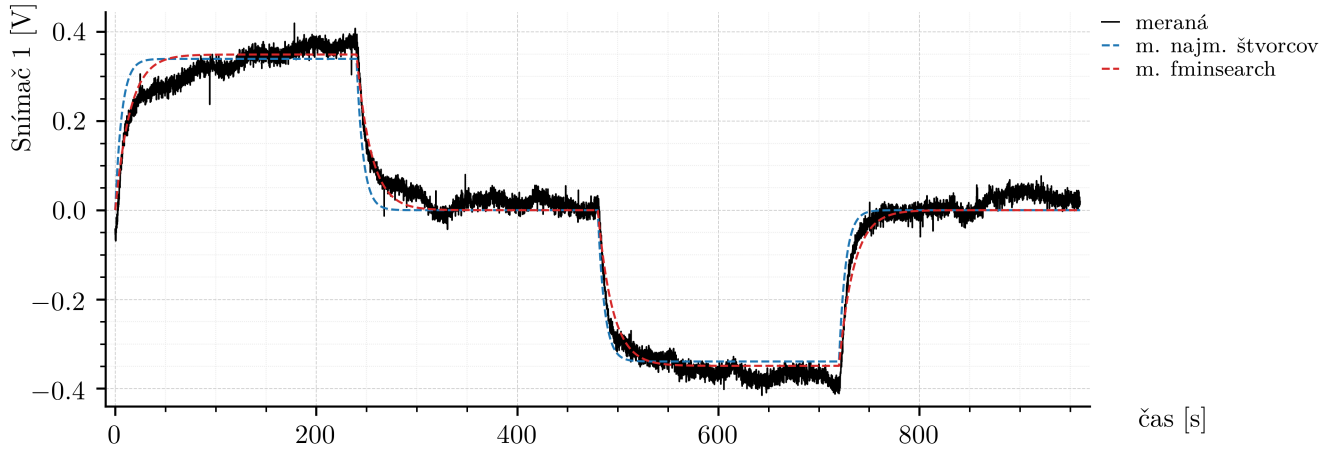
$$RMSE_{LS} = 0.036792, \quad (2.10)$$

$$RMSE_{fmin} = 0.026996 \quad (2.11)$$

Ako je vidieť, oba modely poskytujú veľmi uspokojivú aproximáciu dynamiky systému v okolí zvoleného pracovného bodu, pričom model získaný metódou *fminsearch* dosahuje o niečo menšiu hodnotu RMSE. Treba však zdôrazniť, že oba identifikované modely sú len druhého rádu, čo predstavuje rozumný kompromis medzi jednoduchosťou a presnosťou pre následný návrh regulácie.

2.2.5 Redukcia a zjednodušenie identifikovaných modelov

Na základe získaných prenosových funkcií možno pozorovať, že ich čitatele obsahujú veľmi malé koeficienty pri prvom a druhom rade. Možno ich interpretovať ako numerické artefakty vzniknuté pri prevode z diskkrétnej prenosovej funkcie na spojitú formu.



Obr. 14: Porovnanie skokovej odozvy reálneho systému TS05 a aproximovaných modelov

Zahrnutie týchto vyšších rádov v čitateli by výrazne komplikovalo niektoré analytické metódy návrhu regulátorov, napríklad metódu umiestnenia pólov.

Pre účely ďalšej práce sme sa preto rozhodli tieto prenosové funkcie aproximovať jednoduchšou formou. Najzrejmější prístup spočíva v tom, že malé koeficienty pri s a s^2 v čitateli zanedbáme a overíme vplyv tejto úpravy pomocou simulácie.

Výsledkom tejto aproximácie sú nasledujúce prenosové funkcie:

$$G_{s,LS,approx}(s) = \frac{6.074}{s^2 + 59.01s + 8.95} \quad (2.12)$$

póly:

$$s_1 = -58.8550, \quad s_2 = -0.1521 \quad (2.13)$$

a

$$G_{s,fmin,approx}(s) = \frac{0.9012}{s^2 + 18.94s + 1.291} \quad (2.14)$$

póly:

$$s_1 = -18.8736, \quad s_2 = -0.0684 \quad (2.15)$$

Z uvedených hodnôt pólov je zrejmé, že obidva modely majú póly s negatívnymi reálnymi časťami, teda ide o stabilné systémy.

Následne bola uskutočnená simulácia odozvy systému analogická tej, ktorá bola uvedená na obrázku 13. Porovnanie reálnej odozvy systému TS05 s oboma aproximovanými modelmi je zobrazené na obrázku 14.

Výsledné hodnoty strednej kvadratickej chyby (RMSE) medzi reálnym systémom a simulovanými odozvami boli:

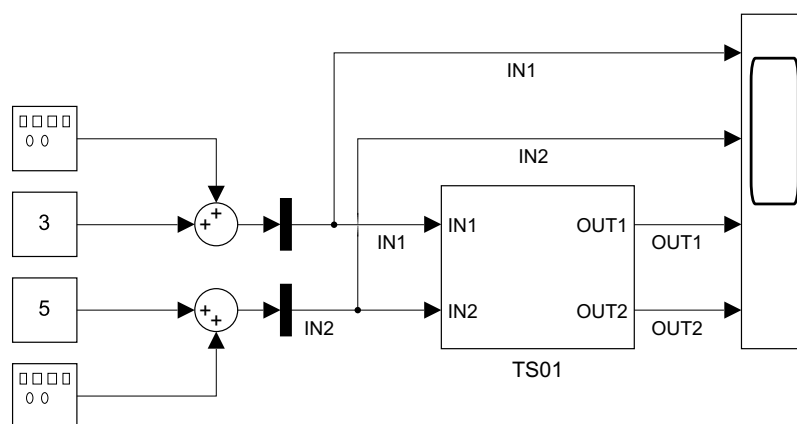
$$RMSE_{LS,approx} = 0.036951, \quad (2.16)$$

$$RMSE_{fmin,approx} = 0.026998 \quad (2.17)$$

Z výsledkov, ako aj z priebehov simulácií, je zrejmé, že rozdiel medzi pôvodnými a aproximovanými modelmi je zanedbateľný. Hodnota RMSE sa zhoršila len minimálne, čo je z pohľadu ďalšieho použitia modelov úplne prijateľné, najmä vzhľadom na zjednodušenie analytického spracovania, ktoré táto aproximácia prináša.

2.3 Komunikačné rozhranie riadeného systému

V tejto časti je opísaný spôsob komunikácie laboratórneho tepelného systému TS05 jednotlivými hardvérovými a softvérovými platformami použitými v práci. Cieľom bolo zabezpečiť jednotný a funkčný prístup k vstupom a výstupom systému nielen



Obr. 15: Základná Simulinková schéma pre spustenie komunikácie so systémom TS05

v prostredí MATLAB/Simulink, ale aj pri implementácii na platforme Arduino UNO v jazyku C++ a na priemyselnom PLC systéme Siemens.

Osobitná pozornosť bola venovaná najmä tomu, aby bolo možné realizovať experimenty aj mimo pôvodného simulačného prostredia, vytvárať nové riadiace algoritmy a zároveň spoľahlivo zaznamenávať namerané údaje pre ďalšie spracovanie.

2.3.1 Pôvodná komunikácia v prostredí MATLAB/Simulink

Komunikácia laboratórneho systému TS05 s počítačom je v prostredí MATLAB/Simulink realizovaná prostredníctvom meracej a komunikačnej karty Advantech, ktorá zabezpečuje prepojenie reálneho fyzikálneho modelu so simulačným prostredím. Táto karta umožňuje obojsmerný prenos analógových signálov, teda snímanie výstupných veličín systému a zároveň generovanie riadiacich zásahov privádzaných na jeho vstupy.

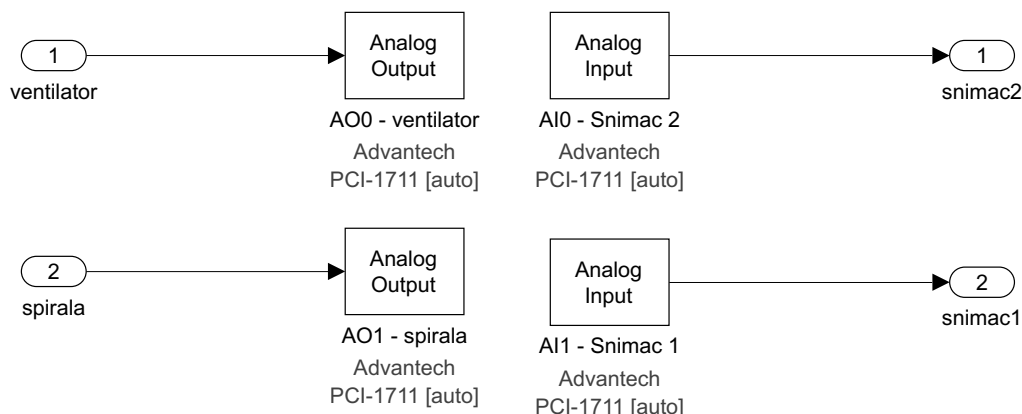
Z pohľadu používateľa je komunikácia realizovaná pomocou vopred pripravených blokov v prostredí Simulink. Základná štartovacia schéma, ktorá slúži na inicializáciu komunikácie a prepojenie jednotlivých vstupov a výstupov systému TS05 s blokom komunikačnej karty, je znázornená na obr. 15. Táto časť schémy predstavuje nadradenú vrstvu zapojenia, v ktorej sú definované jednotlivé signálové väzby medzi reálnym systémom a rozhraním komunikačnej karty.

Vnútroštruktúra komunikačného subsystému, v ktorej sú už explicitne definované jednotlivé porty meracej karty a spôsob spracovania analógových vstupov a výstupov, je zobrazená na obr. 16. Táto schéma predstavuje detailnejšiu implementačnú vrstvu komunikácie, na ktorú nadväzujú ďalšie experimentálne a riadiace úlohy realizované v prostredí MATLAB/Simulink.

Uvedené schémy boli vytvorené ako súčasť projektu KUT – Krátke výučbové materiály a slúžia aj ako podklad pre predmet *Modelovanie a riadenie systémov* [2]. Pre potreby tejto práce predstavovali základné východisko pre realizáciu meraní, identifikácie systému a následný návrh riadenia v prostredí MATLAB/Simulink.

V ďalších experimentoch a pri návrhu regulačných algoritmov bude samotná riadiaca logika realizovaná vždy v bloku MATLAB Function, ktorého výstup bude privedený na vstup vykurovacej špirály. V rámci ďalšieho návrhu riadenia bude systém TS05 zámerne uvažovaný ako systém typu SISO (Single Input – Single Output). Vstup ventilátora bude počas experimentov udržiavaný na konštantnej hodnote 6 V, a preto bude z pohľadu návrhu regulácie chápaný ako konštantná veličina. Ako riadiaci vstup u bude uvažované napätie privádzané na vykurovaciu špirálu a ako výstup y bude použitý signál zo Snímača 1. Signál zo Snímača 2 sa v ďalších návrhoch riadenia nebude uvažovať.

Keďže blok MATLAB Function nemá vlastnú vnútornú pamäť v zmysle uchovávaní stavových veličín medzi jednotlivými diskretnými krokmi, všetky veličiny, ktoré je potrebné zachovať medzi iteráciami algoritmu, musia byť vedené spätnou väzbou z výstupov na vstupy tohto bloku. Aby sa pri takomto zapojení zabránilo vzniku



Obr. 16: Vnútrotná časť Simulinkového bloku komunikácie so systémom TS05

algebraickej slučky [5] v prostredí Simulink, je potrebné do spätnej väzby zaradiť diskretný pamäťový blok typu *Unit Delay*, štandardne reprezentovaný operátorom z^{-1} . Tým sa zabezpečí, že algoritmus bude implementovaný explicitne numerickým spôsobom v rámci vlastného kódu, bez použitia preddefinovaných regulačných blokov prostredia Simulink.

2.3.2 Vlastná komunikácia v jazyku C++

Keďže systém TS05 bol pôvodne používaný predovšetkým v prostredí MATLAB/Simulink a k dispozícii nebola podrobná technická dokumentácia popisujúca jednotlivé vodiče a signály, bolo najprv potrebné experimentálne určiť, ktorý kontakt konektora zodpovedá jednotlivým vstupom a výstupom systému. Súčasne bolo potrebné vytvoriť nový prepojavací kábel a navrhnuť fyzické zapojenie, ktoré by umožnilo komunikáciu systému TS05 s platformou Arduino UNO.

Zo zariadenia TS05 je vyvedený kruhový 8-pinový konektor. Pomocou multimetra boli identifikované jednotlivé kontakty, pričom výsledné priradenie je uvedené v tabuľke 2.

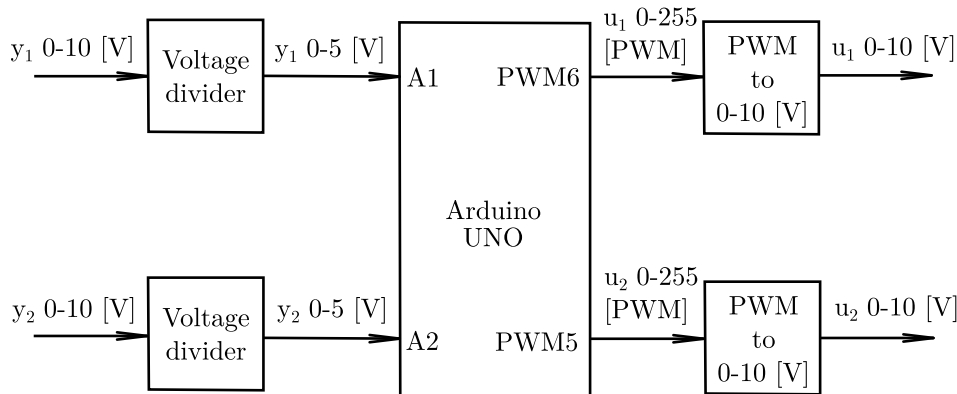
Po identifikácii pinov bol zhotovený nový kábel, ktorý umožnil priamy prístup k meraným aj akčným signálom systému TS05. Následne bola zostavená riadiaca a meracia reťaz založená na mikrokontroléri Arduino UNO.

Akčné veličiny systému, teda ventilátor a vykurovacia špirála, boli z platformy Arduino generované pomocou PWM signálu s rozsahom 0 až 255. Keďže samotný systém TS05 pracuje s analógovými vstupmi v rozsahu 0 až 10 V, bolo potrebné použiť prevodník PWM/0–10 V. Tento prevodník vyžaduje externé napájanie minimálne 12 V a 0,1 A. Takýmto spôsobom bolo možné z digitálnych PWM výstupov Arduino vytvoriť analógové riadiace signály použiteľné pre systém TS05.

Na druhej strane, výstupné signály snímačov systému TS05 sú analógové napätia v rozsahu 0 až 10 V. Keďže analógové vstupy platformy Arduino UNO pracujú iba

Tabuľka 2: Identifikované kontakty konektora systému TS05

Pin	Funkcia
1	Snímač 1
2	Snímač 2
6	Ventilátor
7	Špirála
8	GND



Obr. 17: Schéma prepojenia systému TS05 s platformou Arduino UNO

v rozsahu 0 až 5 V, bolo potrebné tieto signály vhodne upraviť. Na tento účel boli použité napäťové deliče, pričom každý delič bol zostavený z dvoch rezistorov s hodnotou 90 k Ω . Výstupné napätie deliča je dané vzťahom

$$U_{\text{out}} = U_{\text{in}} \frac{R_2}{R_1 + R_2} \quad (2.18)$$

pričom pre $R_1 = R_2 = 90 \text{ k}\Omega$ dostaneme

$$U_{\text{out}} = \frac{U_{\text{in}}}{2} \quad (2.19)$$

Z toho vyplýva, že signál 0 až 10 V zo snímača je po delení prevedený približne na rozsah 0 až 5 V, čo už zodpovedá možnostiam analógových vstupov mikrokontroléra. Samotný prevod v Arduino je následne reprezentovaný hodnotou A/D prevodníka v rozsahu 0 až 1023.

Mapovanie vstupov a výstupov medzi systémom TS05 a platformou Arduino UNO je uvedené v tabuľke 3.

Celkové fyzické zapojenie systému je znázornené na obrázku 17. V schéme označenie u_1 predstavuje napätie privádzané na vykurovaciu špirálu, u_2 napätie privádzané na ventilátor, zatiaľ čo y_1 a y_2 označujú signály zo snímača teploty 1 a snímača teploty 2.

Rovnako ako v podkapitole 2.3.1, aj v prípade implementácie v jazyku C++ bude systém TS05 pre účely ďalších experimentov a návrhu riadenia uvažovaný ako systém typu SISO. Riadiaci vstup u bude predstavovať napätie privádzané na vykurovaciu špirálu, výstup y bude tvorený signálom zo Snímača 1 a vstup ventilátora bude udržiavaný na konštantnej hodnote 6 V.

Po vytvorení hardvérovej časti bolo potrebné vyriešiť aj interpretáciu vstupných a výstupných hodnôt. Najjednoduchším riešením by bolo použiť lineárny prevod medzi digitálnymi rozsahmi 0 až 255, resp. 0 až 1023 a fyzikálnym rozsahom 0 až 10 V. Experimentálne merania však ukázali, že hoci je správanie systému približne lineárne, nejde o ideálne lineárny prevod.

Odchýlky sú spôsobené viacerými faktormi. V prípade napäťových deličov zohráva úlohu tolerancia použitých rezistorov, ktorá spôsobuje, že reálny deliaci pomer sa

Tabuľka 3: Mapovanie vstupov a výstupov medzi TS05 a Arduino UNO

Port Arduino	Signál	Reálny rozsah	Digitálny rozsah
A1	Snímač 1	0–10 V	0–1023
A2	Snímač 2	0–10 V	0–1023
PWM5	Ventilátor	0–10 V	0–255
PWM6	Špirála	0–10 V	0–255

mierne odlišuje od ideálnej hodnoty. Pri PWM/0–10 V prevodníku sa zasa prejavilo, že pri minimálnom použitom napájaní 12 V dochádza k miernemu podhodnocovaniu výstupného napätia. Z tohto dôvodu boli pre všetky štyri signálové cesty vykonané experimentálne merania, v ktorých sa porovnávali hodnoty generované alebo čítané mikrokontrolérom s hodnotami nameranými multimetrom.

Pre každý vstup a výstup bolo realizovaných šesť meraní. Výsledky pre jednotlivé kanály sú uvedené v tabuľkách 4, 5, 6 a 7.

Tabuľka 4: Kalibračné merania pre výstup PWM5 – ventilátor

PWM5	Namerané napätie [V]
0	0.00
50	2.19
100	4.13
150	6.03
200	7.91
250	9.32

Tabuľka 5: Kalibračné merania pre výstup PWM6 – špirála

PWM6	Namerané napätie [V]
0	0.00
50	2.36
100	4.34
150	6.24
200	8.07
250	9.30

Tabuľka 6: Kalibračné merania pre vstup A1 – snímač 1

Hodnota A1	Namerané napätie [V]
0	0.00
365	3.68
390	3.90
739	7.34
904	8.95
972	9.64

Tabuľka 7: Kalibračné merania pre vstup A2 – snímač 2

Hodnota A2	Namerané napätie [V]
0	0.00
383	3.83
406	4.06
655	6.55
755	7.47
769	7.68

Z uvedených hodnôt je zrejmé, že priebehy sú síce blízke lineárnemu správaniu, avšak nie natolko, aby bolo možné bezvýhradne použiť jednoduchú lineárnu kalibráciu. Preto bolo rozhodnuté overiť viacero typov aproximačných funkcií. Pre každý kanál bolo testovaných päť modelov: lineárny, kvadratický, kubický, mocninový a exponenciálny. Kvalita aproximácie bola hodnotená pomocou ukazovateľa RMSE.

Výsledky ukázali, že vo všetkých štyroch prípadoch dosiahla najnižšiu chybu kubická funkcia. Z tohto dôvodu bola pre prevod medzi digitálnymi a fyzikálnymi veličinami zvolená práve kubická aproximácia. Jej všeobecný tvar možno zapísať ako

$$y = ax^3 + bx^2 + cx + d \quad (2.20)$$

kde x predstavuje vstupnú hodnotu a y kalibrovaný výstup.

Pre jednotlivé kanály boli použité nasledujúce kalibračné funkcie. Pre prehľadnosť zápisu sú v ďalšom texte koeficienty uvedené so zaokrúhlením na štyri desatinné miesta štandardným matematickým zaokrúhľovaním. V samotnej softvérovej implementácii však boli použité presnejšie hodnoty koeficientov, konkrétne s ôsmimi desatinnými miestami.

$$\text{PWM5}_{\text{vent}}(u) = 0.0594u^3 - 0.2799u^2 + 24.2048u - 0.5252 \quad (2.21)$$

$$\text{PWM6}_{\text{spir}}(u) = 0.0873u^3 - 0.3932u^2 + 22.8497u - 0.6156 \quad (2.22)$$

$$U_{\text{sn1}}(A1) = 4.6404 \times 10^{-10} A1^3 - 8.4425 \times 10^{-7} A1^2 + 1.0295 \times 10^{-2} A1 + 5.8786 \times 10^{-5} \quad (2.23)$$

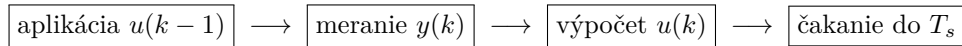
$$U_{\text{sn2}}(A2) = -9.0911 \times 10^{-10} A2^3 + 8.9900 \times 10^{-7} A2^2 + 9.7887 \times 10^{-3} A2 - 9.6519 \times 10^{-5} \quad (2.24)$$

Aj keď sa neskôr experimentálne ukázalo, že ani táto aproximácia nie je úplne ideálna, pre potreby tejto práce bola dosiahnutá presnosť dostatočná. Jedným z možných spôsobov ďalšieho zlepšenia by bolo vykonanie väčšieho počtu kalibračných meraní a následná identifikácia presnejšej prevodovej charakteristiky.

Softvérová implementácia pre platformu Arduino UNO bola realizovaná v jazyku C++ s využitím rozšírenia PlatformIO v prostredí *Visual Studio Code*. Program je rozdelený na niekoľko základných častí: definície hardvérových pinov a časových parametrov, pomocné funkcie, obsluha sériovej komunikácie a hlavná riadiaca slučka.

V programe je použitá pevná vykonávací postupnosť, ktorá zodpovedá diskretnému riadeniu. Jej cieľom je zabezpečiť, aby aktuálne meraná veličina $y(k)$ závisela iba od riadiaceho zásahu z predchádzajúceho kroku $u(k-1)$. Nový zásah $u(k)$ sa naopak vypočíta na základe práve odmeranej hodnoty $y(k)$ a na systém sa aplikuje až v nasledujúcom kroku. Tým sa zabráni miešaniu aktuálne meranej odozvy s práve vypočítavaným riadiacim zásahom.

Postup jedného vykonávacieho kroku možno znázorniť nasledovne:



Pri začiatku každého kroku sa preto najprv na výstupy odošlú hodnoty **vent** a **spirala**, ktoré boli vypočítané už v predchádzajúcej iterácii. Pred odoslaním sa tieto veličiny obmedzia na rozsah 0 až 10 V a následne sa pomocou kubických kalibračných funkcií prevedú na PWM hodnoty v rozsahu 0 až 255. Takto sa na systém aplikuje zásah $u(k-1)$.

Následne sa načítajú analógové vstupy A1 a A2 a pomocou kalibračných funkcií sa prepočítajú na fyzikálne veličiny snímačov. Z týchto údajov sa v riadiacej časti programu vypočítajú nové hodnoty **vent** a **spirala**, teda zásah $u(k)$. Tieto hodnoty sa však už v danom kroku na systém neposielaajú, ale použijú sa až pri nasledujúcej iterácii.

Časovanie programu je realizované pomocou funkcie `micros()`. Hlavná časť algoritmu sa vykoná iba vtedy, keď od posledného kroku uplynie požadovaný interval

$T_s = 0,1$ s. Tým sa zabezpečí, že riadiaca logika nebeží pri každom priechode funkciou `loop()`, ale iba raz za každú vzorkovaciu periódu. Okrem globálneho času experimentu sa zaznamenáva aj skutočná dĺžka vykonaného kroku, čo umožňuje kontrolu odchýlok od požadovanej periódy.

Automatický záznam dát je riešený cez sériový port. Po inicializácii odošle mikrokontrolér správu `READY`. Externý Python skript po jej prijatí odošle príkaz `START`, čím sa spustí experiment a zároveň aj odosielanie dát v CSV formáte. Po uplynutí definovaného času experimentu sa výstupy nastaví na nulové hodnoty, experiment sa ukončí a cez sériový port sa odošle správa `DONE`. Externý skript tým získa informáciu o ukončení merania a uzavrie výstupný súbor.

Takto navrhnutá štruktúra programu oddeľuje obsluhu komunikácie, časovanie, prevody veličín a samotný riadiaci zákon. Pri zmene algoritmu je preto potrebné upravovať predovšetkým iba časť programu, v ktorej sa z nameraných veličín vypočítavajú nové hodnoty `vent` a `spirala`. Ostatné časti implementácie môžu zostať nezmenené. Úplné zdrojové kódy hlavného programu pre Arduino UNO a externého skriptu pre logovanie dát sú uvedené v prílohách 30 a 31.

Pomocné funkcie použité v programe sú uvedené v nasledujúcej ukážke.

```

1 float cubic(float x, float a, float b, float c, float d) {
2     return a * x * x * x + b * x * x + c * x + d;
3 }
4
5 float clampFloat(float value, float minValue, float maxValue) {
6     if (value < minValue) {
7         return minValue;
8     }
9     if (value > maxValue) {
10        return maxValue;
11    }
12    return value;
13 }
14
15 int clampInt(int value, int minValue, int maxValue) {
16     if (value < minValue) {
17         return minValue;
18     }
19     if (value > maxValue) {
20         return maxValue;
21     }
22     return value;
23 }

```

Výpis kódu 5: Pomocné funkcie použité v programe

Funkcia `cubic()` slúži na výpočet hodnoty kubického kalibračného polynómu. Funkcia `clampFloat()` obmedzuje reálne hodnoty do zvoleného intervalu. Funkcia `clampInt()` obmedzuje celočíselné hodnoty, najmä PWM výstupy, na povolený rozsah.

2.3.3 Komunikácia s programovateľným logickým automatom

Ako riadiaca platforma pre túto časť práce bol použitý moderný priemyselný PLC (programovateľný logický automat) systém Siemens SIMATIC S7-1200 G2. Základné technické informácie, spôsob konfigurácie a vlastnosti tejto platformy sú uvedené v systémovej dokumentácii výrobcu [12]. Cieľom tejto implementácie bolo overiť možnosti riadenia tepelného systému TS05 aj na štandardnej priemyselnej platforme, ktorá sa svojím charakterom výrazne približuje reálnym nasadeniam v praxi.

Z pohľadu priemyselnej automatizácie je tradičným štandardom pre analógové signály najmä prúdová slučka 4–20 mA, keďže ide o riešenie široko používané v technologických prevádzkach a priemyselných zariadeniach. Existujú aj spôsoby, ako prevádzať napäťové signály 0–10 V na prúdový štandard 4–20 mA, napríklad pomocou externých prevodníkov. Podobný princíp externého prevodu bol v tejto práci použitý aj pri implementácii komunikácie v jazyku C++, kde bolo potrebné prevádzať PWM signál na analógový rozsah 0–10 V v podkapitole 2.3.2.

V tomto prípade však nebolo potrebné realizovať externý prevod signálov, pretože k PLC systému boli k dispozícii aj nové analógové rozširujúce moduly Siemens typu SM

1233 AI 4×14 bit / AQ 4×14 bit, 6ES7 233-4HF50-0XB0. Tento modul obsahuje dva analógové vstupy a dva analógové výstupy, čo pre potreby systému TS05 plne postačuje. Z hľadiska počtu kanálov teda presne zodpovedá požiadavkám danej experimentálnej zostavy.

Z hľadiska spracovania analógových vstupov je potrebné uviesť, že modul umožňuje parametrizovať digitálne vyhladzovanie nameraných hodnôt (*smoothing*). V rámci merania bolo toto vyhladzovanie nastavené na možnosť *None*, aby modul nevykonával dodatočné priemerovanie vzoriek. Súčasne však zostáva aktívna funkcia potlačenia rušenia (*noise rejection*), ktorá je súčasťou prevodu analógovej hodnoty a jej frekvenciu možno iba parametrizovať. V tomto prípade bolo zvolené potlačenie rušenia s frekvenciou 50 Hz.

Analógové signály sú v PLC systémoch Siemens interne reprezentované ako celočíselné hodnoty s rozsahom 0 až 27648, pričom tento digitálny rozsah zodpovedá fyzikálnemu napäťovému rozsahu 0 až 10 V. Na rozdiel od realizácie s platformou Arduino UNO, kde bolo vzhľadom na použitú jednoduchšiu a lacnejšiu hardvérovú architektúru potrebné experimentálne vykonať vlastnú kalibráciu signálových ciest, sa v prípade PLC systému Siemens predpokladalo, že originálny analógový modul zabezpečuje dostatočne presný a lineárny prevod medzi digitálnou a fyzikálnou reprezentáciou. Z tohto dôvodu nebola v tejto časti práce realizovaná dodatočná manuálna kalibrácia.

Z hľadiska hardvérového zapojenia bol PLC systém napájaný zo zdroja 24 V DC, ktorý zabezpečoval jeho štandardnú prevádzku. Súčasne bol PLC pripojený do rovnakej ethernetovej siete ako riadiaci počítač, pričom bolo potrebné nastaviť IP adresu sieťovej karty počítača tak, aby bola v rovnakej podsieti ako PLC zariadenie. Tým sa zabezpečila korektná komunikácia medzi vývojovým prostredím a samotným automatom.

Vstupy a výstupy systému TS05 boli následne pripojené priamo na analógový modul PLC. Okrem samotných signálových vodičov bolo, samozrejme, potrebné pripojiť aj referenčnú zem. Použitý modul obsahuje spolu osem svoriek, teda dva analógové vstupy, dva analógové výstupy a ku každému z nich príslušný záporný pól.

Softvérová časť implementácie bola realizovaná v prostredí TIA Portal V20. Programové bloky použité pri implementácii boli vytvárané prevažne v jazyku SCL (*Structured Control Language*), ktorý predstavuje textový programovací jazyk používaný v prostredí TIA Portal. V prvom kroku bolo potrebné určiť adresy jednotlivých analógových vstupov a výstupov modulu. Výsledné adresovanie je uvedené v tabuľke 8.

Keďže analógová hodnota je v tomto prípade reprezentovaná ako slovo (*word*), teda ako dvojbajtová veličina, jednotlivé adresy sa posúvajú po dvoch bajtoch, napríklad z %IW80 na %IW82, resp. z %QW80 na %QW82.

V hlavnom organizačnom bloku PLC programu bolo rozhodnuté realizovať iba základné servisné operácie, konkrétne prevod medzi internou digitálnou reprezentáciou analógových signálov a ich inžinierskymi jednotkami, ako aj výpočet času od začiatku experimentu. Tento čas sa ďalej používa výhradne na účely logovania dát, samotné riadiace algoritmy od neho nebudú závislé.

Prevod vstupných analógových hodnôt do napäťového rozsahu 0–10 V (z pohľadu modulu PLC ide o čítané vstupy) bol realizovaný ako

Tabuľka 8: Adresovanie analógových vstupov a výstupov PLC modulu pre systém TS05

Signál	Typ	Adresa
Ventilátor	Int	%QW80
Špirála	Int	%QW82
Snímač 1	Int	%IW80
Snímač 2	Int	%IW82

$$U_{in} = \frac{INT}{2764.8} \quad (2.25)$$

Naopak, pri generovaní výstupných analógových signálov bolo potrebné vykonať opačný prevod z reálnej hodnoty napätia na internú celočíselnú reprezentáciu:

$$INT = U_{out} \cdot 2764.8 \quad (2.26)$$

Na výpočet aktuálneho času od začiatku experimentu bol vytvorený vlastný funkčný blok `Current_time`. Tento blok využíva systémovú funkciu `RD_SYS_T`, ktorá vracia aktuálny systémový čas PLC vo forme dátového typu *DTL*. Uvedený typ obsahuje viacero zložiek, ako sú rok, mesiac, deň a denný čas. Keďže na účely logovania nebolo potrebné uchovávať kompletnú časovú štruktúru, ale iba čas od začiatku merania v sekundách, bol vytvorený samostatný blok, ktorý pri prvom úspešnom volaní uloží počiatočný čas a pri ďalších volaniach vypočítava časový rozdiel voči tomuto okamihu.

Implementácia bloku `Current_time` je uvedená v nasledujúcej ukážke.

```

1 #retVal := RD_SYS_T(OUT => #currentTime);
2
3 IF (NOT #started) AND (#retVal = 0) THEN
4     #startTime := #currentTime;
5     #started := TRUE;
6 END_IF;
7
8 IF #started AND (#retVal = 0) THEN
9     #elapsedTime := T_DIFF(IN1 := #currentTime, IN2 := #startTime);
10    #CurrentTimeSec := DINT_TO_LREAL(TIME_TO_DINT(#elapsedTime)) / 1000.0;
11 ELSE
12    #CurrentTimeSec := 0.0;
13 END_IF;
```

Výpis kódu 6: Implementácia funkčného bloku `Current_time`

Zoznam interných premenných použitých v tomto bloku je uvedený v tab. 9.

Celková štruktúra hlavného organizačného bloku, ktorý je vykonávaný v každom cykle PLC, je zobrazená na obr. 18. Ako už bolo uvedené, v tomto bloku sa realizuje iba škálovanie analógových premenných a výpočet aktuálneho času od začiatku experimentu.

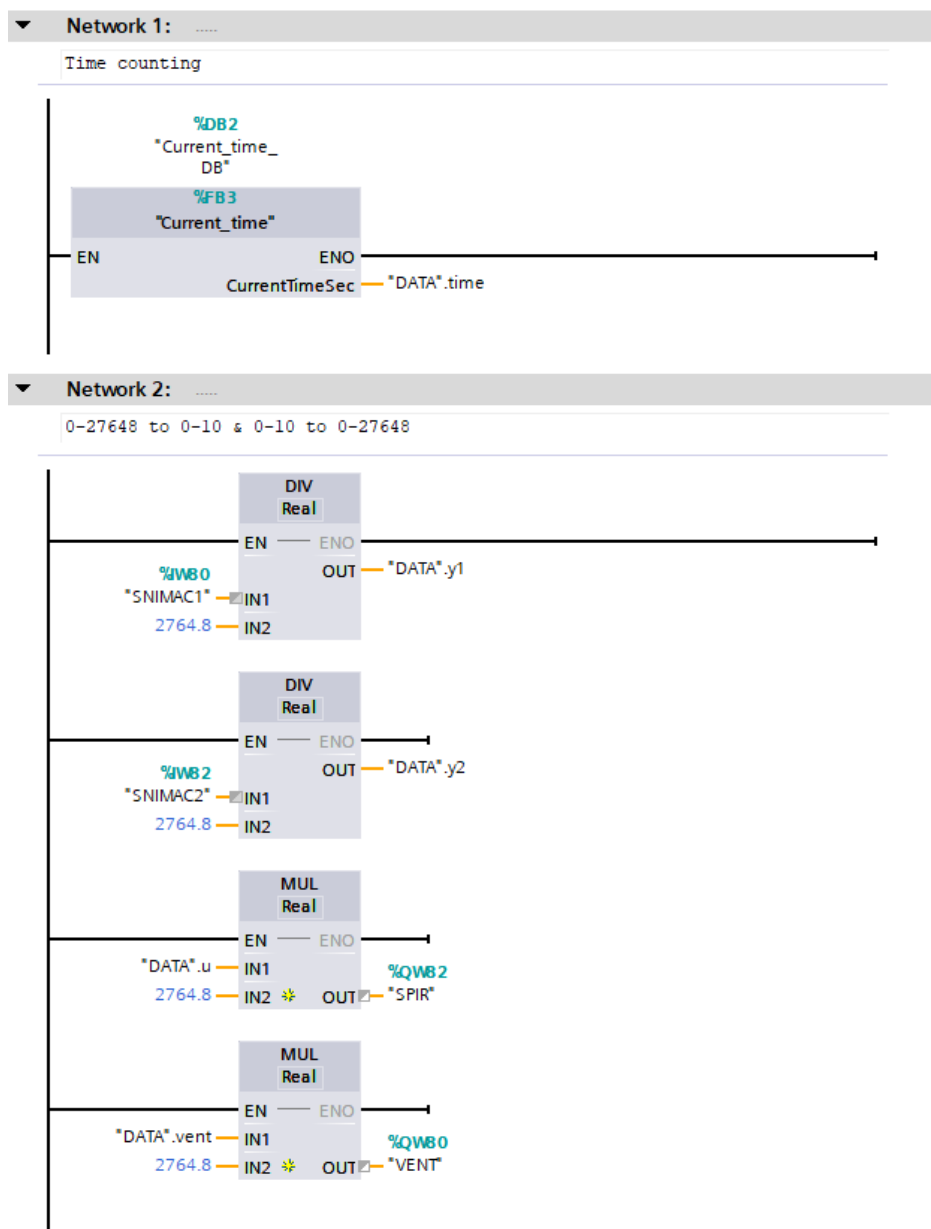
Platforma Siemens zároveň umožňuje realizovať používateľskú logiku v blokoch spúšťaných s pevne definovanou periódou. Na tento účel bol vytvorený organizačný blok typu *Cyclic Interrupt*, v ktorom bola nastavená perióda 0.1 s. Práve v tomto bloku bude v ďalších častiach práce realizovaná riadiaca logika a zároveň aj logovanie nameraných a vypočítaných veličín.

Dôležité je, že tento blok pracuje ako prerušovacia úloha. To znamená, že servisné operácie, ako sú prevody veličín a výpočet času, prebiehajú v každom bežnom cykle PLC, zatiaľ čo samotné riadenie a logovanie sa vykonávajú iba raz za definovanú vzorkovaciu periódu. Takéto rozdelenie umožňuje prehľadnú separáciu pomocných častí programu od vlastnej regulačnej úlohy.

Pre jednotlivé riadiace algoritmy bude následne úlohou implementovať dve hlavné časti: samotný riadiaci zákon a príslušné logovanie údajov. Riadiaca logika bude

Tabuľka 9: Premenné funkčného bloku `Current_time` v prostredí TIA Portal

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Output	<code>CurrentTimeSec</code>	LReal	0.0	Non-retain
Static	<code>started</code>	Bool	false	Non-retain
Static	<code>retVal</code>	Int	0	Non-retain
Static	<code>startTime</code>	DTL	DTL#1970-01-01-...	Non-retain
Static	<code>currentTime</code>	DTL	DTL#1970-01-01-...	Non-retain
Static	<code>elapsedTime</code>	Time	T#0ms	Non-retain



Obr. 18: Servisná časť hlavného organizačného bloku PLC programu

umiestnená vo funkčných blokoch (*Function Block*), keďže tieto bloky majú vlastnú vnútornú pamäť. Práve táto vlastnosť je pri implementácii regulátorov v PLC prirodzená a veľmi výhodná. Operácie, ako je čakanie definovaného času, oneskorené spustenie alebo časovo podmienené prechody medzi stavmi, budú realizované pomocou štandardných časovačov. Takýto prístup je z pohľadu PLC programovania prirodzenejší a tradičnejší než priame využívanie času počítaného v bloku **Current_time**.

Na účely logovania boli rovnako vytvorené samostatné funkčné bloky. Ich úlohou je prijímať na vstup vybrané premenné a pri každom povolenom vzorkovaní ich ukladať do interných polí. Takto uložené údaje je následne možné manuálne skopírovať do tabuľkového procesora a uložiť napríklad vo formáte **.csv**. V rámci tejto práce sa nepodarilo nájsť natívny spôsob pohodlného logovania dát priamo z PLC bez použitia nastavbového HMI systému typu WinCC alebo bez použitia špecializovaných hardvérových modulov. Uvedené riešenie však bolo pre potreby experimentov plne postačujúce.

Keďže jednotlivé bloky pre logovanie sa líšia iba v tom, ktoré konkrétne premenné ukladajú, bolo rozhodnuté uviesť iba jeden reprezentatívny príklad takéhoto logovacieho

bloku, konkrétne pre PI regulátor opísaný v sekcii 3.5. Tento príklad je dostatočný na vysvetlenie princípu a zároveň zabráňuje zbytočnému opakovaniu veľmi podobných implementácií.

Zdrojový kód bloku FB_PI_LOGGER je uvedený nižšie. Zoznam premenných tohto logovacieho bloku je uvedený v tab. 10.

```

1 IF NOT #Full AND #Enable THEN
2   IF #idx <= 9999 THEN
3     #TimeLog_s[#idx] := #CurrentTimeSec;
4     #ULOG[#idx] := #u;
5     #YLOG[#idx] := #y;
6     #RLOG[#idx] := #r;
7     #YPBLOG[#idx] := #ypb;
8     #idx := #idx + 1;
9     #SampleCount := #idx;
10  ELSE
11    #Full := TRUE;
12  END_IF;
13 END_IF;

```

Výpis kódu 7: Implementácia logovacieho bloku FB_PI_LOGGER

Z uvedeného dôvodu už v ďalších častiach práce nebudú logovacie bloky samostatne zobrazované pre každý algoritmus. Pri jednotlivých návrhoch riadenia budú prezentované iba samotné bloky riadiacej logiky a príslušné siete organizačných blokov, v ktorých budú tieto algoritmy volané. Tým sa zachová prehľadnosť textu a zároveň sa zabráni opakovaniu už raz vysvetlených implementačných princípov.

Tabuľka 10: Premenné funkčného bloku FB_PI_LOGGER

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Input	CurrentTimeSec	LReal	0.0	Non-retain
Input	u	Real	0.0	Non-retain
Input	y	Real	0.0	Non-retain
Input	r	Real	0.0	Non-retain
Input	ypb	Real	0.0	Non-retain
Input	Enable	Bool	false	Non-retain
Output	SampleCount	DInt	0	Non-retain
Output	Full	Bool	false	Non-retain
Static	idx	DInt	0	Non-retain
Static	TimeLog_s	Array[0..9999] of LReal	–	Non-retain
Static	ULOG	Array[0..9999] of Real	–	Non-retain
Static	YLOG	Array[0..9999] of Real	–	Non-retain
Static	RLOG	Array[0..9999] of Real	–	Non-retain
Static	YPBLOG	Array[0..9999] of Real	–	Non-retain

3 PI regulátor pre riadenie v okolí jedného pracovného bodu

V tejto kapitole je riešený návrh a implementácia PI (Proporcionálny, Integrálny) regulátora pre tepelný systém TS05. Najskôr je stručne zhrnutá teoretická štruktúra PI regulátora a jeho zapojenie v uzavretom regulačnom obvode, následne je tento prístup aplikovaný na identifikovaný model systému TS05 vrátane ladenia parametrov a realizácie v reálnom čase v prostredí MATLAB a v jazyku C++.

3.1 Rozbor algoritmu riadenia

PI regulátor predstavuje zjednodušenú formu PID regulátora, v ktorej je derivačná zložka vynechaná. Regulátory typu PID a ich modifikácie patria medzi najčastejšie používané štruktúry v priemyselnom riadení, najmä vďaka jednoduchej implementácii a možnosti praktického ladenia [4, 6]. Pre systémy s pomalou dynamikou, ako sú tepelné sústavy, derivačný člen spravidla neprináša zlepšenie regulácie a naopak môže zosilňovať merací šum.

Paralelná forma PI regulátora je definovaná prenosovou funkciou

$$C(s) = K_p + \frac{K_i}{s}, \quad (3.1)$$

kde K_p je proporcionálny zisk a K_i integračný zisk regulátora.

Regulátor je zapojený v štandardnej uzavretej spätnej väzbe so zápornou spätnou väzbou. Bloková schéma uzavretého regulačného obvodu je znázornená na obr. 19. V schéme označenie u_1 predstavuje napätie privádzané na vykurovaciu špirálu, u_2 napätie privádzané na ventilátor, zatiaľ čo y_1 a y_2 označujú signály zo snímača teploty 1 a snímača teploty 2.

Prenosy regulačného obvodu

Nech $G_s(s)$ označuje prenos regulovaného procesu a $C(s)$ prenos regulátora. Prenos otvoreného regulačného obvodu je potom daný vzťahom

$$G_{ORO}(s) = C(s) G_s(s), \quad (3.2)$$

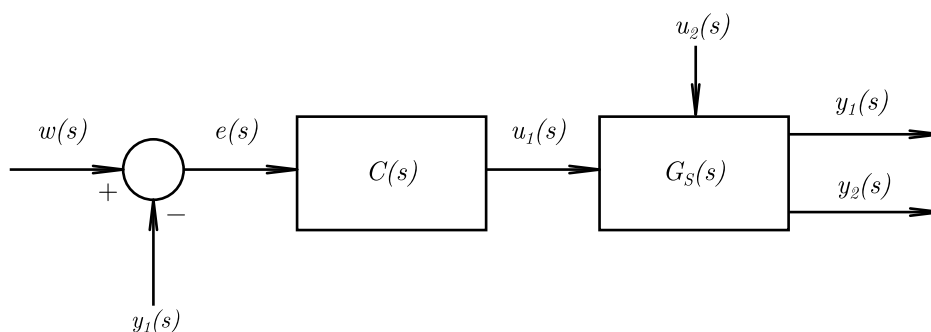
pričom prenos uzavretého regulačného obvodu má tvar

$$G_{URO}(s) = \frac{C(s) G_s(s)}{1 + C(s) G_s(s)}. \quad (3.3)$$

Stabilita uzavretého regulačného obvodu je daná koreňmi charakteristickej rovnice

$$1 + C(s) G_s(s) = 0. \quad (3.4)$$

Pre spojitý systém je podmienkou stability, aby všetky póly uzavretého obvodu mali zápornú reálnu časť.



Obr. 19: Bloková schéma uzavretého regulačného obvodu s PI regulátorom

Zákon riadenia

V časovej oblasti možno PI regulátor zapísať v tvare

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau, \quad (3.5)$$

kde $e(t) = w(t) - y(t)$ predstavuje regulačnú odchýlku, $w(t)$ žiadanú hodnotu a $y(t)$ výstup systému.

Pre diskretnú realizáciu s periódou vzorkovania T_s je integračný člen aproximovaný rekurentným vzťahom

$$e(k) = w(k) - y(k), \quad (3.6)$$

$$I(k) = I(k-1) + T_s e(k), \quad (3.7)$$

$$u(k) = K_p e(k) + K_i I(k). \quad (3.8)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu je v praxi uplatnená saturácia radiaceho signálu.

Pseudokód PI regulátora

```
% Inputs: w (setpoint), y (output), Ts
% States: I (integrator state)
% Params: Kp, Ki, u_min, u_max

e = w - y;

% Integrator update
I = I + Ts * e;

% PI law
u = Kp * e + Ki * I;

% Saturation
u = min(max(u, u_min), u_max);
```

Výpis kódu 8: Pseudokód diskretnéj realizácie PI regulátora so saturáciou

3.2 Nastavenie parametrov PI regulátora

Pri návrhu parametrov PI regulátora bolo najprv potrebné zvoliť vhodnú metódu ladenia. Vzhľadom na štruktúru identifikovaného modelu systému TS05 sa ukázalo, že viaceré bežné analytické metódy ladenia nie je možné použiť priamo alebo v štandardnom tvare. Z tohto dôvodu boli najskôr posúdené ich obmedzenia a následne bol na samotné ladenie zvolený numerický nástroj **piddtune** v prostredí MATLAB.

Charakteristický polynóm uzavretého regulačného obvodu pre PI regulátor má po dosadení tvar

$$s^3 + \frac{b_1}{b_2}s^2 + \left(\frac{b_0}{b_2} + \frac{a_0 K_p}{b_2}\right)s + \frac{a_0 K_i}{b_2} = 0 \quad (3.9)$$

Ak by sme chceli použiť metódu umiestnenia pólov pomocou rozšíreného želaného polynómu tretieho rádu, jeho všeobecný tvar je

$$s^3 + (p + 2k\omega_0)s^2 + (2k\omega_0p + \omega_0^2)s + \omega_0^2p = 0 \quad (3.10)$$

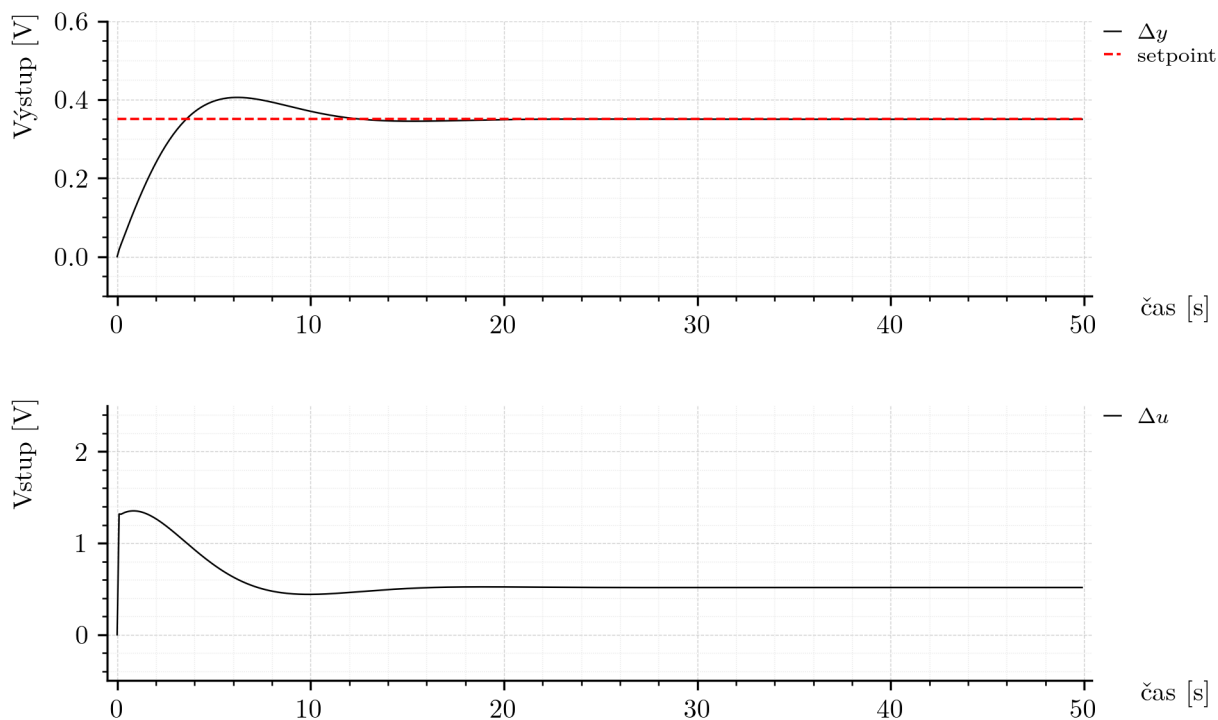
Porovnaním koeficientov polynómov (3.9) a (3.10) dostaneme sústavu rovníc:

$$1 = 1 \quad (3.11)$$

$$p + 2k\omega_0 = \frac{b_1}{b_2} \quad (3.12)$$

$$2k\omega_0p + \omega_0^2 = \frac{b_0}{b_2} + \frac{a_0 K_p}{b_2} \quad (3.13)$$

$$\omega_0^2p = \frac{a_0 K_i}{b_2} \quad (3.14)$$



Obr. 20: Simulačný výsledok uzavretého PI riadenia systému TS05

Táto sústava nemá realizovateľné riešenie pre parametre K_p a K_i . Dôvodom je, že členy závislé od K_p a K_i sa vyskytujú iba v dvoch rovniciach, zatiaľ čo parametre želaného dynamického správania k , ω_0 , p ovplyvňujú tri rovnice. Sústava je preto pre fyzikálne konzistentné riešenie preurčená a neexistuje v nej žiadna kombinácia parametrov, ktorá by spĺňala všetky rovnice súčasne.

Rovnako nebolo vhodné použiť klasickú metódu Ziegler–Nichols založenú na určení kritického zosilnenia, keďže pre identifikovaný model (2.12) nebolo možné týmto spôsobom získať parametre potrebné na jej štandardnú aplikáciu.

Keďže návrh PI regulátora nie je hlavnou témou práce, bol zvolený numerický nástroj `pidtune` [4, 6]. Jeho princíp spočíva v tom, že pre zvolenú požadovanú šírku pásma uzavretého systému vypočíta optimálne hodnoty parametrov PI regulátora tak, aby bola zabezpečená dobrá dynamická odozva a primeraná robustnosť.

Najskôr bol systém prevedený do diskkrétnej podoby a následne bolo vykonané ladenie PI regulátora pomocou funkcie `pidtune()`. Použitý postup je uvedený v nasledujúcej ukážke.

```

1 Gs = Gs_fmin;
2 Ts = 0.1;
3
4 Gz = c2d(Gs, Ts, 'tustin');
5 [Ad,Bd,Cd,Dd] = ssdata(Gz);
6
7 wc = 0.5;
8 Cz = pidtune(Gz, 'pi', wc);
9
10 [numC,denC] = tfdata(Cz,'v');
11 [Ac,Bc,Cc,Dc] = tf2ss(numC,denC);

```

Výpis kódu 9: Diskretizácia modelu a ladenie PI regulátora pomocou `pidtune()`

Získané parametre regulátora:

$$K_p = 3.58, \quad K_i = 1.84 \quad (3.15)$$

Prenosová funkcia otvoreného regulačného obvodu je

$$G_{oro}(s) = \frac{21.21s + 11.15}{s^3 + 59.01s^2 + 8.95s} \quad (3.16)$$

Póly otvoreného systému sú:

$$s_1 = 0, \quad s_2 = -58.855, \quad s_3 = -0.1521 \quad (3.17)$$

Z polohy pólov otvoreného obvodu je zrejmé, že systém obsahuje integrujúci člen a dva póly v ľavej polrovine komplexnej roviny.

Simulácia bola vykonaná s obmedzením odchýlkového akčného zásahu na interval $[-4, 6]$, čo po pripočítaní pracovného bodu $u_{pb} = 4$ V zodpovedá fyzikálnemu rozsahu vstupu špirály 0 až 10 V. Výsledok simulácie je uvedený na obrázku 20.

Z výsledkov boli zistené tieto hodnoty:

$$\text{Čas ustálenia} = 11.5 \text{ s} \quad (3.18)$$

$$\text{Preregulovanie} = 15.9\% \quad (3.19)$$

$$\text{MSE} = 0.04369 \quad (3.20)$$

Dosiahnutý výsledok možno považovať za vhodný pre riadenie tepelného systému. Cieľom je dosiahnuť čo najrýchlejšiu odozvu. Preregulovanie približne 16 percent je akceptovateľné, keďže riadený skok je malý a tepelná sústava je pomalá, čo prirodzene limituje tvar dynamickej odozvy.

3.3 Implementácia: MATLAB/Simulink

Ako je uvedené v zadaní, cieľom práce je implementácia riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Preto bol PI regulátor implementovaný tak, aby mohol pracovať v reálnom čase bez použitia hotových Simulink blokov určených na riadenie. Celá logika riadenia bola zapísaná priamo do bloku MATLAB Function, ktorý sa vykonáva raz za každý riadiaci cyklus s periódou vzorkovania 0.1 sekundy.

Pri implementácii regulátora bol zavedený prístup založený na regulácii odchýlok od pracovného bodu. Namiesto práce s absolútnymi hodnotami veličín sa regulácia vykonáva v inkrementálnom tvare.

Výstupná veličina je vyjadrená ako odchýlka od pracovného bodu:

$$\Delta y = y - y_{pb}, \quad (3.21)$$

kde y_{pb} predstavuje odhad ustáleného stavu systému (pracovného bodu).

Podobne aj riadiaci zásah je počítaný ako odchýlka od pracovného bodu:

$$\Delta u = u - u_{pb}, \quad (3.22)$$

pričom výsledný akčný zásah je následne získaný ako:

$$u = \Delta u + u_{pb}. \quad (3.23)$$

Referenčná hodnota (*setpoint*) je taktiež definovaná vzhľadom na pracovný bod, čo znamená, že regulátor riadi systém v okolí pracovného bodu, nie v absolútnych hodnotách.

Tento prístup zodpovedá riadeniu v okolí pracovného bodu a v praxi sa ukázal ako vhodný pri zmenách prevádzkových podmienok systému.

Implementácia regulátora je nasledovná:

```

1 function [u, integrator_out, y_pb_out] = ...
2     pi_block(y, setpoint_pb, integrator_in, t, y_pb_in)
3
4 % PI controller
5
6 u_pb = 4;
7 Ts = 0.1;
8 u_pb_min = - u_pb;
9 u_pb_max = 10 - u_pb;
10
11 Kp = 3.58;
12 Ki = 1.84;
```

```

13
14 % waiting phase for steady-state detection
15 if t < 10*60
16     u = u_pb;
17     y_pb_out = y;
18     integrator_out = 0;
19
20 else
21     y_pb_out = y_pb_in;
22     y_pb = y_pb_in;
23
24     % output normalization
25     dy = y - y_pb;
26
27     % control error
28     e = setpoint_pb - dy;
29
30     % integrator
31     integrator_out = integrator_in + e * Ts;
32
33     % PI control law
34     du = Kp * e + Ki * integrator_out;
35     du = min(max(du, u_pb_min), u_pb_max);
36
37     % input normalization
38     u = du + u_pb;
39
40 end
41 end

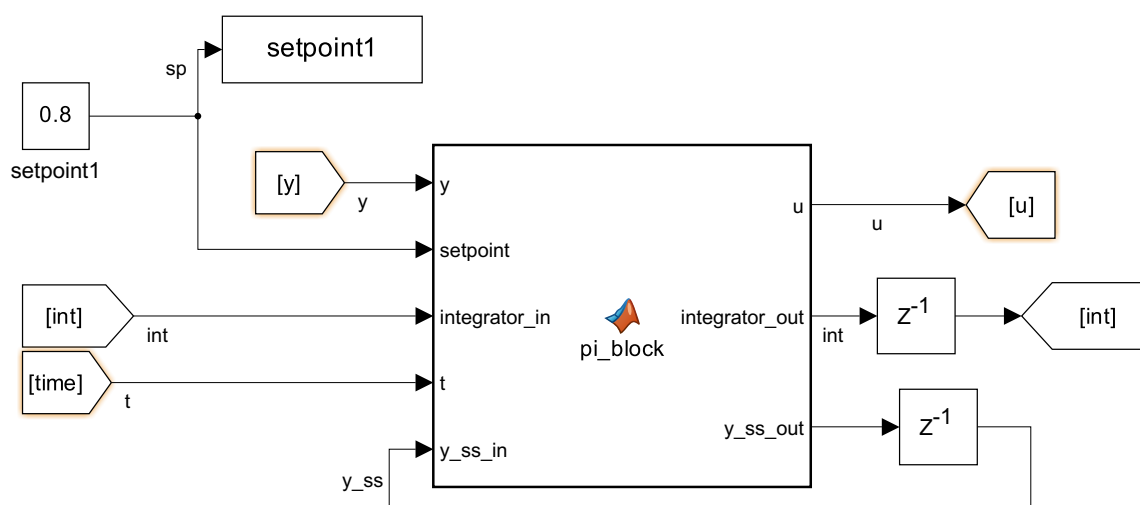
```

Výpis kódu 10: Implementácia PI regulátora v bloku MATLAB Function

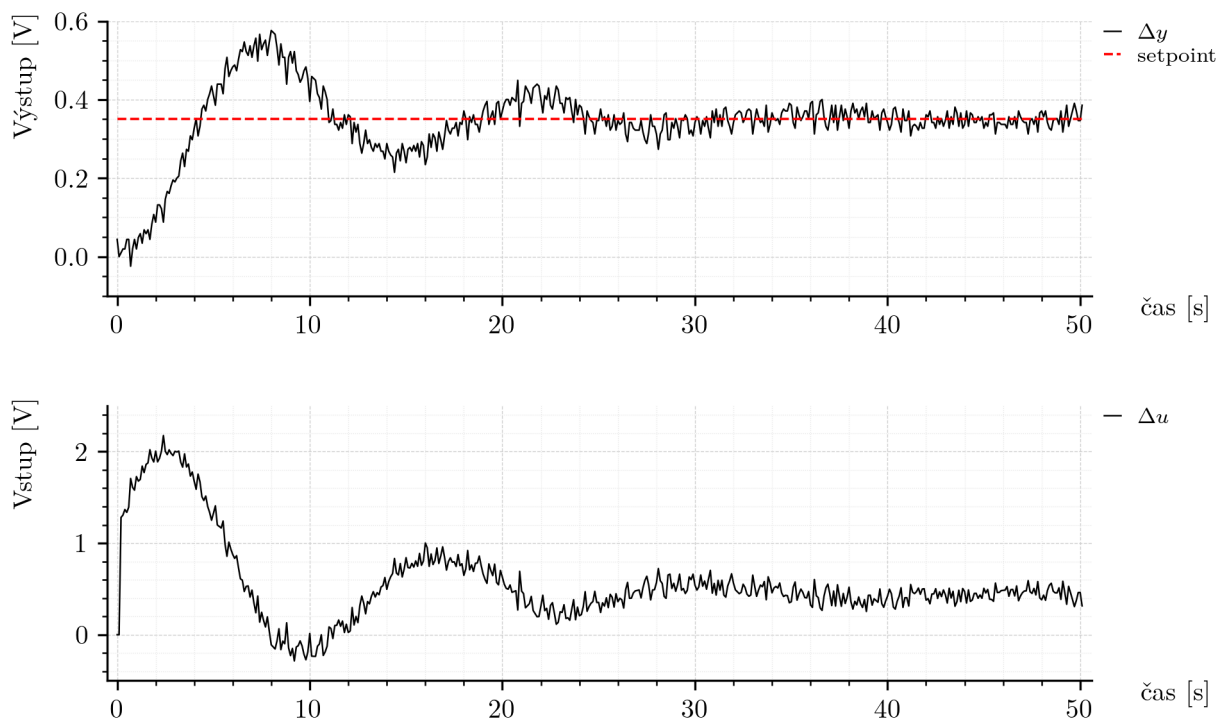
V súlade s princípom implementácie opísaným v podkapitole 2.3.1 blok MATLAB Function ani v tomto prípade neuchováva stavové veličiny automaticky medzi jednotlivými diskretnými krokmi. Preto je potrebné hodnotu integrátora aj odhad pracovného bodu prenášať explicitne cez výstupy bloku a v nasledujúcom kroku ich privádzať späť na jeho vstupy. Týmto spôsobom sa zabezpečí korektné uchovanie vnútorných stavov PI regulátora medzi jednotlivými riadiacimi cyklami.

Logika zapojenia bloku MATLAB Function je zobrazená na obrázku 21, ktorý ukazuje vstupy, výstupy a spätnú väzbu stavových premenných potrebných pre správnu činnosť regulátora.

Pred samotným začiatkom regulácie bol zavedený časový úsek, počas ktorého regulátor nevykonáva žiadny zásah. Počas prvých desiatich minút sa na výstupe drží



Obr. 21: Zapojenie PI regulátora v bloku MATLAB Function so spätnou väzbou stavov



Obr. 22: Experiment s PI regulátorom pri skokovej zmene žiadanej hodnoty

hodnota pracovného bodu a zároveň sa sleduje teplota na výstupe systému. Cieľom je získať reálnu hodnotu ustáleného stavu priamo z aktuálnych podmienok experimentu.

Použitie hodnôt zosadených zo statických charakteristík (obr. 8) nie je vhodné. Tepelný systém je citlivý na teplotu v miestnosti a experimenty nebolo možné realizovať v rovnakých podmienkach. Okrem toho surové merania (obr. 10) ukázali, že teplota narastá veľmi pomaly aj počas dlhého experimentu, čo znamená, že systém nemá pevnú, opakovateľnú hodnotu ustáleného stavu.

Z toho dôvodu je ustálený stav zisťovaný automaticky priamo regulátorom ako posledná nameraná hodnota y pred spustením PI riadenia. Tento prístup funguje bez ohľadu na to, či je systém iba málo zahriaty alebo už dosiahol vysokú prevádzkovú teplotu.

3.3.1 Overenie na reálnom systéme

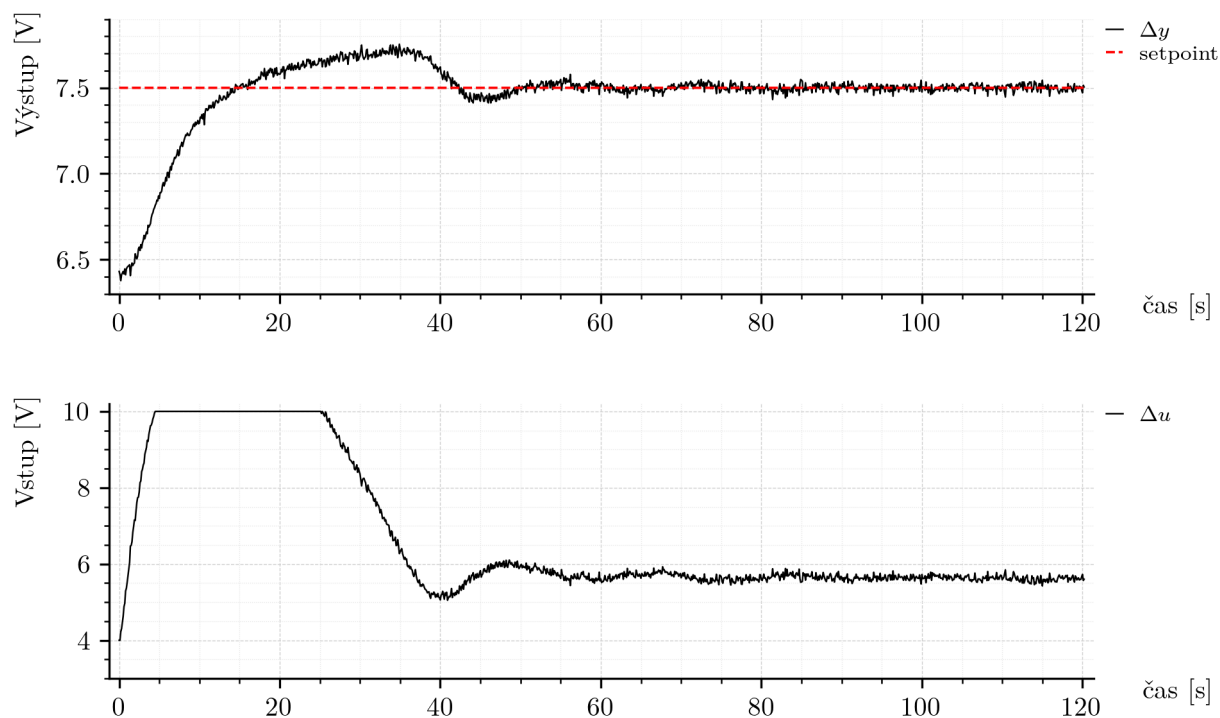
Experiment 1: Regulácia v nominálnom okolí pracovného bodu

Na základe implementácie uvedenej vyššie bol následne vykonaný experiment na reálnom systéme s cieľom overiť kvalitu regulácie. Pribeh regulácie je zobrazený na obrázku 22.

Kvalita regulácie bola hodnotená podľa troch kritérií:

- prekmit: žiadaná hodnota bola 0.35 V, maximálna hodnota výstupu dosiahla približne 0.52 V, čo zodpovedá približne 49.0 percentnému prekmitu,
- čas ustálenia: výstup sa ustálil približne po 28 sekundách,
- trvalá regulačná odchýlka: prakticky nulová.

Pri porovnaní so simuláciou uvedenou v predchádzajúcich kapitolách možno vidieť, že prekmit je výrazne vyšší. Dôvodom je skutočnosť, že identifikované modely nedokázali zachytiť celú dynamiku systému TS05 a viacero pomalých javov, ktoré sa prejavujú až pri reálnej prevádzke. Napriek tomu je regulačný čas veľmi priaznivý vzhľadom na to, že ide o pomalý tepelný systém. Rýchla regulácia bola prioritou, preto možno výsledok považovať za uspokojivý pre riadenie okolo jedného pracovného bodu.



Obr. 23: Experiment PI regulátora pri skoku z 6.4 V na 7.5 V

Experiment 2: Skok želanej hodnoty mimo nominálneho okolia pracovného bodu

Nasledoval druhý experiment, ktorého cieľom bolo overiť správanie regulátora pri väčšej skokovej zmene žiadanej hodnoty, ktorá má simulovať prechod medzi rôznymi pracovnými bodmi systému. Systém bol najprv ustálený v pracovnom bode s výstupnou hodnotou približne 6.4 V a následne bola žiadaná hodnota skokovo zmenená na 7.5 V. Výsledok je zobrazený na obrázku 23.

Z grafu je zrejmé, že počas regulácie došlo k výraznému presýteniu akčného zásahu. Ovládací signál dosiahol svoj limit a počas určitého času na ňom zotrval. Aj keď bola požadovaná hodnota dosiahnutá, takéto správanie je z pohľadu bezpečnosti a kvality regulácie nevyhovujúce.

Z tohto dôvodu možno konštatovať, že navrhnutý PI regulátor je vhodný na riadenie v blízkosti jedného pracovného bodu, avšak nie je vhodný na prepínanie medzi vzdialenejšími pracovnými bodmi. V ďalšej časti práce bude preto implementovaný postup umožňujúci bezpečné a stabilné prepínanie medzi pracovnými bodmi a následne budú jednotlivé regulátory skombinované tak, aby systém disponoval širším využiteľným rozsahom riadenia.

3.4 Implementácia: mikrokontrolér a jazyk C++

Na základe algoritmu uvedeného v prostredí MATLAB bola následne zrealizovaná aj implementácia PI regulátora v jazyku C++. Táto implementácia bola začlenená do hlavného programu uvedeného vo výpise kódu 30 uvedeného v prílohe P.1. Kód prezentovaný v tejto časti zabezpečuje výpočet akčného zásahu pre vykurovaciu špirálu v reálnom čase.

Z hľadiska programovej štruktúry táto implementácia nadväzuje na všeobecný rámec komunikácie a časovania opísaný v podkapitole 2.3.2, pričom v tomto prípade je do neho doplnený konkrétny PI riadiaci zákon.

Použitá logika regulácie zostala z hľadiska riadiaceho princípu rovnaká ako pri implementácii v prostredí MATLAB. Aj v tomto prípade je regulácia realizovaná vzhľadom na pracovný bod, pričom sa využíva odchýlka výstupu od odhadnutého ustáleného stavu a následne sa vypočíta odchýlkový akčný zásah. Rovnako ostali

zachované aj parametre regulátora, hodnota pracovného bodu, saturácia riadiaceho signálu a úvodná čakacia fáza v trvaní desiatich minút, počas ktorej sa systém ustáli a zároveň sa určí hodnota y_{pb} .

Najskôr sú v programe inicializované základné parametre regulátora, pracovného bodu a pomocných premenných:

```
1 float vent = 6.0f;
2 float spirala = 0.0f;
3 float u_pb = 4.0f;
4 float u_pb_min = -u_pb;
5 float u_pb_max = 10 - u_pb;
6 float Kp = 3.58f;
7 float Ki = 1.84;
8 float u = 0.0f;
9 float y_pb = 0.0f;
10 float integrator = 0.0f;
11 float e = 0.0f;
12 float setpoint_pb = 0.35f;
13 float dy = 0.0f;
14 float du = 0.0f;
```

Výpis kódu 11: Inicializácia parametrov PI regulátora v jazyku C++

Samotná regulačná logika je následne realizovaná priamo v hlavnom cykle programu:

```
1 if (globalTime < 10 * 60) {
2     u = u_pb;
3     y_pb = snimac1;
4     integrator = 0.0f;
5 } else {
6     dy = snimac1 - y_pb;
7     e = setpoint_pb - dy;
8
9     integrator += e * timeTickPeriod;
10
11     du = Kp * e + Ki * integrator;
12     du = clampFloat(du, u_pb_min, u_pb_max);
13
14     u = u_pb + du;
15 }
16
17 spirala = u;
```

Výpis kódu 12: Hlavná regulačná logika PI regulátora v jazyku C++

Oproti implementácii v MATLAB Function bloku je však realizácia v jazyku C++ odlišná najmä spôsobom práce so stavmi regulátora. Kým v prostredí Simulink bolo potrebné prenášať stav integrátora a hodnotu pracovného bodu cez vstupy a výstupy bloku explicitne, v programe C++ sú tieto hodnoty uchovávané priamo vo vnútri programu ako interné premenné. Implementácia tak prirodzene disponuje vlastnou vnútornou pamäťou, čo zjednodušuje štruktúru programu a znižuje potrebu vytvárania pomocných spätných väzieb medzi blokmi.

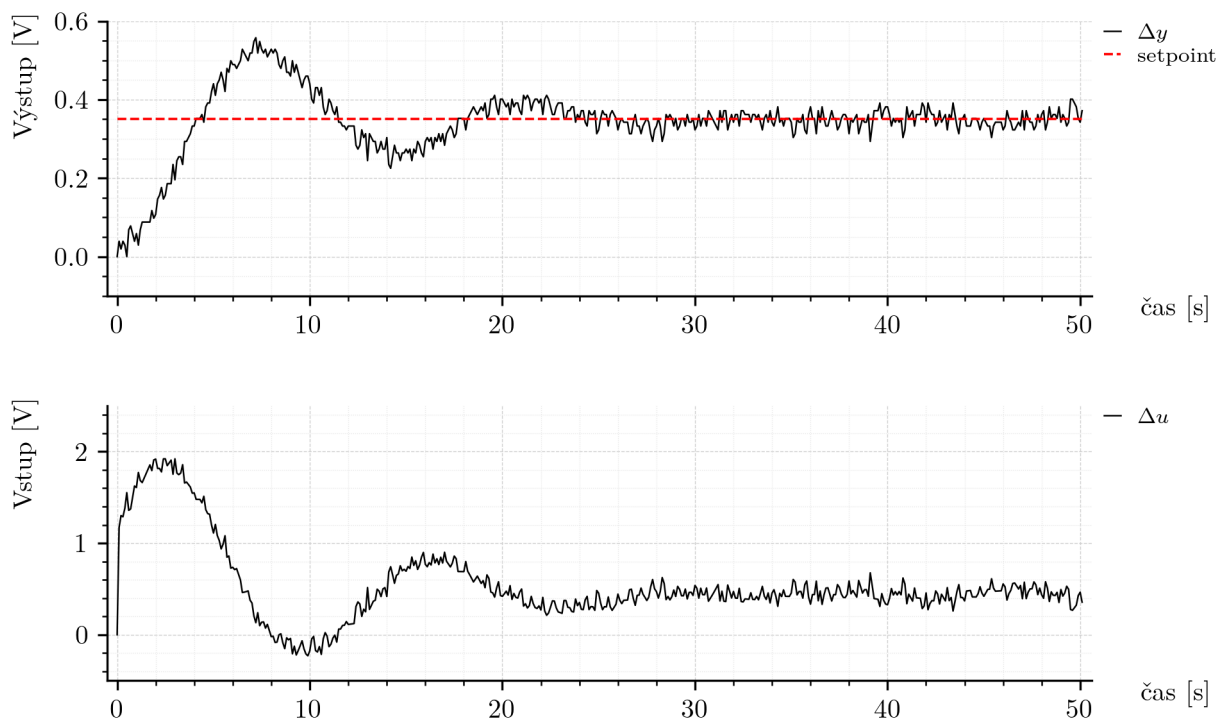
Ďalším rozdielom je spôsob vykonávania algoritmu. V Simulinku je beh regulátora naviazaný na simulačný model a jeho blokovú štruktúru, zatiaľ čo v jazyku C++ je regulačný algoritmus súčasťou hlavného cyklu programu, v ktorom sa postupne vykonáva komunikácia so systémom, čítanie meraných veličín, výpočet regulácie a zápis akčných zásahov na výstupy. Z tohto dôvodu je pri implementácii v C++ potrebné venovať zvýšenú pozornosť správne časovaniu výpočtu, dodržaniu periódy vzorkovania a stabilite komunikácie s hardvérom.

Z numerického hľadiska zostáva princíp výpočtu nezmenený. Integrátor sa aj v tejto implementácii aktualizuje diskretné pomocou periódy vzorkovania a výsledný zásah je následne obmedzený saturáciou.

3.4.1 Overenie na reálnom systéme

Na základe tejto implementácie bol vykonaný experiment na reálnom systéme, ktorého výsledok je uvedený na obrázku 24.

Z priebehu regulácie je zrejmé, že výsledná odozva je veľmi podobná priebehu získanému pri implementácii v prostredí MATLAB, uvedenom na obrázku 22. Tvar



Obr. 24: Experimentálna odozva PI regulátora implementovaného v jazyku C++

regulačnej odozvy, rýchlosť nábehu aj charakter ustálenia zostali vo všeobecnosti zachované. Možno však pozorovať menšie rozdiely v detailoch priebehu, ktoré možno vysvetliť odlišnými vonkajšími podmienkami počas experimentu. Jednotlivé merania boli realizované pri rozdielnej teplote v miestnosti, pričom tepelný systém TS05 je na zmeny okolitej teploty citlivý. Preto možno konštatovať, že implementácia v jazyku C++ potvrdila správnosť navrhnutého algoritmu aj mimo prostredia MATLAB/Simulink a zároveň ukázala, že regulátor si zachováva porovnateľné vlastnosti aj pri nasadení na odlišnej softvérovej platforme.

3.5 Implementácia: programovateľný logický automat (PLC)

Na základe predchádzajúcich implementácií v prostredí MATLAB a v jazyku C++ bola následne realizovaná aj implementácia PI regulátora na platforme Siemens SIMATIC S7-1200 G2. Cieľom tejto časti bolo overiť, či je možné zachovať rovnaký princíp riadenia aj v prostredí priemyselného PLC systému a zároveň dosiahnuť porovnateľné dynamické vlastnosti regulačného obvodu.

Samotný PI algoritmus bol realizovaný vo funkčnom bloku FB_PI_TS05. Týmto spôsobom bolo možné prirodzene využiť vnútornú pamäť funkčného bloku, a teda uchovávať stavové veličiny regulátora medzi jednotlivými vzorkami bez potreby vytvárať externé pomocné štruktúry. Rovnako ako pri ostatných algoritmoch bol k implementácii doplnený aj logovací blok, ktorý zabezpečoval záznam vybraných premenných počas experimentu.

Úvodná čakacia fáza v trvaní 600 sekúnd bola realizovaná pomocou časovača typu TON. Výstup tohto časovača nebol použitý na priame povolovanie samotného funkčného bloku cez jeho štandardný vstup EN, ale iba na aktiváciu pomocnej logickej premennej, ktorá sa následne privádzala na vstup **Enable** riadiaceho algoritmu vo vnútri bloku FB_PI_TS05. Tento detail je z hľadiska PLC implementácie dôležitý. Ak totiž na funkčný blok nie je privedený signál na štandardný vstup EN, blok sa vôbec nevykonáva a nemôže realizovať žiadnu vnútornú logiku. V danom prípade však bolo žiaduce, aby blok pracoval aj počas neaktívnej fázy, teda ešte pred samotným spustením regulácie. Vďaka tomu nebolo potrebné vytvárať dve samostatné implementácie, jednu

pre pasívnu a druhú pre aktívnu fázu, ale celá logika bola sústredená do jediného funkčného bloku.

V neaktívnej fáze blok nastavuje akčný zásah na hodnotu pracovného bodu, nulovú regulačnú odchýlku, nulový odchýlkový zásah a zároveň priebežne aktualizuje odhad pracovného bodu výstupu. Po aktivácii regulácie sa už vykonáva vlastný PI algoritmus v odchýlkovej forme. Implementácia bloku FB_PI_TS05 je uvedená nižšie.

```

1 IF NOT #Enable THEN
2   #U := #U_pb;
3   #DU := 0.0;
4   #E := 0.0;
5   #Integrator := 0.0;
6   #Ypb := #Y;
7   #vent := 6.0;
8 ELSE
9   #E := #Setpoint_pb - (#Y - #Ypb);
10  #Integrator := #Integrator + #E * #Ts;
11  #DU := #Kp * #E + #Ki * #Integrator;
12
13  IF #DU > #DU_max THEN
14    #DU := #DU_max;
15  ELSIF #DU < #DU_min THEN
16    #DU := #DU_min;
17  END_IF;
18
19  #U := #U_pb + #DU;
20  #vent := 6.0;
21 END_IF;

```

Výpis kódu 13: Implementácia funkčného bloku FB_PI_TS05 v PLC Siemens

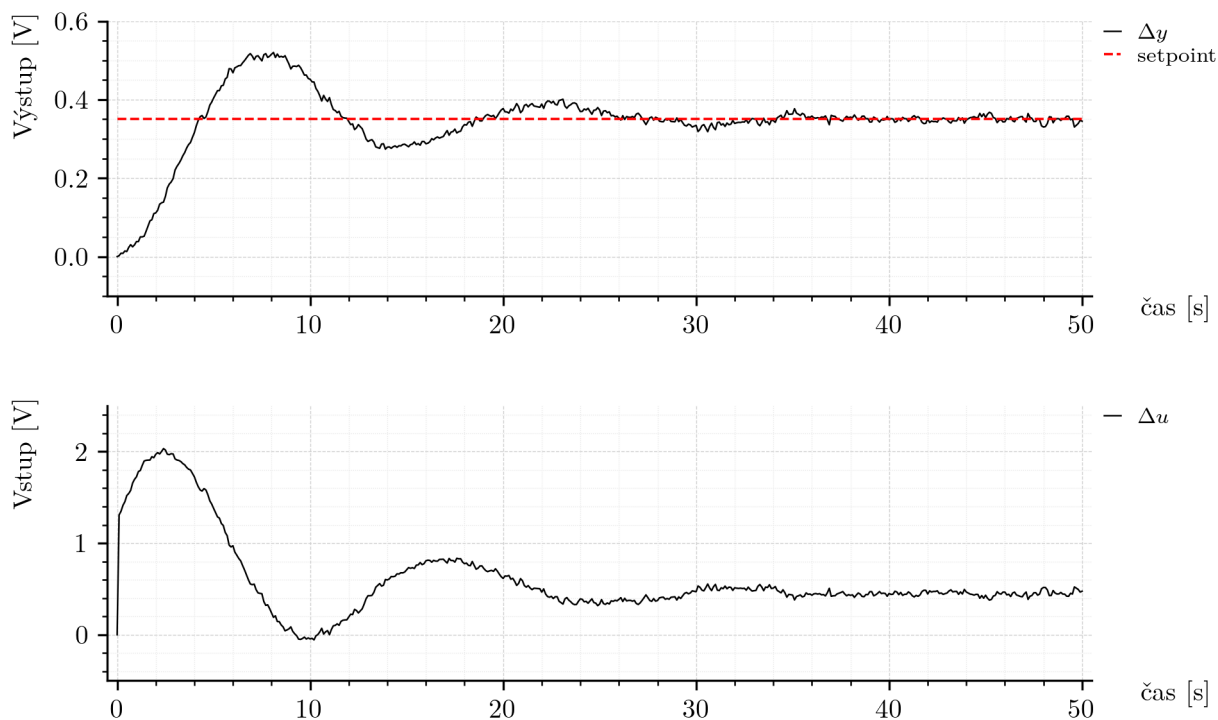
Zoznam interných premenných použitých vo funkčnom bloku je uvedený v tab. 11.

Celkové zapojenie algoritmu v PLC programe, vrátane časovača, riadiaceho bloku a logovania, je uvedené na obr. 26.

Z hľadiska riadiaceho princípu zostáva implementácia zhodná s predchádzajúcimi verziami algoritmu. Aj v tomto prípade sa regulácia vykonáva vzhľadom na pracovný bod, pričom chyba regulácie je počítaná z rozdielu medzi požadovanou odchýlkou a aktuálnou odchýlkou výstupu od pracovného bodu. Integrátor sa aktualizuje diskkrétne

Tabuľka 11: Premenné funkčného bloku FB_PI_TS05

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Input	Y	Real	0.0	Non-retain
Input	Setpoint_pb	Real	0.0	Non-retain
Input	Enable	Bool	false	Non-retain
Output	U	Real	0.0	Non-retain
Output	Ypb	Real	0.0	Non-retain
Output	vent	Real	0.0	Non-retain
Static	Integrator	Real	0.0	Non-retain
Temp	E	Real	–	–
Temp	DU	Real	–	–
Constant	Kp	Real	3.58	–
Constant	Ki	Real	1.84	–
Constant	Ts	Real	0.1	–
Constant	U_pb	Real	4.0	–
Constant	DU_min	Real	–4.0	–
Constant	DU_max	Real	6.0	–



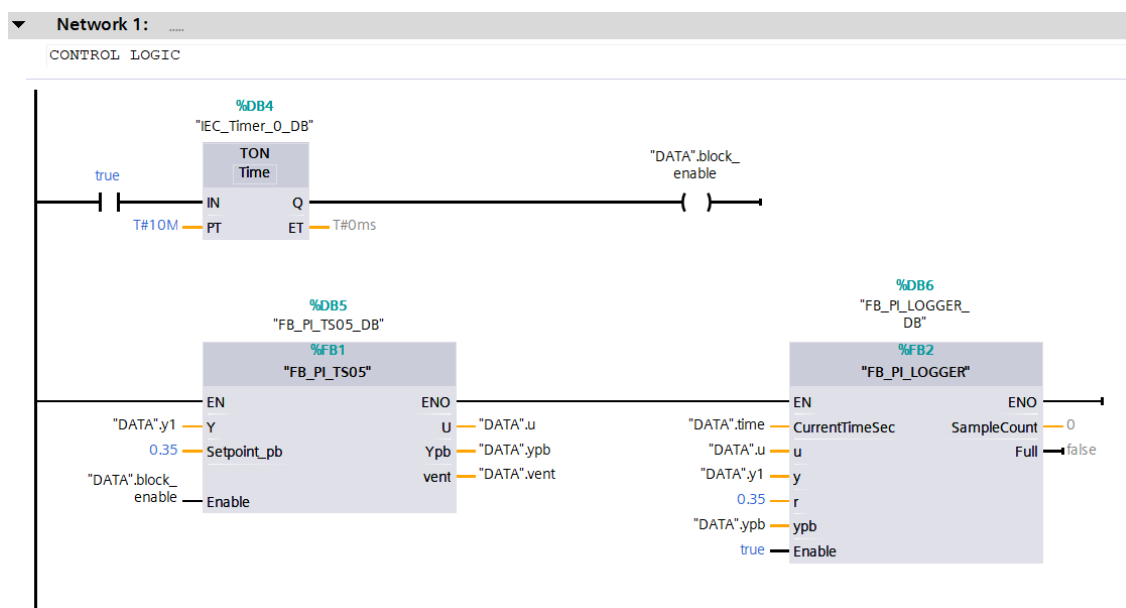
Obr. 25: Experimentálna odozva PI regulátora implementovaného v PLC Siemens

s periódou vzorkovania a výsledný odchýlkový zásah je následne obmedzený saturáciou. Vstup ventilátora zostáva počas celej regulácie nastavený na konštantnej hodnote 6 V.

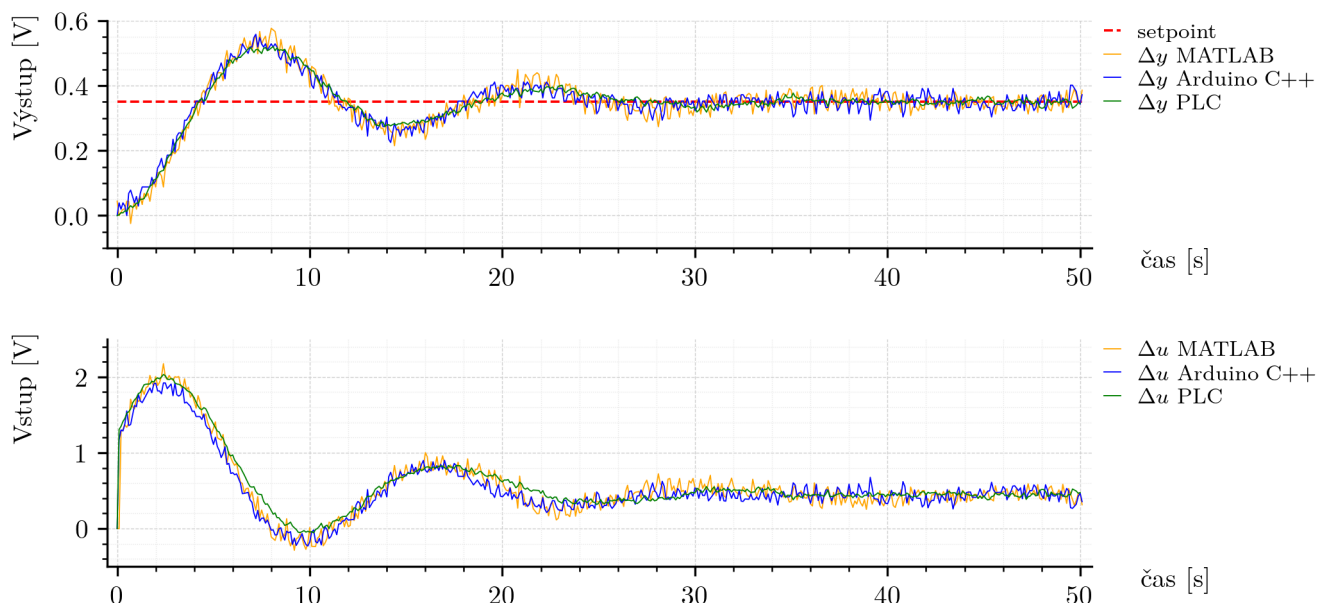
3.5.1 Overenie na reálnom systéme

Na základe tejto implementácie bol vykonaný experiment na reálnom systéme TS05. Výsledok merania je zobrazený na obr. 25.

Z výsledku experimentu je zrejmé, že dynamika regulácie je veľmi podobná tej, ktorá bola pozorovaná pri implementácii v prostredí MATLAB aj v jazyku C++. To potvrdzuje, že navrhnutý algoritmus je prenositeľný aj na platformu PLC bez zásadnej zmeny jeho regulačných vlastností. Zároveň možno pozorovať, že priebeh akčného



Obr. 26: Zapojenie PI regulátora, časovača a logovania v prostredí TIA Portal



Obr. 27: Porovnanie experimentálnych výsledkov PI regulátora na jednotlivých platformách

zásahu je v tomto prípade miernejší a menej zašumený než pri predchádzajúcich implementáciách. Toto správanie môže súvisieť so spôsobom spracovania analógového signálu v použitom PLC module. Ako bolo uvedené v podkapitole 2.3.3, digitálne vyhladzovanie bolo síce vypnuté, avšak funkcia potlačenia rušenia (*noise rejection*) zostáva súčasťou analógového prevodu. Z tohto dôvodu môže byť meraný signál mierne vyhladený už na úrovni samotného vstupného modulu, čo sa následne prejaví aj pokojnejším priebehom akčného zásahu.

Na základe získaných výsledkov možno konštatovať, že implementácia PI regulátora na platforme Siemens PLC bola úspešná a poskytla porovnateľné správanie ako implementácie na ostatných použitých platformách. Tým sa zároveň potvrdilo, že zvolená štruktúra algoritmu je vhodná nielen pre simulačné a mikrokontrolérové prostredie, ale aj pre klasické priemyselné PLC riešenie.

3.6 Porovnanie výsledkov z jednotlivých implementácií

PI regulátor bol implementovaný na všetkých troch uvažovaných platformách: v prostredí MATLAB/Simulink, v jazyku C++ na platforme Arduino a v PLC Siemens. Vo všetkých prípadoch bol zachovaný rovnaký princíp regulácie v okolí pracovného bodu a rovnaké parametre PI regulátora.

Porovnanie experimentálnych výsledkov je uvedené na obr. 27. Z priebehov je zrejmé, že odozvy systému sú na jednotlivých platformách veľmi podobné. Podobnosť je viditeľná nielen na výstupe systému, ale aj na priebehu akčného zásahu vypočítaného regulátorom.

Pre kvantitatívne porovnanie bola vypočítaná hodnota RMSE vzhľadom na príslušnú žiadanú hodnotu každého experimentu:

$$\text{RMSE}_{\text{MATLAB}} = 0.092899 \text{ V}, \quad (3.24)$$

$$\text{RMSE}_{\text{C++}} = 0.085426 \text{ V}, \quad (3.25)$$

$$\text{RMSE}_{\text{PLC}} = 0.085760 \text{ V}. \quad (3.26)$$

Z uvedených hodnôt vyplýva, že rozdiely medzi jednotlivými platformami sú malé. Implementácia PI regulátora tak bola úspešne overená vo všetkých troch uvažovaných prostrediach.

4 Riadiaci systém s prepínaním pre celý rozsah výstupnej veličiny riadeného systému

V tejto časti sa budeme zaoberať návrhom a testovaním riadiacej štruktúry s prepínaním pracovných bodov. Navrhnutá štruktúra pozostáva z globálneho a lokálneho PI regulátora, medzi ktorými rozhoduje logika prepínania. Úlohou globálneho regulátora je zabezpečiť pomalé a robustné prechody medzi pracovnými bodmi systému TS05, zatiaľ čo lokálny regulátor zabezpečuje presnú reguláciu v okolí aktuálneho pracovného bodu. Cieľom je vytvoriť taký spôsob riadenia, ktorý umožní spoľahlivé prechody medzi uvažovanými pracovnými bodmi systému TS05 bez neželaných dynamických javov, ako je presýtenie akčného zásahu alebo príliš veľké prekmitové správanie.

Okrem samotnej prepínacej logiky bola v tejto časti rozšírená aj pracovná oblasť regulácie o dva ďalšie pracovné body systému, a to pre napätia špirály 2 V a 6 V. Pre každý z týchto pracovných bodov bol postup zopakovaný rovnakým spôsobom ako v sekcii 2.2, teda boli:

- vykonané merania reakcií systému na jednotkové skoky ± 0.5 V okolo pracovného bodu,
- z nameraných údajov bola identifikovaná prenosová funkcia v danom pracovnom bode,
- pre každý pracovný bod bol navrhnutý PI regulátor s vlastnými parametrami,
- boli určené hodnoty y_{pb} a rozsah výstupu pre ± 0.5 V zmenu vstupu.

Prehľad všetkých troch pracovných bodov je uvedený v tabuľke 12, kde sú uvedené identifikované prenosové funkcie, hodnoty pracovného bodu y_{pb} a parametre jednotlivých PI regulátorov.

Keďže pôvodné statické charakteristiky systému boli merané ešte počas letného obdobia, bolo rozhodnuté hodnoty y_{pb} pre tieto experimenty zmerať znova, rovnakým spôsobom ako v sekcii 2.1.2. V tabuľke sú preto uvedené aktuálne hodnoty pracovných bodov, ktoré vernejšie reprezentujú stav systému počas meraní v zimnom období.

4.1 Syntéza riadiaceho systému

V tejto časti je prezentovaná riadiaca štruktúra, ktorá pozostáva z dvoch PI regulátorov. Prvý z nich predstavuje globálny PI regulátor, ktorý pracuje s absolútnymi veličinami systému. Druhý predstavuje lokálny PI regulátor, ktorý pracuje s odchýlkami veličín v okolí aktuálneho pracovného bodu.

Lokálny regulátor pracuje s odchýlkami veličín od pracovného bodu

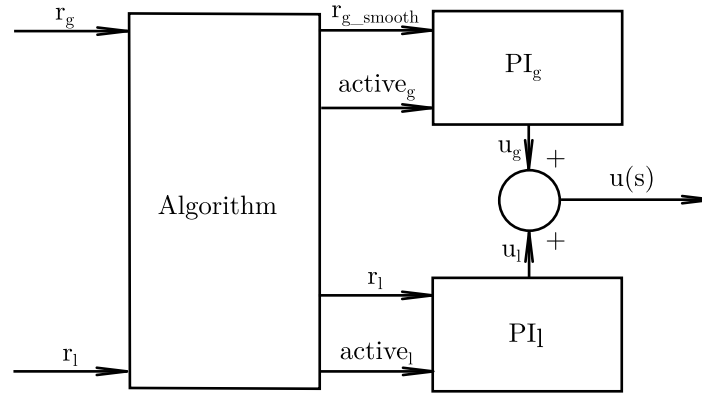
$$\Delta y = y - y_{pb}, \quad \Delta u = u - u_{pb}, \quad (4.1)$$

kde y_{pb} a u_{pb} predstavujú výstupnú a vstupnú hodnotu príslušného pracovného bodu. Parametre lokálneho PI regulátora sú volené pomocou princípu gain scheduling, pričom sa používajú hodnoty uvedené v tabuľke 12.

Globálny regulátor pracuje s absolútnou žiadanou hodnotou. Aby prechod medzi pracovnými bodmi neprebíhal príliš prudko, globálna žiadaná hodnota je pred vstupom

Tabuľka 12: Parametre pracovných bodov, identifikované prenosové funkcie a PI regulátory

u_{pb}	Prenosová funkcia	y_{pb}	Δy_{pb} pri $u_{pb} \pm 0.5$ V	PI parametre
2 V	$\frac{5.79}{s^2 + 44.21s + 7.276}$	4.4884	± 0.41	$K_p = 2.77, K_i = 1.48$
4 V	$\frac{6.074}{s^2 + 59.01s + 8.95}$	6.3412	± 0.35	$K_p = 3.58, K_i = 1.84$
6 V	$\frac{7.161}{s^2 + 52.87s + 21.37}$	7.7441	± 0.18	$K_p = 1.83, K_i = 2.20$



Obr. 28: Bloková schéma riadenia s globálnym a lokálnym PI regulátorom

do regulátora filtrovaná prenosom druhého rádu v tvare

$$G_f(s) = \frac{1}{(10s + 1)^2}. \quad (4.2)$$

Takto zvolený filter zabezpečuje dostatočne plynulý priebeh globálnej žiadanej hodnoty, v dôsledku čoho prechod medzi pracovnými bodmi prebieha pomalšie a s menším rizikom vzniku neželaných prechodových javov.

Parametre globálneho PI regulátora boli zvolené empiricky. Cieľom bolo navrhnuť taký regulátor, ktorý bude dostatočne pomalý a robustný na bezpečný presun medzi vzdialenejšími pracovnými bodmi, no zároveň nie natoľko pomalý, aby boli prechody medzi pracovnými bodmi zbytočne dlhé. Proporcionálna zložka bola preto volená tak, aby zabezpečila primeranú rýchlosť prechodu, zatiaľ čo integračná zložka bola doplnená s cieľom odstrániť trvalú regulačnú odchýlku pri globálnej regulácii. V implementovaných algoritmoch boli použité hodnoty $K_{p,g} = 4$ a $K_{i,g} = 0.7$.

O tom, ktorý PI regulátor bude v danom okamihu aktívny, rozhoduje blok logiky prepínača. Jeho úloha je dvojité. Po prvé určuje, či má byť aktívny globálny alebo lokálny regulátor. Po druhé spracúva globálnu žiadanú hodnotu a vytvára jej filtrovanú verziu pre globálnu reguláciu.

Navrhnutá bloková schéma algoritmu je znázornená na obr. 28.

Logika prepínania je založená na nasledujúcom princípe. Ak je detegovaná skoková zmena globálnej žiadanej hodnoty, aktivuje sa globálny PI regulátor, ktorý zabezpečuje prechod medzi pracovnými bodmi. Ak globálna regulácia nie je aktívna, systém sa automaticky prepne do režimu lokálnej regulácie, v ktorom sa používa PI regulátor naladený pre aktuálny pracovný bod.

V ďalšej časti budú predstavené dve konkrétne realizácie tejto logiky. Budú sa líšiť predovšetkým spôsobom rozhodovania o tom, kedy sa globálna regulácia ukončí a kedy sa aktivuje lokálna regulácia.

V ďalšom texte bude označenie r_g používané pre globálnu žiadanú hodnotu zadávanú v absolútnych veličinách systému, zatiaľ čo r_l bude označovať lokálnu žiadanú hodnotu definovanú ako odchýlku od aktuálneho pracovného bodu. Podobne veličiny y_{pb} a u_{pb} budú predstavovať aktuálne používaný pracovný bod systému, pričom v závislosti od fázy algoritmu môžu nadobúdať buď tabuľkové hodnoty z tabuľky 12, alebo odhadnuté hodnoty získané počas globálnej regulácie po detekcii ustálenia.

Začiatková podmienka integračnej zložky pri prepnutí

Dôležitou požiadavkou navrhnutej štruktúry je zabezpečenie plynulého prepínania medzi globálnym a lokálnym PI regulátorom. Ak by sa po deaktivácii jedného regulátora jeho integračná zložka nechala bez aktualizácie, po opätovnom aktivovaní by mohla mať hodnotu, ktorá nezodpovedá aktuálne realizovanému akčnému zásahu systému. To by viedlo ku skokovej zmene riadenia a k neželanej dynamike pri prepnutí.

Z tohto dôvodu je pri neaktívnom regulátore použitý jednoduchý sledovací mechanizmus integrátora. Jeho úlohou je priebežne nastavovať integračný stav tak, aby po aktivovaní regulátora nedošlo k skokovej zmene jeho výstupu. V prípade lokálneho regulátora sa pri tomto výpočte využíva aj hodnota u_{pb} , ktorá je počas globálnej regulácie zvolená podľa aktuálne uvažovaného pracovného bodu z tabuľky 12. Takto sa dosiahne plynulé prepnutie medzi oboma regulačnými vetvami.

Zákon riadenia

Označme binárne aktivačné signály lokálneho a globálneho regulátora ako a_l a a_g , pričom platí

$$a_l, a_g \in \{0, 1\}. \quad (4.3)$$

Chyba lokálneho regulátora je definovaná vzťahom

$$e_l = r_l - (y - y_{pb}) = r_l - \Delta y, \quad (4.4)$$

kde r_l predstavuje lokálnu žiadanú hodnotu.

Filtrovaná globálna žiadaná hodnota je označená ako $r_{g,f}$, pričom chyba globálneho regulátora je

$$e_g = r_{g,f} - y. \quad (4.5)$$

Lokálny PI regulátor má tvar

$$u_l = u_{pb} + K_{p,l}e_l + K_{i,l}I_l, \quad (4.6)$$

a globálny PI regulátor je definovaný vzťahom

$$u_g = K_{p,g}e_g + K_{i,g}I_g. \quad (4.7)$$

Keďže v každom okamihu je aktívna práve jedna regulačná vetva, možno zákon riadenia zapísať v tvare

$$u = \begin{cases} u_{pb} + K_{p,l}e_l + K_{i,l}I_l, & \text{ak } a_l = 1, a_g = 0, \\ K_{p,g}e_g + K_{i,g}I_g, & \text{ak } a_l = 0, a_g = 1. \end{cases} \quad (4.8)$$

Pri aktívnom regulátore sa integračný stav aktualizuje štandardne na základe regulačnej chyby. Pri neaktívnom regulátore sa integrátor priamo dopočíta z aktuálneho akčného zásahu, aby bol pripravený na plynulé prevzatie riadenia.

Pre lokálny regulátor sa pri neaktívnom stave používa vzťah

$$I_l = \frac{(u - u_{pb}) - K_{p,l}e_l}{K_{i,l}}, \quad a_l = 0, \quad (4.9)$$

a pre globálny regulátor analogicky

$$I_g = \frac{u - K_{p,g}e_g}{K_{i,g}}, \quad a_g = 0. \quad (4.10)$$

Uvedený spôsob sledovania integrátora je z teoretického hľadiska potrebný predovšetkým v poslednom kroku pred opätovnou aktiváciou príslušného regulátora, pretože práve vtedy zabezpečuje vhodnú počiatočnú hodnotu integračnej zložky pre plynulé prevzatie riadenia. Z implementačného hľadiska je však výhodné vykonávať tento výpočet priebežne počas celej neaktívnej fázy, keďže sa tým zjednoduší štruktúra algoritmu a odpadá potreba osobitne detegovať okamih tesne pred prepnutím regulačnej vetvy.

V prípade lokálneho regulátora však možno využiť aj ďalšie zjednodušenie. Keďže lokálna regulácia pracuje v okolí pracovného bodu, pri korektnom prepnutí do lokálneho režimu možno očakávať, že bude približne platiť $u \approx u_{pb}$ a zároveň $e_l \approx 0$. V ideálnom prípade je preto počiatočná hodnota integračnej zložky lokálneho PI regulátora veľmi blízka nule. Z tohto dôvodu možno pri praktickej implementácii lokálny integrátor

aproximovať aj jednoduchým vynulovaním, zatiaľ čo sledovanie integrátora zostáva dôležité najmä pri globálnej regulačnej vetve.

Takto definovaný princíp zabezpečuje, že pri prepnutí medzi lokálnou a globálnou regulačnou vetvou nedochádza k nežiaducej skokovej zmene akčného zásahu.

4.1.1 Globálna regulácia po určitý čas

Prvou uvažovanou realizáciou prepínacej logiky je regulácia založená na pevne zvolenom čase pôsobenia globálneho PI regulátora. Predpokladá sa pritom, že na prechod z jedného pracovného bodu do druhého postačuje približne konštantný čas. Na základe experimentálnych pozorovaní bol tento čas určený ako

$$T_{sw} = 80 \text{ s.} \quad (4.11)$$

Pri skokovej zmene globálnej žiadanej hodnoty sa teda aktivuje globálny PI regulátor, ktorý zostáva aktívny počas intervalu T_{sw} . Po uplynutí tohto času sa globálna vetva deaktivuje a riadenie automaticky preberá lokálny PI regulátor. Pri lokálnej regulácii sa používajú hodnoty u_{pb} , y_{pb} a parametre PI regulátora z tabuľky 12.

Filter globálnej žiadanej hodnoty z rovnice (4.2) je pre potreby diskretnej implementácie vhodné previesť do stavového priestoru. Pre prenos

$$G_f(s) = \frac{R_{g,f}(s)}{R_g(s)} = \frac{1}{(10s+1)^2} = \frac{0.01}{s^2 + 0.2s + 0.01} \quad (4.12)$$

zvolíme stavový vektor

$$x_f(t) = \begin{bmatrix} x_{f1}(t) \\ x_{f2}(t) \end{bmatrix}. \quad (4.13)$$

V kanonickom tvare má filter podobu

$$\dot{x}_f(t) = A_f x_f(t) + b_f r_g(t), \quad (4.14)$$

$$r_{g,f}(t) = c_f^T x_f(t), \quad (4.15)$$

kde

$$A_f = \begin{bmatrix} 0 & 1 \\ -0.01 & -0.2 \end{bmatrix}, \quad b_f = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_f = \begin{bmatrix} 0.01 \\ 0 \end{bmatrix}. \quad (4.16)$$

Pri Eulerovej diskretizácii so vzorkovacou periódou T_s má stavová rovnica tvar

$$x_f(k+1) = x_f(k) + T_s (A_f x_f(k) + b_f r_g(k)), \quad (4.17)$$

a výstup filtra je určený vzťahom

$$r_{g,f}(k) = c_f^T x_f(k). \quad (4.18)$$

Z implementačného hľadiska je preto potrebné prenášať medzi jednotlivými krokmi stav filtra globálnej žiadanej hodnoty, integračné stavy oboch PI regulátorov a predchádzajúcu hodnotu globálnej žiadanej hodnoty. V prípade časovo riadeného prepínania je zároveň potrebné poznať čas aktivácie globálnej regulačnej vetvy, pričom aktuálny čas je dostupný ako vstup algoritmu. Okrem toho je potrebné mať k dispozícii tabuľkové hodnoty pracovných bodov y_{pb} , u_{pb} a príslušné parametre lokálnych PI regulátorov, keďže lokálna regulačná vetva je realizovaná s využitím gain scheduling.

Pseudokód tejto realizácie je uvedený nižšie.

```
% Inputs:
% t(k) - current time
% r_g(k) - global reference
% r_l(k) - local reference
% y(k) - plant output
%
% States:
```

```

% x_f(2x1)      - state of global reference filter
% I_l           - integral state of local PI controller
% I_g           - integral state of global PI controller
% t_g_start     - activation time of global mode
% r_g_prev      - previous global reference
% active_g      - activity flag of global controller
%
% Parameters:
% Ts, T_sw      - global PI parameters
% Kp_g, Ki_g    - global PI parameters
%
% Tables:
% y_pb_table, u_pb_table
% Kp_l_table, Ki_l_table

% Filter matrices
A_f = [ 0, 1;
        -0.01, -0.2 ];

b_f = [ 0;
        1 ];

c_f = [ 0.01, 0 ];

% -----
% 1. Detect step change of global reference
% -----
if r_g ~= r_g_prev
    active_g = 1;
    t_g_start = t;
end

% -----
% 2. Filter global reference
% -----
r_gf = c_f * x_f;
x_f_next = x_f + Ts * ( A_f * x_f + b_f * r_g );

% -----
% 3. Select local operating point and PI parameters from table
% -----
select y_pb, u_pb, Kp_l, Ki_l from table according to current operating point

% -----
% 4. Switching logic
% -----
if active_g == 1
    if ( t - t_g_start ) >= T_sw
        active_g = 0;
    end
end

active_l = 1 - active_g;

% -----
% 5. Control errors
% -----
e_l = r_l - ( y - y_pb );
e_g = r_gf - y;

% -----
% 6. Control law
% -----
u_l = active_l * ( u_pb + Kp_l * e_l + Ki_l * I_l );
u_g = active_g * ( Kp_g * e_g + Ki_g * I_g );
u = u_l + u_g;

% -----
% 7. Integral states update
% -----
if active_l == 1
    I_l_next = I_l + Ts * e_l;
else
    I_l_next = ( (u - u_pb) - Kp_l * e_l ) / Ki_l;
end

if active_g == 1
    I_g_next = I_g + Ts * e_g;

```

```

else
    I_g_next = ( u - Kp_g * e_g ) / Ki_g;
end

% -----
% 8. Memory update
% -----
r_g_prev_next = r_g;
t_g_start_next = t_g_start;

```

Výpis kódu 14: Pseudokód prepínania s globálnou reguláciou po určitý čas

Uvedený spôsob prepínania je založený na predpoklade, že globálny PI regulátor potrebuje na presun systému do nového pracovného bodu približne konštantný čas. Takýto prístup je implementačne jednoduchý, avšak jeho nevýhodou je, že dynamika systému nie je v reálnych podmienkach úplne konštantná. Mení sa okrem iného aj v závislosti od aktuálnych podmienok okolia, napríklad od teploty v miestnosti. To znamená, že hodnoty u_{pb} a y_{pb} uvedené v tabuľke 12 nemusia v danom experimente presne zodpovedať skutočnému ustálenému stavu systému.

Rovnako nie je zaručené, že za pevne stanovený čas 80 s sa systém vždy ustáli dostatočne blízko požadovaného pracovného bodu. Z tohto dôvodu bol navrhnutý aj druhý spôsob prepínania, ktorý je založený priamo na detekcii ustálenia.

Časovo riadené prepínanie tak predstavuje najmä jednoduchší referenčný variant, na základe ktorého bolo možné ilustrovať základný princíp navrhnutej štruktúry. Pre finálnu implementáciu však bol uprednostnený prístup založený na detekcii ustálenia, ktorý lepšie zodpovedá reálnemu správaniu systému.

4.1.2 Prepínanie na základe detekcie ustálenia v okolí želaného pracovného bodu

Druhá uvažovaná realizácia prepínacej logiky nie je založená na predpoklade, že ustálenie nastane po vopred známom čase. Namiesto toho sa globálna regulácia ukončí až vtedy, keď sa na základe meraných údajov zistí, že systém sa skutočne ustálil.

V tejto štruktúre má globálny PI regulátor úlohu pomalého a robustného regulátora, ktorý zabezpečuje prechod medzi pracovnými bodmi v rámci celého pracovného rozsahu systému. Lokálny PI regulátor má naopak za úlohu presnú reguláciu v okolí aktuálneho pracovného bodu, teda v oblasti približne $u_{pb} \pm 0.5$ V. Z tohto dôvodu pracuje s odchýlkami veličín od pracovného bodu a jeho parametre sa volia pomocou gain scheduling na základe tabuľkových hodnôt pracovných bodov.

Pri tejto metóde sa okrem stavu filtra, integračných stavov a predchádzajúcej hodnoty globálnej žiadanej hodnoty uchováva aj posledných 30 hodnôt výstupu y a posledných 30 hodnôt vstupu u . Globálna regulácia sa opäť aktivuje pri skoku globálnej žiadanej hodnoty, avšak jej deaktivácia nastane až po splnení podmienky ustálenia.

Za ustálený stav sa v tomto prípade považuje situácia, keď sa aspoň 25 z posledných 30 hodnôt výstupu nachádza v intervale

$$\langle r_g - 0.03 r_g, r_g + 0.03 r_g \rangle. \quad (4.19)$$

Po splnení tejto podmienky sa ako aktuálna hodnota pracovného bodu určí aritmetický priemer posledných 30 hodnôt výstupu a vstupu, teda

$$y_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} y_i, \quad u_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} u_i. \quad (4.20)$$

Takto získané hodnoty reprezentujú odhad skutočného pracovného bodu, ktorý bol dosiahnutý globálnou reguláciou. Následne sa vyberú parametre lokálneho PI regulátora podľa toho, ku ktorému pracovnému bodu z tabuľky 12 je hodnota y_{pb}^* najbližšia.

Pseudokód tejto realizácie je uvedený nižšie.

```

% Inputs:
% t(k) - current time
% r_g(k) - global reference

```

```

% r_l(k) - local reference
% y(k) - plant output
%
% States:
% x_f(2x1) - state of global reference filter
% I_l - integral state of local PI controller
% I_g - integral state of global PI controller
% y_hist(30) - history of output values
% u_hist(30) - history of input values
% r_g_prev - previous global reference
% active_g - activity flag of global controller
%
% Parameters:
% Ts
% Kp_g, Ki_g - global PI parameters
%
% Tables:
% y_pb_table, u_pb_table
% Kp_l_table, Ki_l_table

% Filter matrices
A_f = [ 0, 1;
        -0.01, -0.2 ];

b_f = [ 0;
        1 ];

c_f = [ 0.01, 0 ];

% -----
% 1. Detect step change of global reference
% -----
if r_g ~= r_g_prev
    active_g = 1;
end

% -----
% 2. Filter global reference
% -----
r_gf = c_f * x_f;
x_f_next = x_f + Ts * ( A_f * x_f + b_f * r_g );

% -----
% 3. Update histories
% -----
shift y_hist and append y(k)
shift u_hist and append u(k)

% -----
% 4. Select local operating point and PI parameters from table
% -----
select y_pb, u_pb, Kp_l, Ki_l from table according to current operating point

% -----
% 5. Control errors
% -----
e_l = r_l - ( y - y_pb );
e_g = r_gf - y;

% -----
% 6. Control law
% -----
u_l = (1 - active_g) * ( u_pb + Kp_l * e_l + Ki_l * I_l );
u_g = active_g * ( Kp_g * e_g + Ki_g * I_g );
u = u_l + u_g;

% -----
% 7. Settling detection
% -----
count_ok = number of elements in y_hist within r_g +/- 3%

if active_g == 1 && count_ok >= 25
    y_pb_est = mean( y_hist );
    u_pb_est = mean( u_hist );
    select local PI parameters corresponding to nearest tabulated y_pb
    set y_pb = y_pb_est;
    set u_pb = u_pb_est;
    active_g = 0;
end

```

```

end

active_l = 1 - active_g;

% -----
% 8. Integral states update
% -----
if active_l == 1
    I_l_next = I_l + Ts * e_l;
else
    I_l_next = ( (u - u_pb) - Kp_l * e_l ) / Ki_l;
end

if active_g == 1
    I_g_next = I_g + Ts * e_g;
else
    I_g_next = ( u - Kp_g * e_g ) / Ki_g;
end

% -----
% 9. Memory update
% -----
r_g_prev_next = r_g;

```

Výpis kódu 15: Pseudokód prepínania na základe detekcie ustálenia

Táto metóda je z výpočtového hľadiska zložitejšia, pretože vyžaduje uchovávanie histórie signálov a priebežné vyhodnocovanie podmienky ustálenia. Na druhej strane je však podstatne robustnejšia, pretože nepredpokladá fixný čas ustálenia, ale vychádza zo skutočného správania systému.

Keďže perióda regulácie je $T_s = 0.1$ s, čo predstavuje z pohľadu použitého hardvéru dostatočne veľký čas na vykonanie potrebných výpočtov, bolo rozhodnuté implementovať práve tento prístup. V porovnaní s časovo riadeným prepínaním poskytuje spoľahlivejšie ukončenie globálnej regulácie a lepšie zohľadňuje zmeny dynamiky systému spôsobené reálnymi prevádzkovými podmienkami.

4.2 Realizácia riadiaceho systému v prostredí MATLAB/Simulink

Navrhnutý algoritmus prepínania pracovných bodov bol implementovaný v prostredí MATLAB/Simulink pomocou bloku MATLAB Function. Celá regulačná logika bola realizovaná explicitne vo forme vlastného kódu, bez použitia preddefinovaných blokov.

Keďže blok MATLAB Function neuchováva automaticky vnútorné stavy, všetky potrebné premenné (stav filtra, integračné stavy, príznak aktivity, história signálov a predchádzajúca hodnota žiadanej veličiny) boli prenášané medzi krokmi ako vstupno-výstupné signály.

Schéma implementácie je uvedená na obr. 29.

Implementovaný kód je uvedený v nasledujúcom výpise.

```

1 function [u, x_f_new, I_l_new, I_g_new, y_hist_new, u_hist_new, ...
2     r_g_prev_new, active_g_new, y_pb_new, u_pb_new, r_gf, Kp_l, Ki_l, u_l,
3     u_g] = ...
4     pi_switch(r_g, r_l, y, x_f, I_l, I_g, y_hist, u_hist, ...
5         r_g_prev, active_g, y_pb, u_pb)
6
7 Ts = 0.1;
8
9 % Global PI
10 Kp_g = 4;
11 Ki_g = 0.7;
12
13 % Gain scheduling table
14 y_pb_table = [4.4884; 6.3412; 7.7441];
15 u_pb_table = [2; 4; 6];
16 Kp_l_table = [2.77; 3.58; 1.83];
17 Ki_l_table = [1.48; 1.84; 2.2];
18
19 % Filter matrices
20 A_f = [0, 1;
21     -0.01, -0.2];
22 b_f = [0;

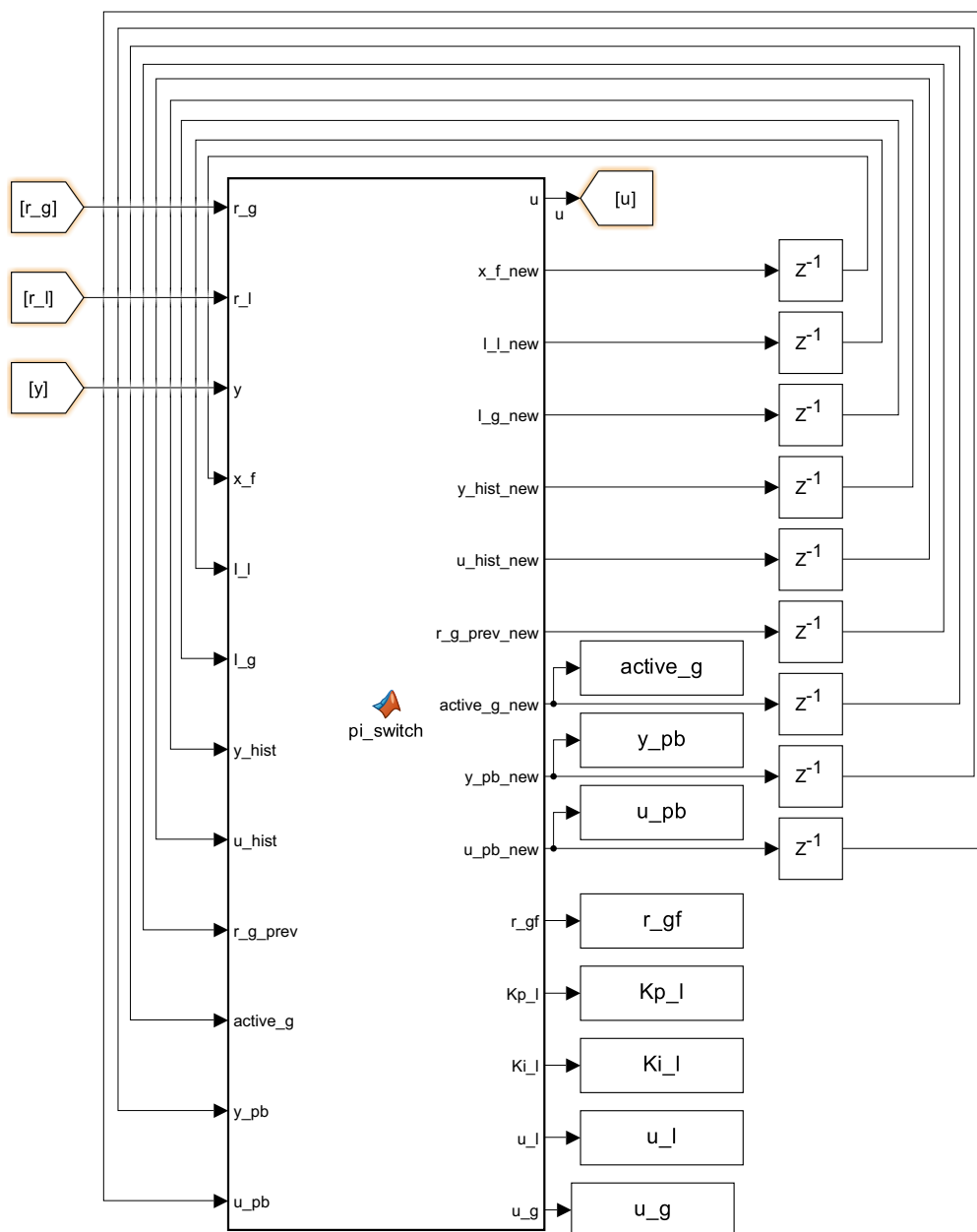
```

```

23     1];
24
25     c_f = [0.01, 0];
26
27     % 1 Detect step change
28     if r_g ~= r_g_prev
29         active_g = 1;
30     end
31
32     % 2 Filter global reference
33     r_gf = c_f * x_f;
34     x_f_new = x_f + (A_f * x_f + b_f * r_g) * Ts;
35
36     % 3 Select local points PI params
37     dist_pb = abs(y_pb_table - y_pb);
38     [~, idx_pb] = min(dist_pb);
39
40     Kp_l = Kp_l_table(idx_pb);
41     Ki_l = Ki_l_table(idx_pb);
42
43     % 4 Control errors
44     e_l = r_l - (y - y_pb);
45     e_g = r_gf - y;
46
47     % 5 Control law
48     u_l = (1 - active_g) * (u_pb + Kp_l * e_l + Ki_l * I_l);
49     u_g = active_g * (Kp_g * e_g + Ki_g * I_g);
50     u = u_l + u_g;
51     u = max(min(u, 10), 0);
52
53     % 6 Update histories
54     y_hist_new = [y_hist(2:end); y];
55     u_hist_new = [u_hist(2:end); u];
56
57     % 7 Settling detection
58     lower_bound = r_g - 0.03 * r_g;
59     upper_bound = r_g + 0.03 * r_g;
60
61     count_ok = 0;
62     for i = 1:30
63         if (y_hist_new(i) >= lower_bound) && (y_hist_new(i) <= upper_bound)
64             count_ok = count_ok + 1;
65         end
66     end
67
68     y_pb_new = y_pb;
69     u_pb_new = u_pb;
70     active_g_new = active_g;
71
72     if active_g == 1 && count_ok >= 25
73         y_pb_est = mean(y_hist_new);
74         u_pb_est = mean(u_hist_new);
75
76         y_pb_new = y_pb_est;
77         u_pb_new = u_pb_est;
78
79         active_g_new = 0;
80     end
81
82     active_l = 1 - active_g;
83
84     % 8 Integral states update
85     if active_l == 1
86         I_l_new = I_l + e_l * Ts;
87     else
88         I_l_new = 0;
89     end
90
91     if active_g == 1
92         I_g_new = I_g + e_g * Ts;
93     else
94         I_g_new = (u - Kp_g * e_g) / Ki_g;
95     end
96
97     % 10 Memory update
98     r_g_prev_new = r_g;

```

Výpis kódu 16: Implementácia prepínacej logiky v bloku MATLAB Function



Obr. 29: Implementácia prepínacej logiky v prostredí MATLAB/Simulink

Pri prepnutí z globálnej na lokálnu reguláciu bola v implementácii integračná zložka lokálneho PI regulátora zjednodušená vynulovaná. Toto zjednodušenie vychádza z predpokladu, že v okamihu prepnutia systém už pracuje v blízkosti zvoleného pracovného bodu, takže približne platí $u \approx u_{pb}$ a zároveň $e_l \approx 0$. V takom prípade je ideálna počiatočná hodnota lokálneho integrátora blízka nule, a preto je takéto zjednodušenie pre praktickú implementáciu prijateľné.

4.2.1 Simulačné overenie

Funkčnosť navrhnutého algoritmu bola overená simuláciou na modeli systému identifikovanom v predchádzajúcich častiach, konkrétne na prenose (2.12).

V simulácii boli realizované dva prechody medzi pracovnými bodmi, pričom v každom z nich bola následne vykonaná aj zmena lokálnej žiadanej hodnoty. Cieľom

bolo overiť správanie algoritmu pri prechode medzi pracovnými bodmi a zároveň jeho presnosť v lokálnom režime.

Na obr. 30 sú zobrazené priebehy jednotlivých veličín.

Na prvom grafe sú znázornené výstup systému y , hodnota pracovného bodu y_{pb} a celková žiadaná hodnota $r_{g,f} + r_l$. Je možné pozorovať mierny offset výstupu voči žiadanej hodnote, ktorý je spôsobený toleranciou 3 % použitou pri detekcii ustálenia. Po ustálení sa však výstup presne viaže na hodnotu y_{pb} .

Druhý graf zobrazuje priebehy r_l , r_g a $r_{g,f}$. Je zrejmé, že globálna žiadaná hodnota je filtrovaná, čo vedie k plynulejšiemu priebehu prechodov. Zároveň je vidieť, že lokálna žiadaná hodnota začína od nulovej hodnoty, keďže je definovaná ako odchýlka od pracovného bodu.

Na treťom grafe je znázornený celkový akčný zásah u .

Štvrtý graf obsahuje jednotlivé zložky u_g , u_l a u_{pb} . V aktívnych fázach regulácie je viditeľné, že príslušná regulačná vetva generuje rovnaký priebeh ako výsledný signál u . Zároveň možno pozorovať, že po ukončení globálnej regulácie a pred vznikom odchýlky v lokálnej žiadanej hodnote sú všetky tri priebehy zhodné, čo zodpovedá princípu navrhnutého algoritmu.

Na piatom grafe sú zobrazené parametre lokálneho PI regulátora, čím je demonštrovaná funkčnosť princípu gain scheduling.

Posledný graf zobrazuje aktivačný signál globálneho regulátora. Hodnota 1 znamená aktívnu globálnu reguláciu, zatiaľ čo hodnota 0 zodpovedá lokálnej regulácii.

4.2.2 Overenie na reálnom systéme

Navrhnutý algoritmus bol následne overený aj na reálnom systéme TS05. Výsledky experimentu sú uvedené na obr. 31.

Správanie systému je kvalitatívne veľmi podobné výsledkom získaným simuláciou. Aj v tomto prípade je možné pozorovať mierny offset výstupu voči celkovej žiadanej hodnote, ktorý je spôsobený použitou toleranciou 3 % pri detekcii ustálenia.

Na priebehoch akčného zásahu je viditeľné, že jednotlivé zložky u_g a u_l v aktívnych fázach regulácie reprodukovujú výsledný signál u , čo potvrdzuje správnu funkciu prepínacej logiky.

Na rozdiel od simulácie je však zrejmé, že aj po ustálení systému v pracovnom bode nie je akčný zásah konštantný. Dôvodom sú vonkajšie vplyvy, najmä malé zmeny teploty v okolí systému. Aj v ustálenom stave je preto potrebné neustále jemne upravovať akčný zásah, aby bol pracovný bod udržiavaný.

Zvyšné priebehy a dynamika systému sú veľmi podobné simulovanému prípadu, čo potvrdzuje správnosť navrhnutého algoritmu.

Pre detailnejšie posúdenie správania systému po ukončení globálnej regulácie je na obr. 32 zobrazený detail priebehov výstupu.

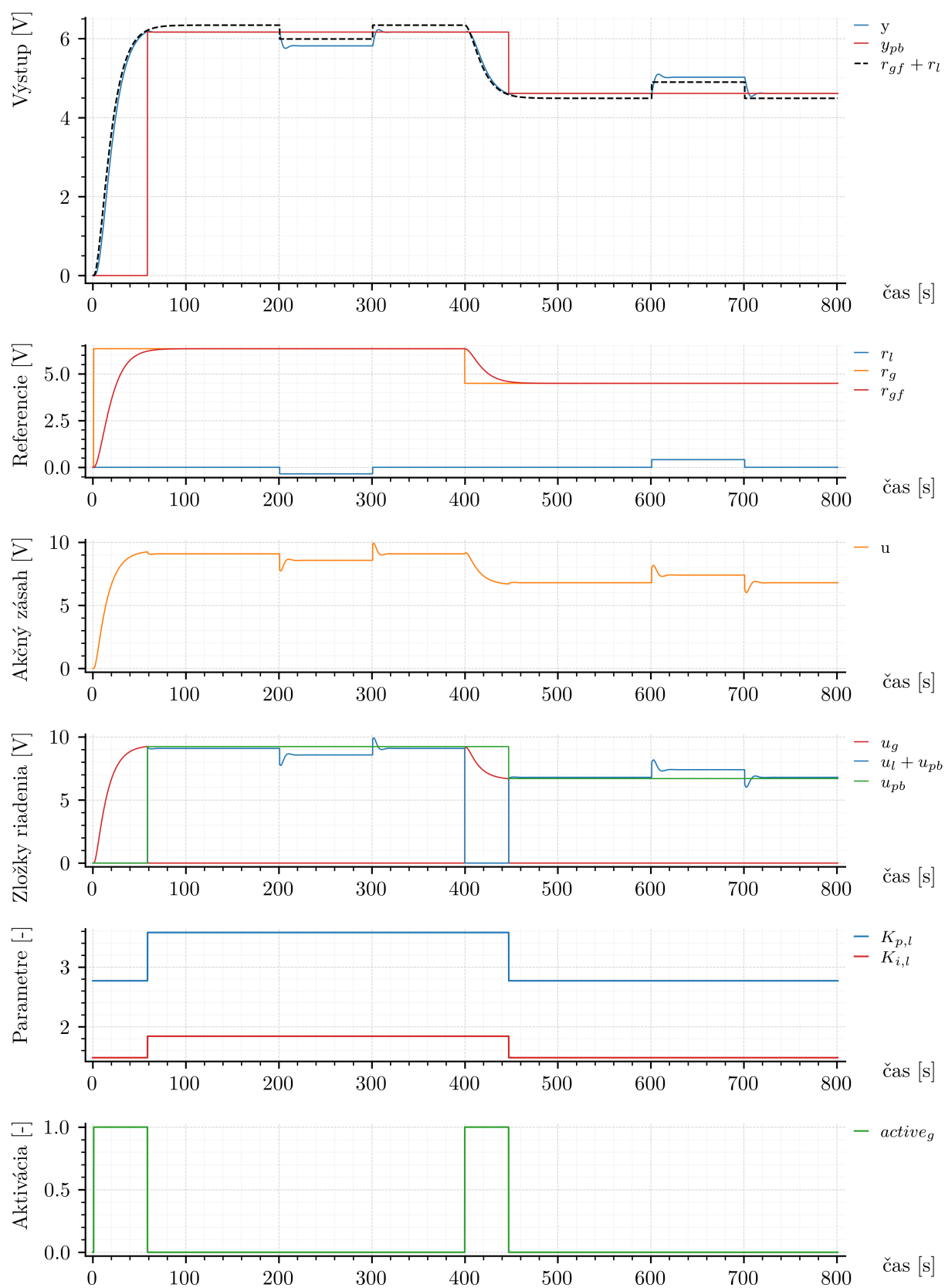
Z detailu je zrejmé, že výstup systému y sa presnejšie viaže na hodnotu $y_{pb} + r_l$ ako na globálnu žiadanú hodnotu. To znamená, že systém interpretuje y_{pb} ako referenčný pracovný bod, okolo ktorého prebieha lokálna regulácia.

Zelenou oblasťou je na grafe vyznačený interval $r_g \pm 3\%$, ktorý predstavuje podmienku ustálenia. Je viditeľné, že hodnota y_{pb} sa nachádza práve v tomto intervale, čo potvrdzuje, že určenie pracovného bodu nie je náhodné, ale priamo vyplýva z definovanej detekcie ustálenia.

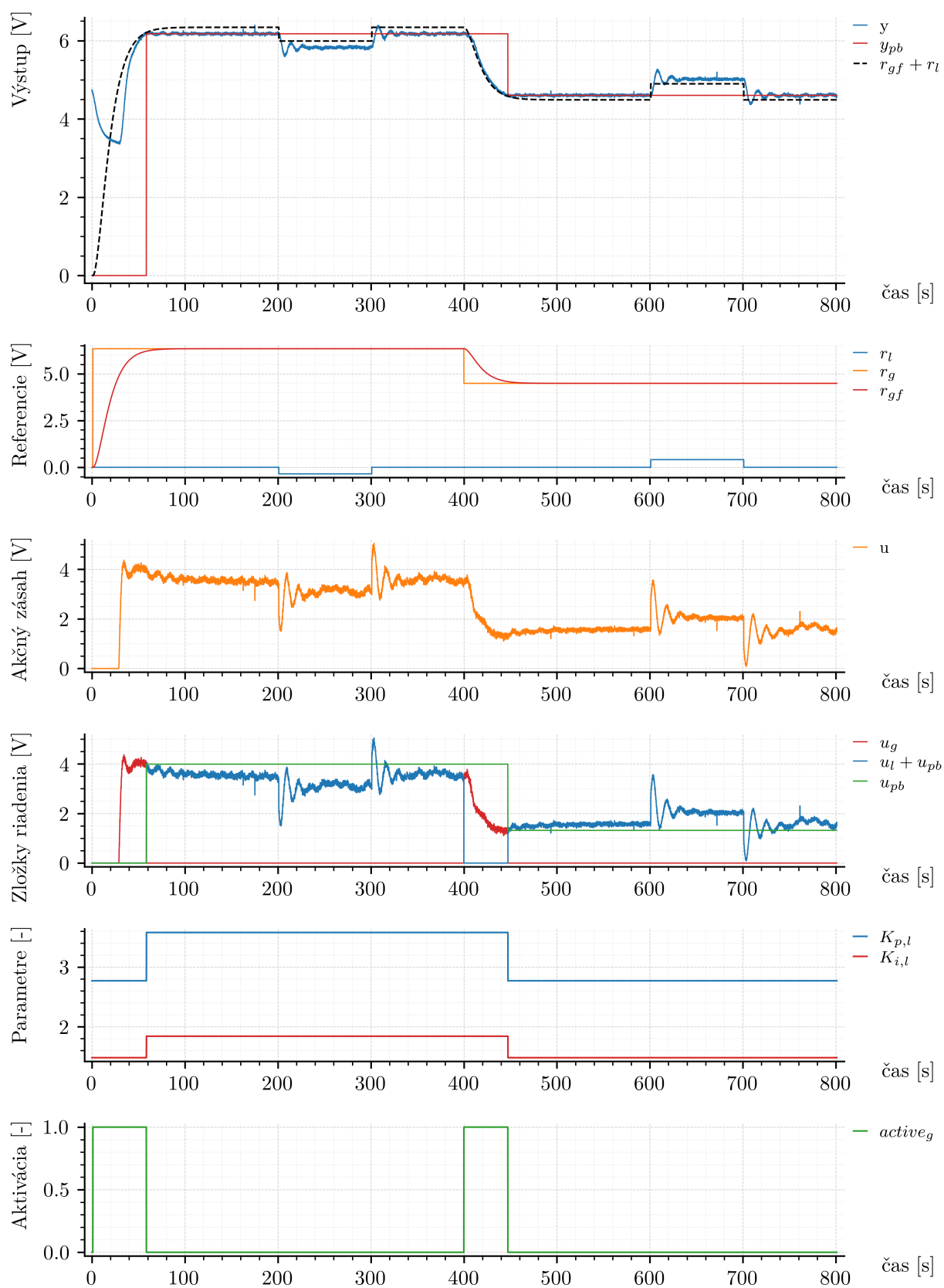
Výsledky experimentu tak potvrdzujú, že navrhnutý algoritmus dokáže spoľahlivo realizovať prechody medzi pracovnými bodmi a následne zabezpečiť presnú lokálnu reguláciu aj v podmienkach reálneho systému.

4.3 Realizácia riadiaceho systému v jazyku C++ pre mikrokontrolér

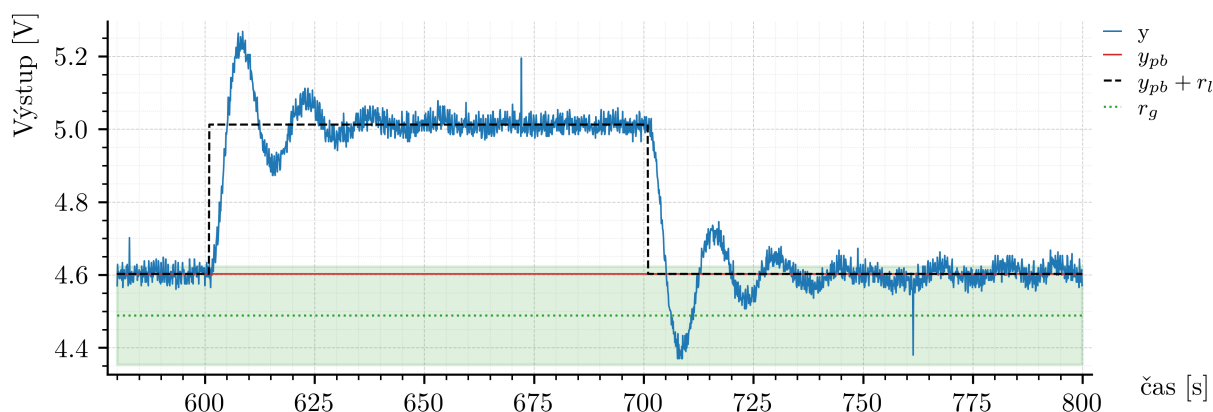
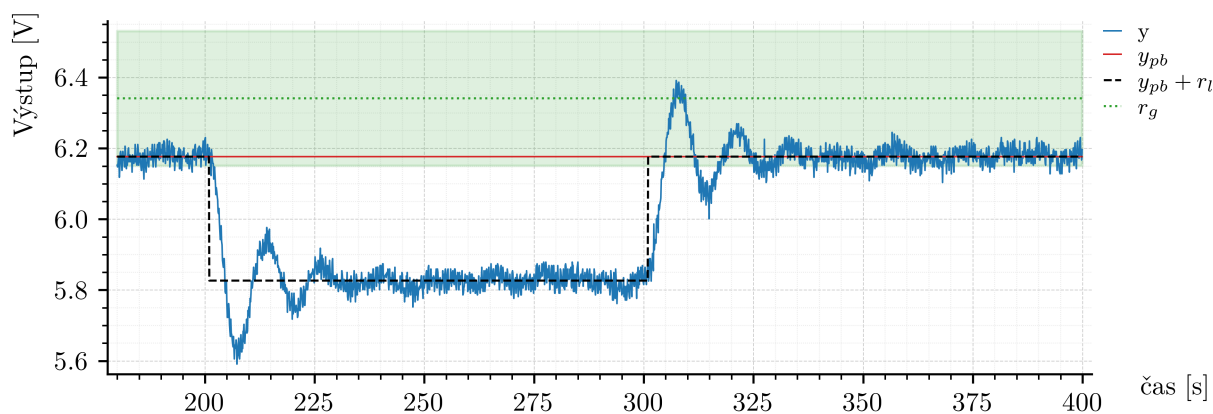
Po overení funkčnosti navrhnutého algoritmu v prostredí MATLAB/Simulink bola rovnaká prepínacia logika implementovaná aj v jazyku C++ pre mikrokontrolérovú platformu Arduino. Všeobecný princíp pripojenia mikrokontroléra, spôsob komunikácie so systémom TS05 a základná štruktúra programu boli opísané už v sekcii 2.3.2. V tejto



Obr. 30: Simulácia prepínania pracovných bodov a správanie regulačných veličín



Obr. 31: Experimentálne overenie prepínania pracovných bodov na systéme TS05



Obr. 32: Detail regulácie v okolí pracovného bodu po ukončení globálnej regulácie

časti sa preto pozornosť sústreďuje už na samotnú realizáciu algoritmu prepínania pracovných bodov.

Z hľadiska implementácie nebolo potrebné meniť celkovú štruktúru programu. Bolo potrebné iba doplniť pomocné funkcie na výpočty súvisiace so stavovým modelom filtra a s manipuláciou s históriou signálov, následne inicializovať všetky stavové premenné algoritmu a nakoniec implementovať samotnú prepínaciu logiku do hlavnej časti regulačného cyklu.

Keďže implementácia prebiehala v jazyku C++, nebolo už potrebné prenášať vnútorné stavy algoritmu medzi jednotlivými krokmi výpočtu rovnakým spôsobom ako v bloku MATLAB Function. Stavové premenné programu totiž prirodzene uchovávajú svoju hodnotu medzi jednotlivými iteráciami cyklu `loop()`. Z tohto dôvodu bolo možné pracovať priamo s integračnými stavmi regulátorov, so stavom filtra, s históriou výstupu a vstupu a s odhadnutými hodnotami pracovného bodu bez potreby zavádzať samostatné výstupné premenné typu `_new` pre každý stav algoritmu. Takýto prístup zjednodušil implementáciu a zároveň zachoval rovnakú logiku riadenia ako v simulácii.

Pre potreby implementácie bolo najprv potrebné doplniť niekoľko pomocných funkcií. Prvá skupina z nich zabezpečuje základné operácie s maticami a vektormi stavového modelu filtra. Druhá skupina slúži na aktualizáciu histórie signálov a na výpočet aritmetického priemeru posledných 30 vzoriek, ktorý sa využíva pri odhade nového pracovného bodu. Tieto pomocné funkcie sú uvedené vo výpise 17.

```

1 // 2x2 matrix times 2x1 vector
2 void mat2x2Vec2Mul(const float A[2][2], const float x[2], float result[2]) {
3     result[0] = A[0][0] * x[0] + A[0][1] * x[1];
4     result[1] = A[1][0] * x[0] + A[1][1] * x[1];
5 }
6
7 // 2x1 vector times scalar

```

```

8 void vec2ScalarMul(const float v[2], float scalar, float result[2]) {
9     result[0] = v[0] * scalar;
10    result[1] = v[1] * scalar;
11 }
12
13 // 2x1 vector plus 2x1 vector
14 void vec2Add(const float a[2], const float b[2], float result[2]) {
15     result[0] = a[0] + b[0];
16     result[1] = a[1] + b[1];
17 }
18
19 // Dot product of 1x2 and 2x1
20 float rowVec2ColVec2Mul(const float c[2], const float x[2]) {
21     return c[0] * x[0] + c[1] * x[1];
22 }
23
24 // Shift history left and append new value
25 void shiftAppend30(float hist[30], float newValue) {
26     for (int i = 0; i < 29; ++i) {
27         hist[i] = hist[i + 1];
28     }
29     hist[29] = newValue;
30 }
31
32 // Mean of 30 samples
33 float mean30(const float hist[30]) {
34     float sum = 0.0f;
35     for (int i = 0; i < 30; ++i) {
36         sum += hist[i];
37     }
38     return sum / 30.0f;
39 }

```

Výpis kódu 17: Pomocné funkcie použité pri implementácii algoritmu v jazyku C++

Po definovaní pomocných funkcií bolo potrebné inicializovať všetky premenné, ktoré reprezentujú vnútorný stav algoritmu. Ide najmä o globálnu a lokálnu žiadanú hodnotu, predchádzajúcu hodnotu globálnej referencie, príznaky aktivity regulačných vetiev, integračné stavy regulátorov, stav filtra, históriu vstupu a výstupu, tabuľkové hodnoty pracovných bodov a parametre jednotlivých PI regulátorov. Táto inicializácia je uvedená vo výpise 18.

```

1 float vent = 6.0f;
2 float spirala = 0.0f;
3
4 // References
5 float r_g = 0.0f;
6 float r_l = 0.0f;
7 float r_g_prev = 0.0f;
8 float r_gf = 0.0f;
9
10 // Controller activity
11 int active_g = 0;
12 int active_l = 1;
13
14 // Operating point
15 float y_pb = 0.0f;
16 float u_pb = 0.0f;
17
18 // Controller states
19 float I_l = 0.0f;
20 float I_g = 0.0f;
21
22 // Filter state
23 float x_f[2] = {0.0f, 0.0f};
24
25 // Filter matrices
26 const float A_f[2][2] = {
27     {0.0f, 1.0f},
28     {-0.01f, -0.2f}
29 };
30
31 const float b_f[2] = {0.0f, 1.0f};
32 const float c_f[2] = {0.01f, 0.0f};
33
34 // Histories
35 float y_hist[30] = {0.0f};

```

```

36 float u_hist[30] = {0.0f};
37
38 // Gain scheduling table
39 const float y_pb_table[3] = {4.4884f, 6.3412f, 7.7441f};
40 const float u_pb_table[3] = {2.0f, 4.0f, 6.0f};
41 const float Kp_l_table[3] = {2.77f, 3.58f, 1.83f};
42 const float Ki_l_table[3] = {1.48f, 1.84f, 2.20f};
43
44 // Global PI
45 const float Kp_g = 4.0f;
46 const float Ki_g = 0.7f;
47
48 // Active local PI gains
49 float Kp_l = 0.0f;
50 float Ki_l = 0.0f;
51
52 // Errors and control components
53 float e_l = 0.0f;
54 float e_g = 0.0f;
55 float u_l = 0.0f;
56 float u_g = 0.0f;
57 float u = 0.0f;
58
59 // Settling detection
60 float lower_bound = 0.0f;
61 float upper_bound = 0.0f;
62 int count_ok = 0;

```

Výpis kódu 18: Inicializácia stavových premenných algoritmu v jazyku C++

Samotná regulačná logika bola následne vložená do hlavného riadiaceho cyklu programu. V jej úvode sa najprv generujú priebehy globálnej a lokálnej žiadanej hodnoty. Globálna referencia zabezpečuje prechod medzi pracovnými bodmi systému, zatiaľ čo lokálna referencia predstavuje odchýlku od aktuálneho pracovného bodu. Následne sa vykoná detekcia skokovej zmeny globálnej žiadanej hodnoty, výpočet filtrovanej referencie, voľba parametrov lokálneho PI regulátora na základe princípu gain scheduling, výpočet akčného zásahu, aktualizácia histórie signálov a vyhodnotenie podmienky ustálenia.

Ak je splnená podmienka ustálenia, z posledných 30 hodnôt výstupu a vstupu sa odhadnú nové hodnoty y_{pb} a u_{pb} . Tieto hodnoty sa však nezačnú používať okamžite v tom istom vzorkovacom kroku. Najprv sa iba pripraví nové hodnoty pracovného bodu a nový stav aktivity globálneho regulátora, pričom samotné prepnutie sa prejaví až v nasledujúcom kroku výpočtu. Tým sa zachová diskretný charakter algoritmu a zabráni sa tomu, aby sa regulačný zákon menil dvakrát v rámci jednej periódy vzorkovania.

Implementovaný kód hlavnej regulačnej logiky je uvedený vo výpise 19.

```

1 //////////////////////////////////////////////////
2 // Reference generation
3
4 // Global reference
5 if (globalTime < 1.0f) {
6     r_g = 0.0f;
7 } else if (globalTime < 401.0f) {
8     r_g = 6.3412f;
9 } else if (globalTime < 801.0f) {
10    r_g = 4.4884f;
11 } else {
12    r_g = 4.4884f;
13 }
14
15 // Local reference
16 if (globalTime < 201.0f) {
17     r_l = 0.0f;
18 } else if (globalTime < 301.0f) {
19     r_l = -0.35f;
20 } else if (globalTime < 601.0f) {
21     r_l = 0.0f;
22 } else if (globalTime < 701.0f) {
23     r_l = 0.41f;
24 } else if (globalTime < 801.0f) {
25     r_l = 0.0f;
26 } else {

```

```

27     r_l = 0.0f;
28 }
29
30 ////////////////////////////////////////////////////
31 // Control law area - PI switching logic
32
33 // 1 Detect step change
34 if (r_g != r_g_prev) {
35     active_g = 1;
36 }
37
38 // 2 Filter global reference
39 r_gf = rowVec2ColVec2Mul(c_f, x_f);
40
41 float Afx[2];
42 float bfr[2];
43 float filterSum[2];
44
45 mat2x2Vec2Mul(A_f, x_f, Afx);
46 vec2ScalarMul(b_f, r_g, bfr);
47 vec2Add(Afx, bfr, filterSum);
48
49 x_f[0] += filterSum[0] * timeTickPeriod;
50 x_f[1] += filterSum[1] * timeTickPeriod;
51
52 // 3 Select local points PI params
53 float dist_pb = fabs(y_pb_table[0] - y_pb);
54 int idx_pb = 0;
55
56 for (int i = 1; i < 3; ++i) {
57     float dist_i = fabs(y_pb_table[i] - y_pb);
58     if (dist_i < dist_pb) {
59         dist_pb = dist_i;
60         idx_pb = i;
61     }
62 }
63
64 Kp_l = Kp_l_table[idx_pb];
65 Ki_l = Ki_l_table[idx_pb];
66
67 // 4 Control errors
68 e_l = r_l - (snimac1 - y_pb);
69 e_g = r_gf - snimac1;
70
71 // 5 Control law
72 u_l = (1 - active_g) * (u_pb + Kp_l * e_l + Ki_l * I_l);
73 u_g = active_g * (Kp_g * e_g + Ki_g * I_g);
74
75 u = u_l + u_g;
76 u = clampFloat(u, 0.0f, 10.0f);
77
78 // 6 Update histories
79 shiftAppend30(y_hist, snimac1);
80 shiftAppend30(u_hist, u);
81
82 // 7 Settling detection
83 lower_bound = r_g - 0.03f * r_g;
84 upper_bound = r_g + 0.03f * r_g;
85
86 count_ok = 0;
87 for (int i = 0; i < 30; ++i) {
88     if (y_hist[i] >= lower_bound && y_hist[i] <= upper_bound) {
89         count_ok++;
90     }
91 }
92
93 // Two-level switching logic
94 float y_pb_new = y_pb;
95 float u_pb_new = u_pb;
96 int active_g_new = active_g;
97
98 if (active_g == 1 && count_ok >= 25) {
99     y_pb_new = mean30(y_hist);
100    u_pb_new = mean30(u_hist);
101    active_g_new = 0;
102 }
103
104 active_l = 1 - active_g;

```



```

105
106 // 8 Integral states update
107 if (active_l == 1) {
108     I_l += e_l * timeTickPeriod;
109 } else {
110     I_l = 0.0f;
111 }
112
113 if (active_g == 1) {
114     I_g += e_g * timeTickPeriod;
115 } else {
116     I_g = (u - Kp_g * e_g) / Ki_g;
117 }
118
119 // 9 Memory update
120 r_g_prev = r_g;
121 y_pb = y_pb_new;
122 u_pb = u_pb_new;
123 active_g = active_g_new;
124
125 // Final output
126 spirala = u;
127 //////////////////////////////////////

```

Výpis kódu 19: Implementácia prepínacej logiky v hlavnom regulačnom cykle programu

Z hľadiska samotného algoritmu zostala implementácia v jazyku C++ veľmi blízka verzii použitej v MATLAB/Simulink. Najdôležitejším rozdielom je skutočnosť, že pri aktualizácii stavov filtra a integračných stavov regulátorov nebola použitá konštantná hodnota vzorkovacej periódy $T_s = 0.1$ s, ale reálne nameraná dĺžka aktuálnej iterácie cyklu, označená v programe ako `timeTickPeriod`. Tým sa dosiahlo, že algoritmus zohľadňuje malé odchýlky skutočnej periódy vykonávania kódu od nominálnej hodnoty.

Ďalším rozdielom oproti implementácii v bloku MATLAB Function je spôsob práce so stavovými premennými. V prostredí MATLAB bolo potrebné všetky vnútorné stavy explicitne prenášať medzi jednotlivými krokmi výpočtu vo forme vstupných a výstupných signálov. V jazyku C++ sú však tieto veličiny uchovávané priamo v pamäti programu, a preto nebolo potrebné zavádzať samostatné pomocné premenné pre každý aktualizovaný stav. Napriek tomu zostal princíp výpočtu nezmenený a poradie jednotlivých krokov algoritmu bolo zachované.

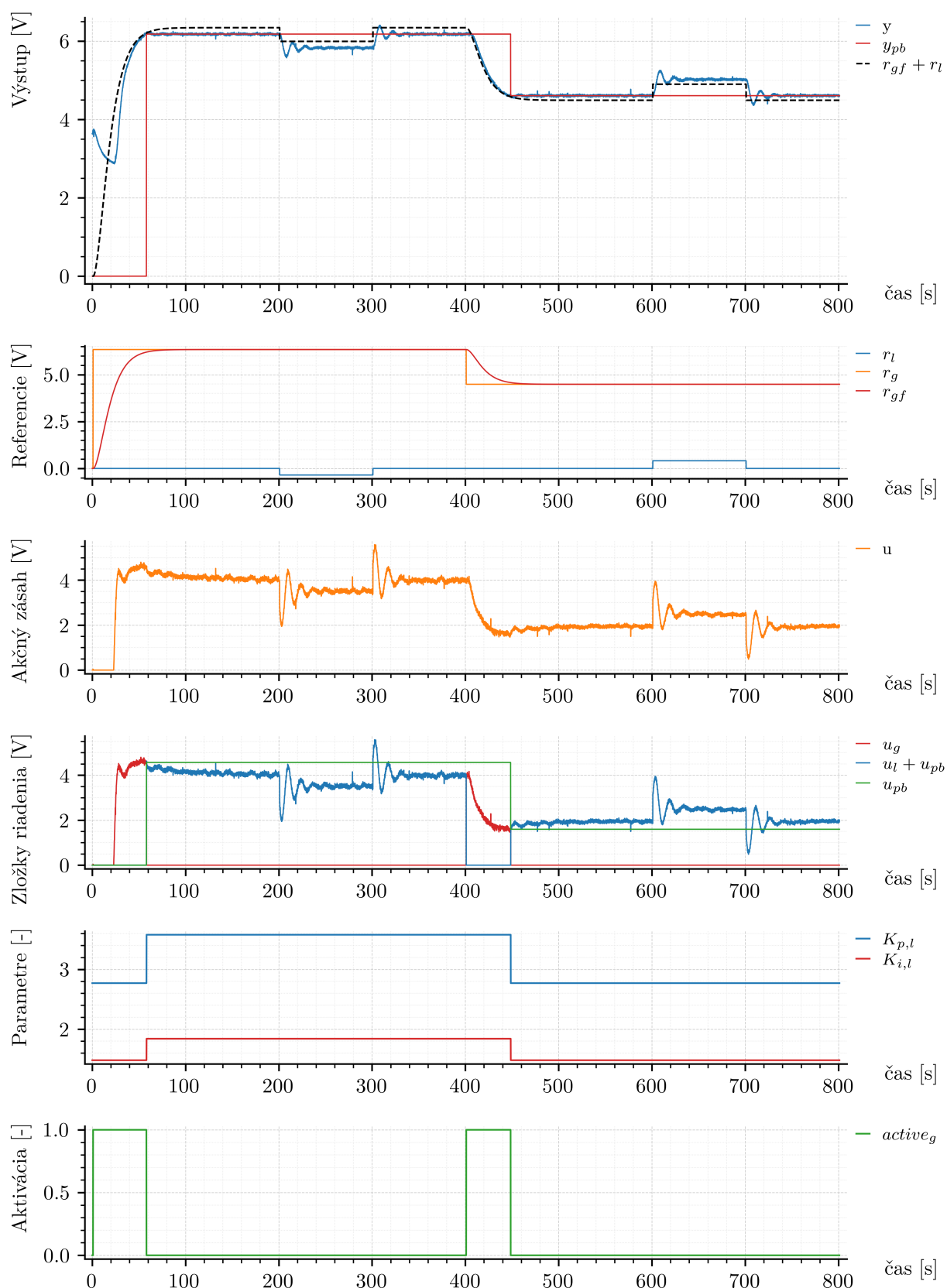
4.3.1 Overenie na reálnom systéme

Výsledky experimentálneho overenia implementácie v jazyku C++ sú uvedené na obr. 33. Z porovnania s experimentálnymi výsledkami získanými pri realizácii algoritmu v prostredí MATLAB/Simulink na obr. 31 je zrejmé, že správanie systému je veľmi podobné. Aj v tomto prípade možno pozorovať, že globálna regulácia zabezpečuje plynulý prechod do nového pracovného bodu a po jej ukončení preberá riadenie lokálny PI regulátor.

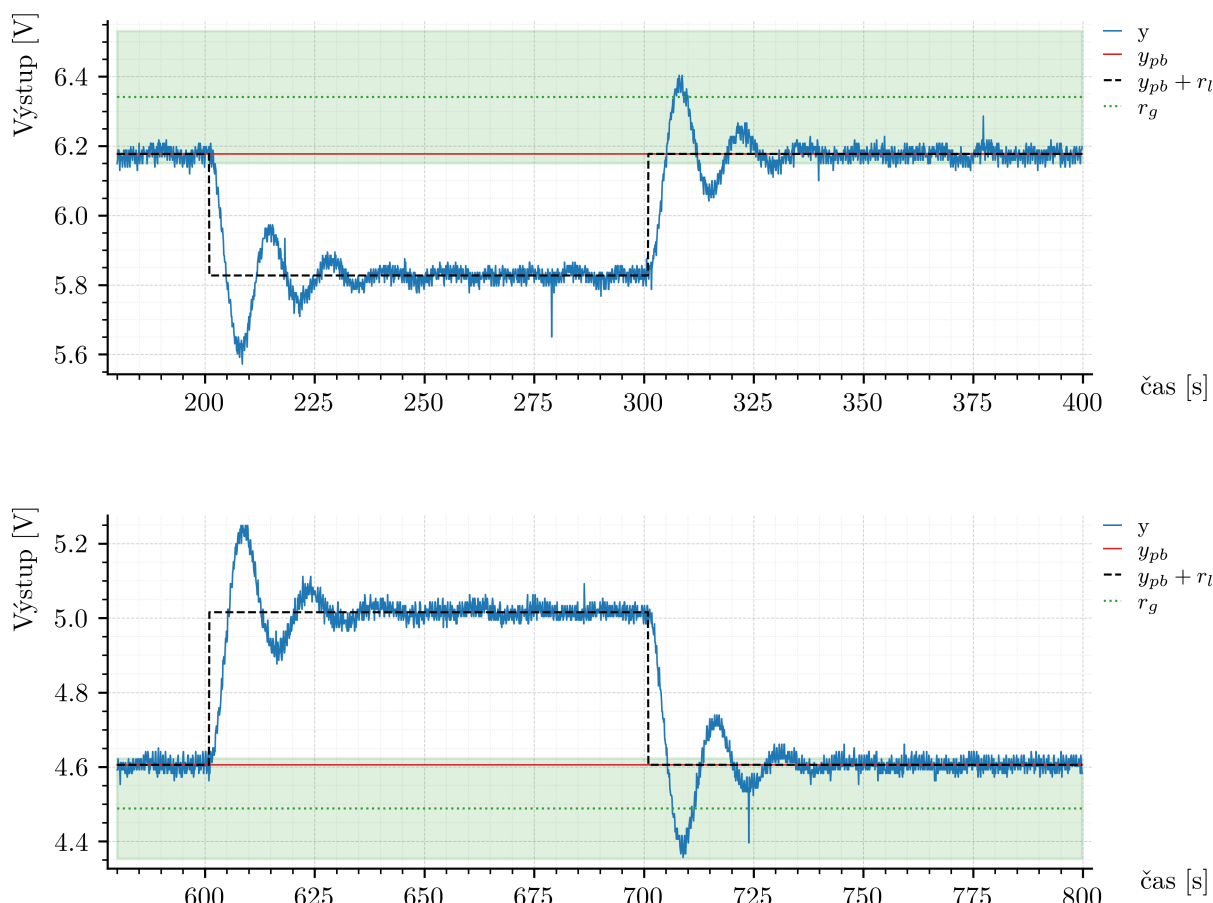
Pre detailnejšie posúdenie správania systému po ukončení globálnej regulácie je na obr. 34 zobrazený zväčšený detail priebehov výstupu. Aj v tomto prípade je viditeľné, že algoritmus pracuje v súlade s navrhnutým princípom. Po prepnutí do lokálneho režimu sa výstup systému viaže na hodnotu $y_{pb} + r_l$, pričom odhadnutý pracovný bod zostáva v intervale definovanom podmienkou ustálenia. Takéto správanie je veľmi podobné výsledkom, ktoré boli pozorované už na obr. 32, čo potvrdzuje správnosť implementácie algoritmu aj na mikrokontrolérovej platforme.

4.4 Implementácia pre programovateľný logický kontrolér (PLC)

Po overení funkčnosti navrhnutého algoritmu v prostredí MATLAB/Simulink a po jeho implementácii v jazyku C++ bola rovnaká prepínacia logika realizovaná aj na platforme Siemens SIMATIC S7-1200 G2. V tejto časti sa preto už nebudeme vracieť k teoretickému princípu globálnej a lokálnej regulačnej vetvy ani k samotnej detekcii ustálenia, pretože tieto časti boli vysvetlené v predchádzajúcich podkapitolách. Pozornosť je sústredená na samotnú implementáciu algoritmu v prostredí TIA Portal a na jeho experimentálne overenie.



Obr. 33: Experimentálne overenie prepínania pracovných bodov pri implementácii algoritmu v jazyku C++



Obr. 34: Detail regulácie v okolí pracovného bodu pri implementácii algoritmu v jazyku C++

Celá prepínacia logika bola realizovaná vo funkčnom bloku FB_PREP_TS05. Vo vnútri bloku sú uložené všetky potrebné stavové premenné algoritmu, teda stav filtra globálnej žiadanej hodnoty, integračné stavy oboch regulátorov, história vstupu a výstupu, odhadnuté hodnoty pracovného bodu aj pomocné premenné prepínacej logiky. Rovnako ako pri predchádzajúcich algoritmoch bol k implementácii pripojený aj logovací blok.

Okrem samotného riadiaceho bloku bol vytvorený aj samostatný funkčný blok na generovanie žiadanej hodnoty. Tento blok generuje priebeh globálnej a lokálnej referencie v závislosti od času experimentu, a tým umožňuje realizovať vopred definované scenáre prechodu medzi pracovnými bodmi bez potreby manuálneho zásahu počas merania.

Na začiatku vykonávania algoritmu je ošetrený neaktívny stav bloku. Ak vstup **Enable** nie je aktívny, blok nastaví výstupy na definované hodnoty a ukončí vykonávanie príkazom **RETURN**. Následne je realizovaná jednorazová inicializácia stavových premenných, tabuliek pracovných bodov a parametrov lokálnych PI regulátorov.

Ďalšia časť algoritmu zodpovedá štruktúre opísanej v predchádzajúcich implementáciách. Najprv sa deteguje skoková zmena globálnej žiadanej hodnoty, potom sa vypočíta filtrovaná globálna referencia, určia sa parametre lokálneho PI regulátora podľa aktuálneho pracovného bodu, vypočíta sa akčný zásah, aktualizuje sa história signálov a vyhodnotí sa podmienka ustálenia. Ak je táto podmienka splnená, z posledných 30 hodnôt vstupu a výstupu sa vypočíta nový odhad pracovného bodu, ktorý sa začne používať až v nasledujúcom vzorkovacom kroku.

Implementácia funkčného bloku FB_PREP_TS05 je uvedená v nasledujúcej ukážke.

```

1 IF NOT #Enable THEN
2   #U := 0.0;
3   #R_gf := 0.0;
4   #Y_pb_out := #Y_pb;
5   #U_pb_out := #U_pb;
6   #U_g := 0.0;
7   #U_l := 0.0;
8   #Active_g := #Active_g_m;
9   #Kp_l_out := 0.0;
10  #Ki_l_out := 0.0;
11  #Vent_out := #Vent_in;
12  RETURN;
13 END_IF;
14
15 // One-time initialization
16 IF NOT #Initialized THEN
17   #X_f1 := 0.0;
18   #X_f2 := 0.0;
19   #I_l := 0.0;
20   #I_g := 0.0;
21   #R_g_prev := #R_g;
22   #Active_g_m := FALSE;
23   #Y_pb := 0.0;
24   #U_pb := 0.0;
25   #Y_pb_table[0] := 4.4884;
26   #Y_pb_table[1] := 6.3412;
27   #Y_pb_table[2] := 7.7441;
28   #U_pb_table[0] := 2.0;
29   #U_pb_table[1] := 4.0;
30   #U_pb_table[2] := 6.0;
31   #Kp_l_table[0] := 2.77;
32   #Kp_l_table[1] := 3.58;
33   #Kp_l_table[2] := 1.83;
34   #Ki_l_table[0] := 1.48;
35   #Ki_l_table[1] := 1.84;
36   #Ki_l_table[2] := 2.20;
37   FOR #I := 0 TO 29 DO
38     #Y_hist[#I] := 0.0;
39     #U_hist[#I] := 0.0;
40   END_FOR;
41   #Initialized := TRUE;
42 END_IF;
43
44 // 1 Detect step change of global reference
45 IF #R_g <> #R_g_prev THEN
46   #Active_g_m := TRUE;
47 END_IF;
48
49 // 2 Filter global reference
50 #R_gf := 0.01 * #X_f1;
51
52 #X_f1_new := #X_f1 + #Ts * #X_f2;
53 #X_f2_new := #X_f2 + #Ts * (-0.01 * #X_f1 - 0.2 * #X_f2 + #R_g);
54
55 // 3 Select local operating point and PI parameters from table
56 #Dist_pb := ABS(#Y_pb_table[0] - #Y_pb);
57 #Idx_pb := 0;
58
59 FOR #I := 1 TO 2 DO
60   #Dist_i := ABS(#Y_pb_table[#I] - #Y_pb);
61   IF #Dist_i < #Dist_pb THEN
62     #Dist_pb := #Dist_i;
63     #Idx_pb := #I;
64   END_IF;
65 END_FOR;
66
67 #Kp_l := #Kp_l_table[#Idx_pb];
68 #Ki_l := #Ki_l_table[#Idx_pb];
69
70 // 4 Control errors
71 #E_l := #R_l - (#Y - #Y_pb);
72 #E_g := #R_gf - #Y;
73
74 // 5 Control law
75 IF #Active_g_m THEN
76   #U_l := 0.0;
77   #U_g := #Kp_g * #E_g + #Ki_g * #I_g;

```

```

78 ELSE
79     #U_l := #U_pb + #Kp_l * #E_l + #Ki_l * #I_l;
80     #U_g := 0.0;
81 END_IF;
82
83 #U_tmp := #U_l + #U_g;
84
85 // Saturation
86 IF #U_tmp > 10.0 THEN
87     #U_tmp := 10.0;
88 ELSIF #U_tmp < 0.0 THEN
89     #U_tmp := 0.0;
90 END_IF;
91
92 #U := #U_tmp;
93
94 // 6 Update histories
95 FOR #I := 0 TO 28 DO
96     #Y_hist[#I] := #Y_hist[#I + 1];
97     #U_hist[#I] := #U_hist[#I + 1];
98 END_FOR;
99
100 #Y_hist[29] := #Y;
101 #U_hist[29] := #U;
102
103 // 7 Settling detection
104 #Lower_bound := #R_g - 0.03 * #R_g;
105 #Upper_bound := #R_g + 0.03 * #R_g;
106
107 #Count_ok := 0;
108 FOR #I := 0 TO 29 DO
109     IF (#Y_hist[#I] >= #Lower_bound) AND (#Y_hist[#I] <= #Upper_bound) THEN
110         #Count_ok := #Count_ok + 1;
111     END_IF;
112 END_FOR;
113
114 // Two-level switching logic
115 #Y_pb_new := #Y_pb;
116 #U_pb_new := #U_pb;
117 #Active_g_new := #Active_g_m;
118
119 IF #Active_g_m AND (#Count_ok >= 25) THEN
120     #Mean_y := 0.0;
121     #Mean_u := 0.0;
122     FOR #I := 0 TO 29 DO
123         #Mean_y := #Mean_y + #Y_hist[#I];
124         #Mean_u := #Mean_u + #U_hist[#I];
125     END_FOR;
126     #Y_pb_est := #Mean_y / 30.0;
127     #U_pb_est := #Mean_u / 30.0;
128     #Y_pb_new := #Y_pb_est;
129     #U_pb_new := #U_pb_est;
130     #Active_g_new := FALSE;
131 END_IF;
132
133 IF #Active_g_m THEN
134     #Active_l := FALSE;
135 ELSE
136     #Active_l := TRUE;
137 END_IF;
138
139 // 8 Integral states update
140 IF #Active_l THEN
141     #I_l := #I_l + #E_l * #Ts;
142 ELSE
143     #I_l := 0.0;
144 END_IF;
145
146 IF #Active_g_m THEN
147     #I_g := #I_g + #E_g * #Ts;
148 ELSE
149     #I_g := (#U - #Kp_g * #E_g) / #Ki_g;
150 END_IF;
151
152 // 9 Memory update
153 #R_g_prev := #R_g;
154 #Y_pb := #Y_pb_new;
155 #U_pb := #U_pb_new;

```

```

156 #Active_g_m := #Active_g_new;
157 #X_f1 := #X_f1_new;
158 #X_f2 := #X_f2_new;
159
160 // Outputs for monitoring/logging
161 #Y_pb_out := #Y_pb;
162 #U_pb_out := #U_pb;
163 #Active_g := #Active_g_m;
164 #Kp_l_out := #Kp_l;
165 #Ki_l_out := #Ki_l;
166 #Vent_out := #Vent_in;

```

Výpis kódu 20: Implementácia funkčného bloku FB_PREP_TS05 v PLC Siemens

Schéma zapojenia algoritmu v PLC programe, vrátane bloku pre generovanie žiadanej hodnoty, riadiaceho bloku a logovania, je uvedená na obr. 35.

Zoznam premenných funkčného bloku je uvedený v tab. 13.

4.4.1 Overenie na reálnom systéme

Na základe tejto implementácie bol vykonaný experiment na reálnom systéme TS05. Výsledok merania je zobrazený na obr. 36.

Z priebehu experimentu je zrejmé, že algoritmus bol implementovaný korektne a zachováva rovnaký princíp správania ako v predchádzajúcich platformách. Globálna regulačná vetva zabezpečuje prechod do nového pracovného bodu a po detekcii ustálenia preberá riadenie lokálny PI regulátor.

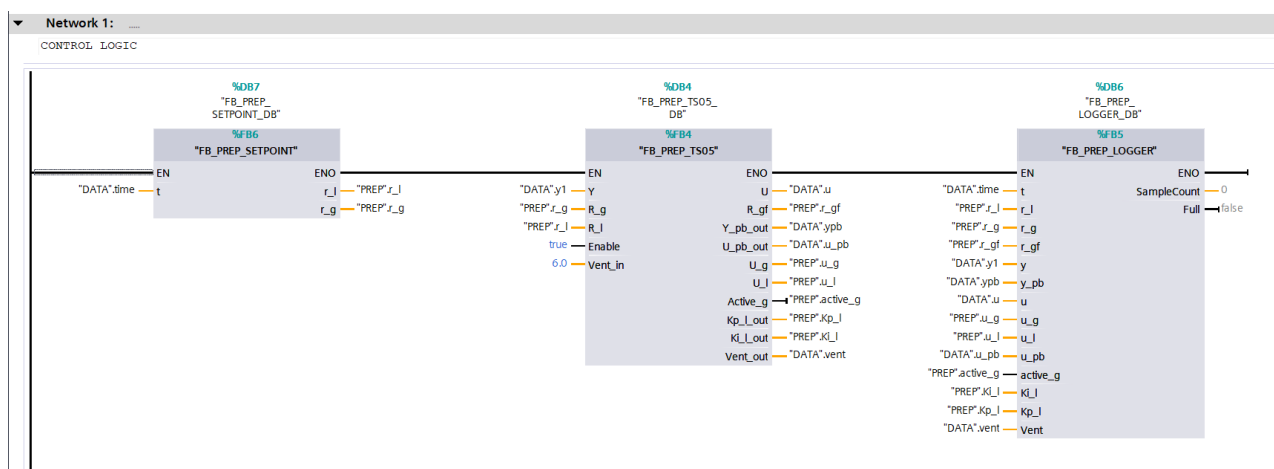
Správanie systému je kvalitatívne veľmi podobné výsledkom získaným v prostredí MATLAB/Simulink aj v jazyku C++. Zachovaný zostal charakter prechodov medzi pracovnými bodmi aj následná lokálna regulácia v ich okolí. Zároveň možno pozorovať, že akčný zásah zostáva plynulý aj pri prepnutí regulačných vetiev, čo potvrdzuje správnosť implementácie prepínacej logiky aj aktualizácie vnútorných stavov algoritmu.

Pre kontrolu práce detekcie ustálenia je na obr. 37 uvedený detail vybraných úsekov experimentu. Vidno, že odhadnutý pracovný bod sa po prepnutí nachádza v tolerančnom pásme $\pm 3\%$, čo potvrdzuje správnu činnosť použitej podmienky ustálenia aj v PLC implementácii.

4.5 Porovnanie výsledkov z jednotlivých implementácií

Algoritmus prepínania pracovných bodov bol implementovaný v MATLAB/Simulink, v jazyku C++ a v PLC Siemens. Porovnanie experimentálnych výsledkov je uvedené na obr. 38.

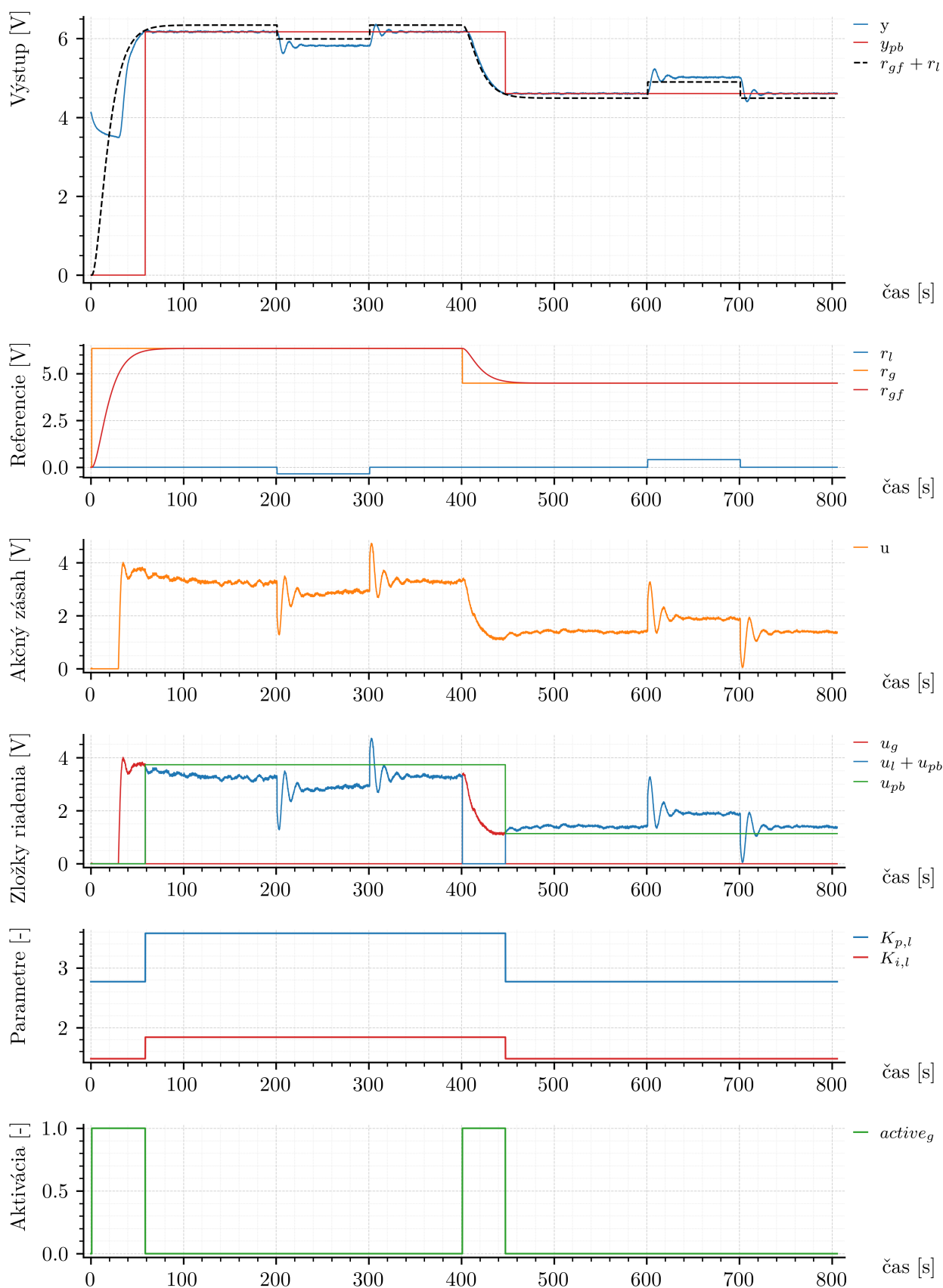
Z priebehov výstupu je viditeľné, že správanie systému je vo všetkých troch prípadoch podobné. Priebehy akčného zásahu majú podobný charakter, avšak medzi



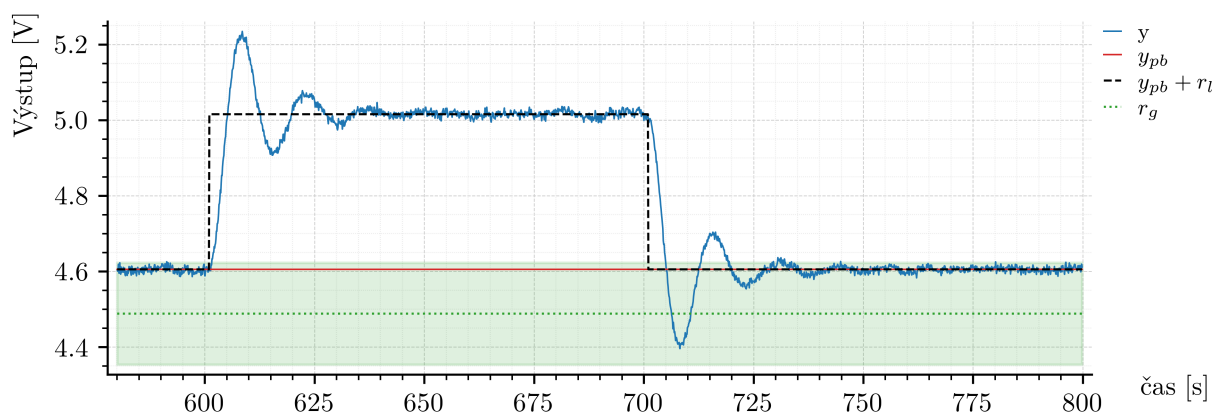
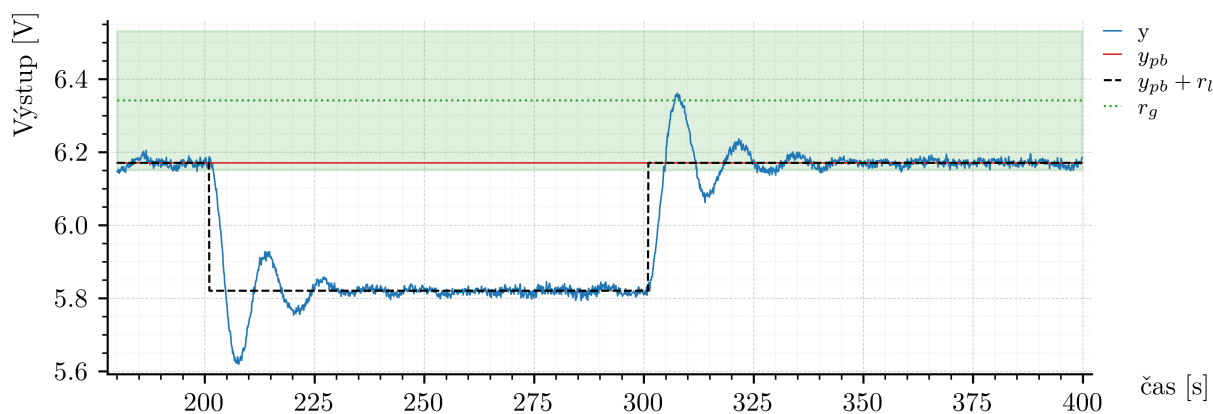
Obr. 35: Zapojenie algoritmu prepínania pracovných bodov v prostredí TIA Portal

Tabuľka 13: Premenné funkčného bloku FB_PREP_TS05

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Input	Y	Real	0.0	Non-retain
Input	R_g	Real	0.0	Non-retain
Input	R_l	Real	0.0	Non-retain
Input	Enable	Bool	false	Non-retain
Input	Vent_in	Real	0.0	Non-retain
Output	U	Real	0.0	Non-retain
Output	R_gf	Real	0.0	Non-retain
Output	Y_pb_out	Real	0.0	Non-retain
Output	U_pb_out	Real	0.0	Non-retain
Output	U_g	Real	0.0	Non-retain
Output	U_l	Real	0.0	Non-retain
Output	Active_g	Bool	false	Non-retain
Output	Kp_l_out	Real	0.0	Non-retain
Output	Ki_l_out	Real	0.0	Non-retain
Output	Vent_out	Real	0.0	Non-retain
Static	X_f1	Real	0.0	Non-retain
Static	X_f2	Real	0.0	Non-retain
Static	I_l	Real	0.0	Non-retain
Static	I_g	Real	0.0	Non-retain
Static	Y_hist	Array[0..29] of Real	–	Non-retain
Static	U_hist	Array[0..29] of Real	–	Non-retain
Static	R_g_prev	Real	0.0	Non-retain
Static	Active_g_m	Bool	false	Non-retain
Static	Y_pb	Real	0.0	Non-retain
Static	U_pb	Real	0.0	Non-retain
Static	Initialized	Bool	false	Non-retain
Static	Y_pb_table	Array[0..2] of Real	–	Non-retain
Static	U_pb_table	Array[0..2] of Real	–	Non-retain
Static	Kp_l_table	Array[0..2] of Real	–	Non-retain
Static	Ki_l_table	Array[0..2] of Real	–	Non-retain
Temp	E_l	Real	–	–
Temp	E_g	Real	–	–
Temp	U_tmp	Real	–	–
Temp	Lower_bound	Real	–	–
Temp	Upper_bound	Real	–	–
Temp	Y_pb_new	Real	–	–
Temp	U_pb_new	Real	–	–
Temp	Count_ok	Int	–	–
Temp	Idx_pb	Int	–	–
Temp	I	Int	–	–
Temp	Active_g_new	Bool	–	–
Temp	Active_l	Bool	–	–
Temp	Kp_l	Real	–	–
Temp	Ki_l	Real	–	–
Temp	Y_pb_est	Real	–	–
Temp	U_pb_est	Real	–	–
Temp	Dist_pb	Real	–	–
Temp	Dist_i	Real	–	–
Temp	X_f1_new	Real	–	–
Temp	X_f2_new	Real	–	–
Temp	Mean_y	Real	–	–
Temp	Mean_u	Real	–	–
Constant	Ts	Real	0.1	–
Constant	Kp_g	Real	4.0	–
Constant	Ki_g	Real	0.7	–



Obr. 36: Experimentálne overenie prepínania pracovných bodov pri implementácii algoritmu v PLC Siemens



Obr. 37: Detail regulácie v okolí pracovného bodu pri implementácii algoritmu v PLC Siemens

jednotlivými experimentmi možno pozorovať rozdielny *offset*. Ten môže súvisieť s odlišnými vonkajšími podmienkami počas meraní.

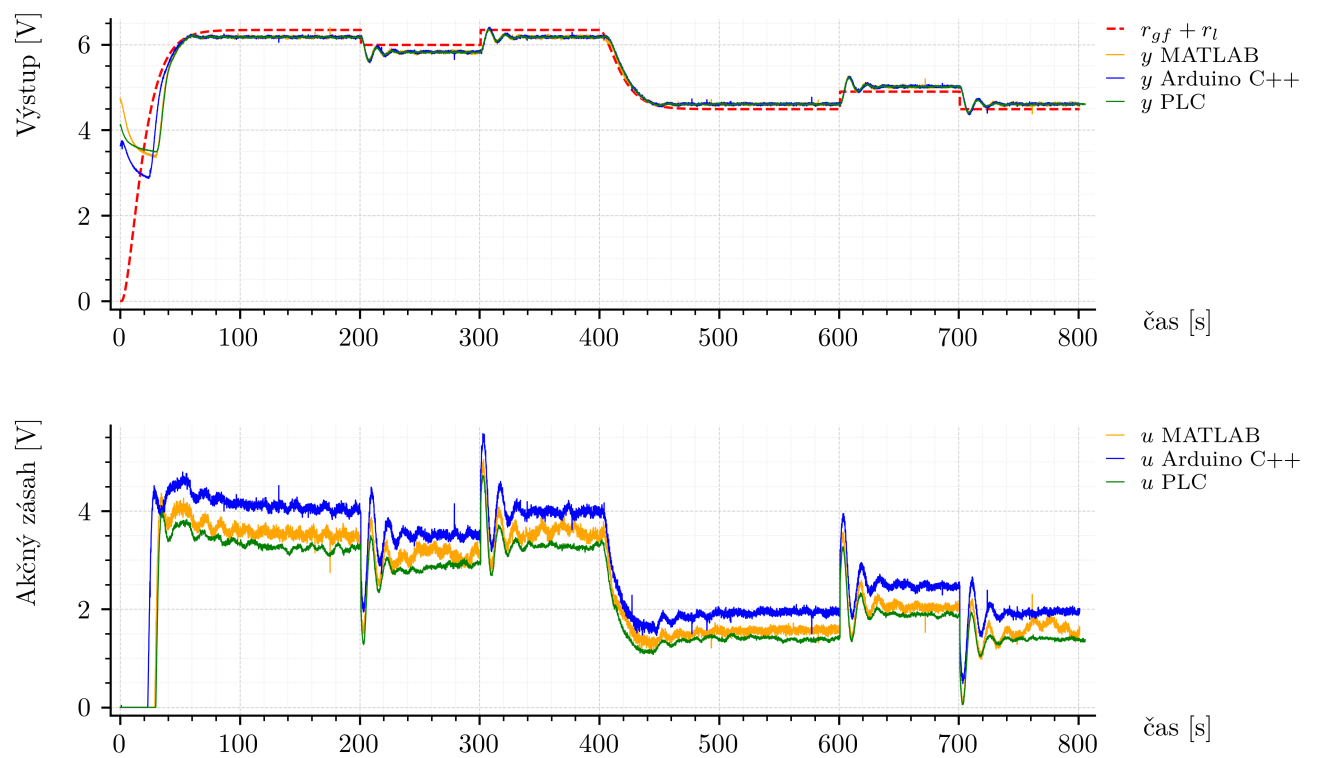
Pre kvantitatívne porovnanie bola vypočítaná hodnota RMSE vzhľadom na príslušnú hodnotu $r_{gf} + r_l$ každého experimentu:

$$\text{RMSE}_{\text{MATLAB}} = 0.497536 \text{ V}, \quad (4.21)$$

$$\text{RMSE}_{\text{C++}} = 0.405534 \text{ V}, \quad (4.22)$$

$$\text{RMSE}_{\text{PLC}} = 0.456926 \text{ V}. \quad (4.23)$$

Výsledky ukazujú, že algoritmus bol realizovaný na všetkých troch platformách s porovnateľným správaním výstupu systému.



Obr. 38: Porovnanie experimentálnych výsledkov prepínania pracovných bodov na jednotlivých platformách

5 Adaptívne riadenie s referenčným modelom

V tejto kapitole je implementovaný algoritmus riadenia typu MRAC (*Model Reference Adaptive Control*), ktorý umožňuje automatické prispôbovanie parametrov regulátora pri nepresne známej dynamike riadeného systému. Adaptívne riadenie a jeho použitie pri technologických procesoch je podrobne spracované napríklad v [11, 10]. Na rozdiel od klasických regulátorov s pevnými parametrami MRAC využíva referenčný model, ktorý definuje požadované správanie uzavretého regulačného obvodu, a adaptačný mechanizmus, ktorý zabezpečuje sledovanie tohto správania v čase.

V ďalšej časti je popísaný samotný algoritmus gradientnej MRAC regulácie, vrátane voľby referenčného modelu, štruktúry zákona riadenia a adaptačného zákona založeného na gradientnej metóde.

5.1 Rozbor algoritmu riadenia

Gradientný MRAC algoritmus je založený na myšlienke, že dynamika riadeného systému má byť v čase prispôbovaná tak, aby výstup systému sledoval výstup zvoleného referenčného modelu. Návrh algoritmu pozostáva z troch základných častí: voľba referenčného modelu, návrh zákona riadenia a odvodenie adaptačného zákona.

Model riadeného systému

Uvažujme lineárny spojitý systém druhého rádu

$$G_s(s) = \frac{b_0}{s^2 + a_1 s + a_0}, \quad (5.1)$$

kde a_0 , a_1 , b_0 sú neznáme konštanty. Predpokladá sa znalosť znamienka parametra b_0 .

Referenčný model a statické zosilnenie

Pri voľbe referenčného modelu je potrebné zabezpečiť, aby jeho statické zosilnenie bolo rovné jednej, t. j.

$$K = 1,$$

aby nedochádzalo k nechcenému zosilňovaniu referenčného signálu.

Pre systém druhého rádu zvolíme referenčný model

$$\frac{y_m(s)}{r(s)} = \frac{b_{0m}}{s^2 + a_{1m}s + a_{0m}}, \quad (5.2)$$

pričom sa volí

$$b_{0m} = a_{0m}, \quad (5.3)$$

čím sa zabezpečí jednotkové statické zosilnenie modelu.

Zákon riadenia

Všeobecný tvar zákona riadenia pri splnenej podmienke zhody je

$$u(t) = \Theta^T(t) \omega(t), \quad (5.4)$$

kde regresný vektor obsahuje merateľné signály.

Pre systém druhého rádu zvolíme

$$\omega(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \\ r(t) \end{bmatrix}. \quad (5.5)$$

Zákon riadenia potom nadobúda tvar

$$u(t) = \Theta_1 y(t) + \Theta_2 \dot{y}(t) + \Theta_3 r(t). \quad (5.6)$$

Podmienka zhody a ideálne parametre

Po dosadení (5.6) do systému (5.1) dostávame

$$y(s) (s^2 + (a_1 - b_0\Theta_2)s + a_0 - b_0\Theta_1) = b_0\Theta_3 r(s). \quad (5.7)$$

Prenos uzavretého regulačného obvodu je

$$\frac{y(s)}{r(s)} = \frac{b_0\Theta_3}{s^2 + (a_1 - b_0\Theta_2)s + (a_0 - b_0\Theta_1)}. \quad (5.8)$$

Porovnaním s referenčným modelom získame ideálny vektor parametrov

$$\Theta_1^* = \frac{a_0 - a_{0m}}{b_0}, \quad (5.9)$$

$$\Theta_2^* = \frac{a_1 - a_{1m}}{b_0}, \quad (5.10)$$

$$\Theta_3^* = \frac{b_{0m}}{b_0}. \quad (5.11)$$

Adaptačná odchýlka

Adaptačná odchýlka je definovaná ako

$$e(t) = y(t) - y_m(t), \quad (5.12)$$

pričom cieľom riadenia je dosiahnuť

$$\lim_{t \rightarrow \infty} e(t) = 0. \quad (5.13)$$

Účelová funkcia a MIT pravidlo

Účelová funkcia sa uvažuje v tvare

$$J(\Theta) = \frac{1}{2}e^2(t). \quad (5.14)$$

a využíva sa v rámci tzv. MIT pravidla (vzniklo na Massachusetts Institute of Technology), ktoré patrí medzi základné gradientné prístupy používané pri návrhu adaptačných zákonov v adaptívnom riadení [10]. Je formulované ako

$$\frac{d\Theta}{dt} = -\alpha \frac{\partial J}{\partial \Theta} = -\alpha e \frac{\partial e}{\partial \Theta}. \quad (5.15)$$

Keďže y_m nezávisí od Θ , platí

$$\frac{d\Theta}{dt} = -\alpha e \frac{\partial y}{\partial \Theta}. \quad (5.16)$$

Zákon adaptácie pre systém druhého rádu

Pre uvažovaný prípad sú zákony adaptácie dané vzťahmi

$$\dot{\Theta}_1 = -\alpha_1 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} y, \quad (5.17)$$

$$\dot{\Theta}_2 = -\alpha_2 e \operatorname{sign}(b_0) \frac{s}{s^2 + a_{1m}s + a_{0m}} y, \quad (5.18)$$

$$\dot{\Theta}_3 = -\alpha_3 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} r. \quad (5.19)$$

pričom s v adaptačnom zákone pre parameter Θ_2 zohľadňuje dynamický vplyv derivácie výstupu v zákone riadenia.

5.1.1 Prepis vstupno-výstupného opisu systému do stavového opisu

Pre potreby diskretnej implementácie je vhodné preniesť opis dynamického systému z tvaru prenosovej funkcie do tvaru stavového opisu. Stavový opis predstavuje prirodzený spôsob reprezentácie dynamických systémov, keďže umožňuje pracovať priamo s vnútornými stavmi systému a ich časovým vývojom [5, 7].

Uvažujme všeobecný lineárny systém druhého rádu

$$G_s(s) = \frac{Y(s)}{U(s)} = \frac{b_1 s + b_0}{s^2 + a_1 s + a_0}. \quad (5.20)$$

Tento prenos je možné interpretovať ako kaskádové spojenie dvoch čiastkových prenosov zavedením pomocnej premennej $Z(s)$

$$\frac{Z(s)}{U(s)} = \frac{1}{s^2 + a_1 s + a_0}, \quad (5.21)$$

$$\frac{Y(s)}{Z(s)} = b_1 s + b_0. \quad (5.22)$$

Platí teda

$$\frac{Y(s)}{U(s)} = \frac{Y(s)}{Z(s)} \frac{Z(s)}{U(s)}. \quad (5.23)$$

Prvý z uvedených prenosov (5.21) možno zapísať v časovej oblasti ako diferenciálnu rovnicu

$$\ddot{z}(t) + a_1 \dot{z}(t) + a_0 z(t) = u(t). \quad (5.24)$$

Táto rovnica je druhého rádu a je možné ju previesť na sústavu diferenciálnych rovníc prvého rádu vhodnou voľbou stavových veličín.

Zavedme prvú stavovú veličinu

$$x_1(t) = z(t), \quad (5.25)$$

z čoho vyplýva

$$\dot{x}_1(t) = \dot{z}(t). \quad (5.26)$$

Ako druhú stavovú veličinu zvolíme

$$x_2(t) = \dot{z}(t), \quad (5.27)$$

a teda

$$\dot{x}_2(t) = \ddot{z}(t). \quad (5.28)$$

Z rovnice (5.24) je možné vyjadriť druhú deriváciu

$$\ddot{z}(t) = -a_1 \dot{z}(t) - a_0 z(t) + u(t), \quad (5.29)$$

čo po dosadení za stavové premenné vedie k

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (5.30)$$

Sústava diferenciálnych rovníc prvého rádu má teda tvar

$$\dot{x}_1(t) = x_2(t), \quad (5.31)$$

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (5.32)$$

V maticovom zápise možno systém vyjadriť ako

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t), \quad (5.33)$$

kde stavový vektor je definovaný ako

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}. \quad (5.34)$$

Výstupná rovnica vyplýva z prenosu (5.22). V časovej oblasti platí

$$y(t) = b_1 \dot{z}(t) + b_0 z(t). \quad (5.35)$$

Po vyjadrení pomocou stavových veličín dostávame

$$y(t) = b_1 x_2(t) + b_0 x_1(t), \quad (5.36)$$

resp. v maticovom tvare

$$y(t) = \begin{bmatrix} b_0 & b_1 \end{bmatrix} x(t). \quad (5.37)$$

Celý systém je teda v štandardnom stavovom tvare

$$\dot{x}(t) = Ax(t) + bu(t), \quad (5.38)$$

$$y(t) = c^T x(t), \quad (5.39)$$

kde jednotlivé matice majú tvar

$$A = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}. \quad (5.40)$$

Zavedená pomocná premenná $z(t)$ predstavuje interný stav systému, ktorý bol použitý na rozklad prenosovej funkcie do kaskádového tvaru. Pri zvolenej realizácii teda stavový vektor nepredstavuje priamo $[y(t) \quad \dot{y}(t)]^T$, ale pomocné stavové veličiny, z ktorých sa výstup systému určuje pomocou výstupnej rovnice

$$y(t) = c^T x(t).$$

Takto získaný stavový opis je vhodný pre numerickú implementáciu, pretože dynamika systému je sústredená do stavovej rovnice a výstup je určený ako lineárna kombinácia stavových premenných.

5.1.2 Stavový opis súčastí MRAC algoritmu

Stavový opis referenčného modelu

Referenčný model definovaný prenosom (5.2) je potrebné previesť do stavového tvaru. Zvolíme stavový vektor

$$x_{ym}(t) = \begin{bmatrix} y_m(t) \\ \dot{y}_m(t) \end{bmatrix}. \quad (5.41)$$

Na základe všeobecného tvaru odvodeného v predchádzajúcej časti má referenčný model podobu

$$\dot{x}_{ym}(t) = A_{ym}x_{ym}(t) + b_{ym}r(t), \quad (5.42)$$

$$y_m(t) = c_{ym}^T x_{ym}(t), \quad (5.43)$$

kde

$$A_{ym} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ym} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ym} = \begin{bmatrix} b_{0m} \\ 0 \end{bmatrix}. \quad (5.44)$$

Stavový opis filtra výstupu

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_1 (5.17) je daný prenosom

$$\frac{y_f(s)}{y(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (5.45)$$

Zvolíme stavový vektor

$$x_{yf}(t) = \begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix}. \quad (5.46)$$

Stavový opis má tvar

$$\dot{x}_{yf}(t) = A_{yf}x_{yf}(t) + b_{yf}y(t), \quad (5.47)$$

$$y_f(t) = c_{yf}^T x_{yf}(t), \quad (5.48)$$

kde

$$A_{yf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{yf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{yf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (5.49)$$

Stavový opis filtra derivácie výstupu

V adaptačnom zákone pre parameter Θ_2 (5.18) sa nachádza člen

$$\frac{\dot{y}_f(s)}{y(s)} = \frac{s}{s^2 + a_{1m}s + a_{0m}}, \quad (5.50)$$

ktorý zodpovedá derivácii výstupu predchádzajúceho filtra. Použijeme rovnaký stavový vektor $x_{yf}(t)$, pričom výstupná rovnica reprezentuje druhú stavovú veličinu

$$\dot{x}_{yf}(t) = A_{ydf}x_{yf}(t) + b_{ydf}y(t), \quad (5.51)$$

$$\dot{y}_f(t) = c_{ydf}^T x_{yf}(t), \quad (5.52)$$

kde

$$A_{ydf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (5.53)$$

Stavový opis filtra referencie

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_3 (5.19) je daný prenosom

$$\frac{r_f(s)}{r(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (5.54)$$

Zvolíme stavový vektor

$$x_{rf}(t) = \begin{bmatrix} r_f(t) \\ \dot{r}_f(t) \end{bmatrix}. \quad (5.55)$$

Stavový opis má tvar

$$\dot{x}_{rf}(t) = A_{rf}x_{rf}(t) + b_{rf}r(t), \quad (5.56)$$

$$r_f(t) = c_{rf}^T x_{rf}(t), \quad (5.57)$$

kde

$$A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{rf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (5.58)$$

Zhrnutie štruktúry dynamických blokov

Zo získaných stavových opisov vyplýva, že všetky dynamické bloky využívajú rovnakú menovateľovú dynamiku, teda

$$A_{ym} = A_{yf} = A_{ydf} = A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad (5.59)$$

a rovnako

$$b_{ym} = b_{yf} = b_{ydf} = b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (5.60)$$

Jednotlivé bloky sa odlišujú iba výstupnou maticou c . Táto vlastnosť je výhodná aj z implementačného hľadiska, pretože referenčný model aj oba filtračné bloky možno v diskretnej realizácii aktualizovať rovnakým numerickým postupom. Odlišujú sa iba vstupnými a výstupnými veličinami, nie vnútornou dynamikou modelu.

Pre filter výstupu a jeho derivácie možno zapísať spoločnú výstupnú rovnicu v maticovom tvare

$$\begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix} = C_{yf}x_{yf}(t), \quad (5.61)$$

kde

$$C_{yf} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (5.62)$$

Pre filter referencie platí

$$r_f(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x_{rf}(t). \quad (5.63)$$

Referenčný model je realizovaný samostatne so stavovým vektorom $x_{ym}(t)$, pričom jeho výstup je

$$y_m(t) = \begin{bmatrix} b_{0m} & 0 \end{bmatrix} x_{ym}(t). \quad (5.64)$$

5.1.3 Pseudokód adaptívneho riadiaceho systému

Diskrétna realizácia gradientnej MRAC regulácie je vykonávaná v pevnom vzorkovacom intervale T_s . V každom vzorkovacom kroku sú k dispozícii aktuálne merané veličiny, vnútorné stavové premenné regulátora a hodnoty uložené z predchádzajúceho kroku.

Keďže navrhnutý regulátor vychádza z linearizovaného opisu systému v okolí pracovného bodu, vlastnej adaptácii predchádza fáza ustálenia. Počas tejto fázy je na vstup systému privádzaný konštantný riadiaci zásah u_{pb} po dobu T_{settle} , aby sa sústava dostala do okolia zvoleného pracovného bodu. Po ukončení tejto fázy sa uložia hodnoty výstupu a referencie v pracovnom bode, označené ako y_{pb} a r_{pb} .

Ďalšia činnosť regulátora potom prebieha so signálmi vyjadrenými ako odchýlky od pracovného bodu, teda

$$\Delta y(k) = y(k) - y_{pb}, \quad \Delta r(k) = r(k) - r_{pb}, \quad \Delta u(k) = u(k) - u_{pb}. \quad (5.65)$$

Keďže zákon riadenia využíva aj deriváciu výstupu, je potrebné túto veličinu v diskretnom čase numericky aproximovať. Použitá je spätná diferencia v tvare

$$\Delta \dot{y}(k) = \frac{\Delta y(k) - \Delta y(k-1)}{T_s}, \quad (5.66)$$

kde $\Delta y(k-1)$ predstavuje odchýlku výstupu v predchádzajúcom vzorkovacom kroku. Po skončení fázy ustálenia sa zostaví regresný vektor

$$\omega(k) = \begin{bmatrix} \Delta y(k) \\ \Delta \dot{y}(k) \\ \Delta r(k) \end{bmatrix}, \quad (5.67)$$

na základe ktorého sa určí adaptívna zložka riadiaceho zásahu

$$\Delta u(k) = \Theta^T(k) \omega(k). \quad (5.68)$$

Celkový riadiaci zásah má potom tvar

$$u(k) = u_{pb} + \Delta u(k). \quad (5.69)$$

Chyba sledovania je definovaná ako

$$e(k) = \Delta y(k) - y_m(k), \quad (5.70)$$

kde $y_m(k)$ je výstup referenčného modelu buď odchýlkou referencie $\Delta r(k)$.

Aktualizácia parametrov regulátora je realizovaná diskretným tvarom MIT pravidla pomocou Eulerovej integrácie

$$\Theta(k+1) = \Theta(k) - T_s \text{sign}(b_0) e(k) \begin{bmatrix} \alpha_1 y_f(k) \\ \alpha_2 \dot{y}_f(k) \\ \alpha_3 r_f(k) \end{bmatrix}. \quad (5.71)$$

Stavové rovnice referenčného modelu a filtračných blokov sú integrované rovnakým spôsobom v explicitnom Eulerovom tvare

$$x(k+1) = x(k) + T_s(Ax(k) + bv(k)), \quad (5.72)$$

kde vstup $v(k)$ predstavuje buď odchýlku referencie $\Delta r(k)$, alebo odchýlku výstupu $\Delta y(k)$ v závislosti od konkrétneho dynamického bloku.

Osobitný význam má prechod z fázy ustálenia do adaptívneho režimu. V tomto okamihu sa uložia hodnoty pracovného bodu a súčasne sa vnútorné stavy dynamických blokov inicializujú na nulové hodnoty, aby ďalší výpočet prebiehal už v súradniciach vzťahovaných k pracovnému bodu. Rovnako sa pri prvom kroku po prepnutí nastaví na nulu aj predchádzajúca odchýlka výstupu, čím sa zabezpečí korektný výpočet diskretné derivácie. Pseudokód diskretné realizácie algoritmu je uvedený nižšie vo výpise kódu 21.


```

% Inputs:
% r - reference signal
% y - plant output
% t - current time
%
% States:
% x_ym(2x1) - states of reference model
% x_yf(2x1) - states of output filter
% x_rf(2x1) - states of reference filter
% Theta(3x1) - [Theta1; Theta2; Theta3]
% y_prev - previous plant output
% y_pb - stored output at operating point
% r_pb - stored reference at operating point
% sw - switching flag
%
% Parameters:
% a0m, a1m, b0m
% alpha1, alpha2, alpha3
% sign_b0
% Ts
% u_pb
% T_settle
% u_min, u_max

A = [0, 1;
     -a0m, -a1m];

b = [0;
     1];

C_yf = [1, 0;
        0, 1];

C_rf = [1, 0];

C_ym = [b0m, 0];

% Default outputs and state propagation
u = u_pb;
ym = 0;

propagate all internal states unchanged by default
store current output as y_prev_next
store current operating-point variables as y_pb_next, r_pb_next
store switching flag as sw_next

% Settling phase
if t < T_settle
    keep u = u_pb
    keep adaptation and dynamic blocks inactive
    return
end

if sw < 0.5
    y_pb_next = y;
    r_pb_next = r;

    x_ym_work = [0; 0];
    x_yf_work = [0; 0];
    x_rf_work = [0; 0];

    Theta_work = Theta;

    y_prev_delta = 0;
    sw_next = 1;
else
    x_ym_work = x_ym;
    x_yf_work = x_yf;
    x_rf_work = x_rf;

    Theta_work = Theta;

    y_prev_delta = y_prev - y_pb;
end

delta_y = y - y_pb_next;
delta_r = r - r_pb_next;

```

```

delta_ydot = (delta_y - y_prev_delta) / Ts;

ym = C_ym * x_ym_work;
yf_full = C_yf * x_yf_work;
yf = yf_full(1);
ydot_f = yf_full(2);
rf = C_rf * x_rf_work;

e = delta_y - ym;

omega = [delta_y; delta_ydot; delta_r];

delta_u = Theta_work.' * omega;
u_unsat = u_pb + delta_u;
u = min(max(u_unsat, u_min), u_max);

dTheta = -sign_b0 * e * [alpha1 * yf;
                        alpha2 * ydot_f;
                        alpha3 * rf];

Theta_next = Theta_work + dTheta * Ts;
Theta_used = Theta_work;

x_ym_next = x_ym_work + Ts * (A * x_ym_work + b * delta_r);
x_yf_next = x_yf_work + Ts * (A * x_yf_work + b * delta_y);
x_rf_next = x_rf_work + Ts * (A * x_rf_work + b * delta_r);

y_prev_next = y;

```

Výpis kódu 21: Pseudokód diskrétnej implementácie gradientného MRAC regulátora s prechodom do súradníc pracovného bodu

5.2 Implementácia: MATLAB/Simulink

Navrhnutý algoritmus MRAC bol implementovaný v prostredí MATLAB/Simulink bez využitia špecializovaných blokov adaptívneho riadenia. Celý regulátor je realizovaný v rámci jedného bloku MATLAB Function, pričom jeho štruktúra zodpovedá diskrétnej realizácii uvedenej v časti 5.1.3.

V implementácii bol použitý referenčný model

$$\frac{y_m(s)}{r(s)} = \frac{1}{s^2 + 20s + 1}, \quad (5.73)$$

so vzorkovacou periódou

$$T_s = 0.1 \text{ s}. \quad (5.74)$$

Počas úvodnej fázy ustálenia je na vstup systému privádzaný konštantný zásah

$$u_{pb} = 3, \quad (5.75)$$

po dobu

$$T_{\text{settle}} = 400 \text{ s}. \quad (5.76)$$

Po ukončení tejto fázy implementácia prechádza do režimu riadenia v súradniciach vzťahovaných k pracovnému bodu, pričom sa podľa pseudokódu z listingu 21 uložia hodnoty y_{pb} a r_{pb} , nulovo inicializujú stavy dynamických blokov a aktivuje sa adaptačný mechanizmus.

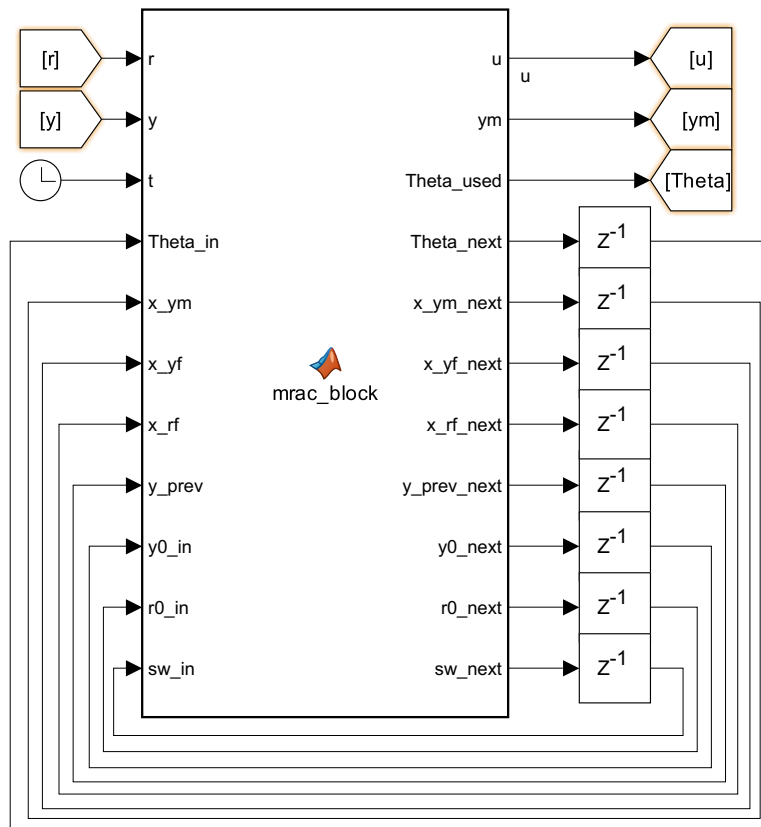
Riadiaci zásah je v implementácii obmedzený saturáciou na interval $\langle 0, 10 \rangle$, aby boli rešpektované fyzikálne obmedzenia sústavy. Adaptačné koeficienty boli zvolené v tvare

$$\alpha_1 = 14 \cdot 10^{-2}, \quad \alpha_2 = 30 \cdot 10^{-2}, \quad \alpha_3 = 14 \cdot 10^{-2}. \quad (5.77)$$

Znamienko parametra b_0 je v implementácii uvažované ako kladné, teda

$$\text{sign}(b_0) = 1. \quad (5.78)$$

Zdrojový kód implementovaného regulátora v bloku MATLAB Function je uvedený nižšie vo výpise kódu 22. Schéma implementovaného bloku MATLAB Function v prostredí Simulink spolu s jeho vstupmi a výstupmi je znázornená na obr. 39.



Obr. 39: Blok MATLAB Function implementujúci gradientný MRAC regulátor v prostredí Simulink

```

1 function [u, ym, Theta_used, Theta_next, x_ym_next, x_yf_next, ...
2           x_rf_next, y_prev_next, y_pb_next, r_pb_next, sw_next] = ...
3           mrac_block(r, y, t, Theta_in, x_ym, x_yf, x_rf, y_prev, ...
4                     y_pb_in, r_pb_in, sw_in)
5
6 % params
7 a0m = 1;
8 a1m = 20;
9 b0m = 1;
10
11 alpha1 = 14 * 0.01;
12 alpha2 = 30 * 0.01;
13 alpha3 = 14 * 0.01;
14
15 sign_b0 = 1;
16 Ts = 0.1;
17
18 u_pb = 3;
19 T_settle = 400;
20
21 u_min = 0;
22 u_max = 10;
23
24 % State space matrices
25 A = [0, 1;
26      -a0m, -a1m];
27
28 b = [0;
29      1];
30
31 C_yf = [1, 0;
32         0, 1];
33
34 C_rf = [1, 0];
35
36 C_ym = [b0m, 0];

```

```

37
38 %
39 Theta = Theta_in;
40 Theta_used = Theta;
41
42 y_pb_next = y_pb_in;
43 r_pb_next = r_pb_in;
44 sw_next = sw_in;
45
46 %---
47 u = u_pb;
48 ym = 0;
49
50 Theta_next = Theta;
51 x_ym_next = x_ym;
52 x_yf_next = x_yf;
53 x_rf_next = x_rf;
54 y_prev_next = y;
55
56 %Plant settle
57 if t < T_settle
58     u = u_pb;
59     ym = 0;
60
61     Theta_next = Theta;
62     x_ym_next = x_ym;
63     x_yf_next = x_yf;
64     x_rf_next = x_rf;
65     y_prev_next = y;
66     y_pb_next = y_pb_in;
67     r_pb_next = r_pb_in;
68     sw_next = sw_in;
69
70     return;
71 end
72
73 % transition
74 if sw_in < 0.5
75     y_pb_next = y;
76     r_pb_next = r;
77
78     x_ym_work = [0; 0];
79     x_yf_work = [0; 0];
80     x_rf_work = [0; 0];
81
82     Theta_work = Theta;
83
84     y_prev_tilde = 0;
85     sw_next = 1;
86 else
87     x_ym_work = x_ym;
88     x_yf_work = x_yf;
89     x_rf_work = x_rf;
90
91     Theta_work = Theta;
92
93     y_prev_tilde = y_prev - y_pb_in;
94 end
95
96 % Normalize
97 dy = y - y_pb_next;
98 dr = r - r_pb_next;
99
100 % deriv
101 ydot_tilde = (dy - y_prev_tilde) / Ts;
102
103 % output
104 ym = C_ym * x_ym_work;
105 yf_full = C_yf * x_yf_work;
106 yf = yf_full(1);
107 ydot_f = yf_full(2);
108 rf = C_rf * x_rf_work;
109
110 % error
111 e = dy - ym;
112
113 omega = [dy; ydot_tilde; dr];
114

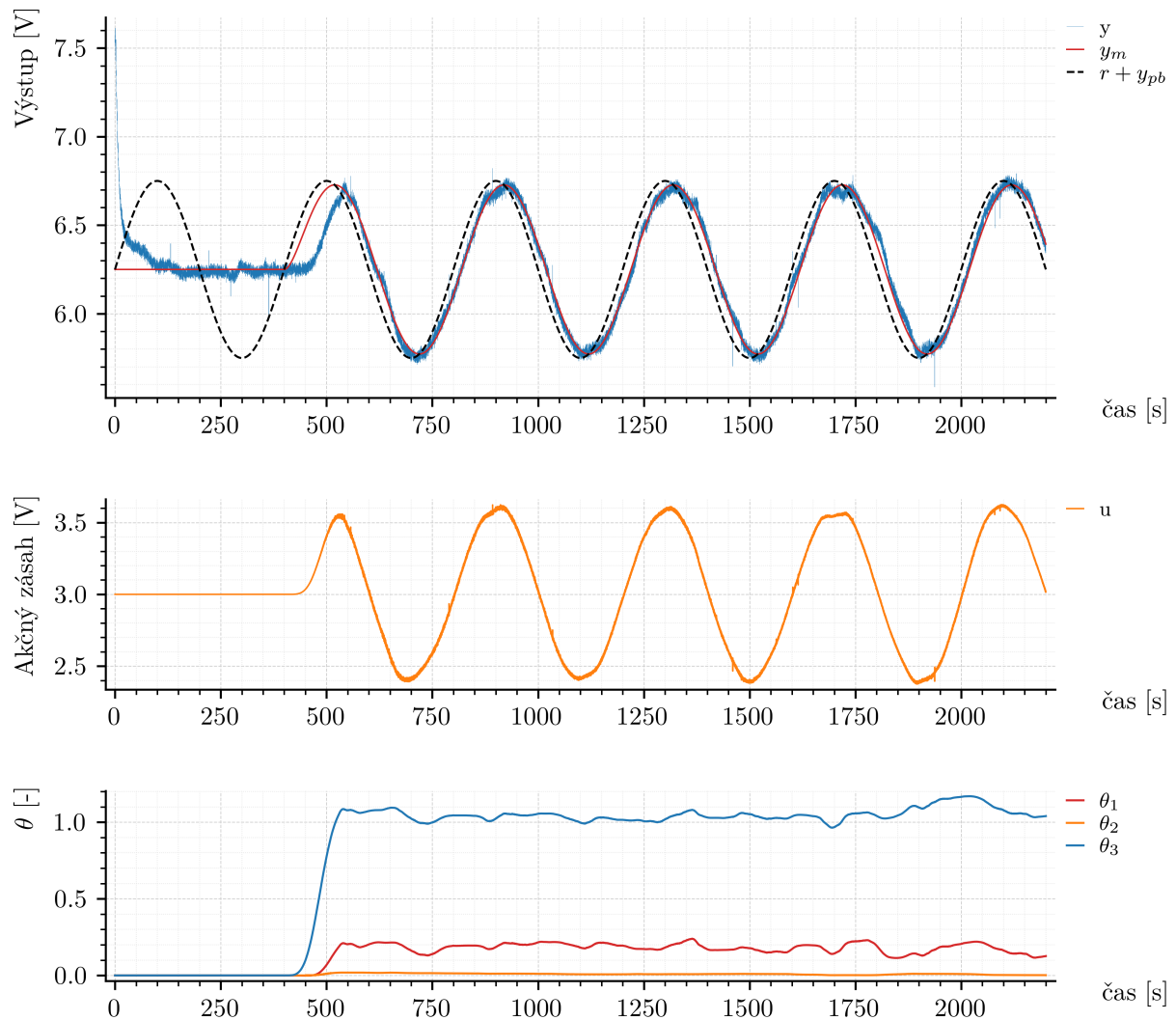
```

```

115 du = Theta_work.' * omega;
116 u_unsat = u_pb + du;
117 u = min(max(u_unsat, u_min), u_max);
118
119 % adapt law
120 dTheta = -sign_b0*e*[alpha1 * yf;
121                  alpha2 * ydot_f;
122                  alpha3 * rf];
123
124 Theta_next = Theta_work + dTheta * Ts;
125 Theta_used = Theta_work;
126
127 % State updates
128 x_ym_next = x_ym_work + Ts * (A * x_ym_work + b * dr);
129 x_yf_next = x_yf_work + Ts * (A * x_yf_work + b * dy);
130 x_rf_next = x_rf_work + Ts * (A * x_rf_work + b * dr);
131
132 y_prev_next = y;
133
134 end

```

Výpis kódu 22: Implementácia gradientného MRAC regulátora v bloku MATLAB Function



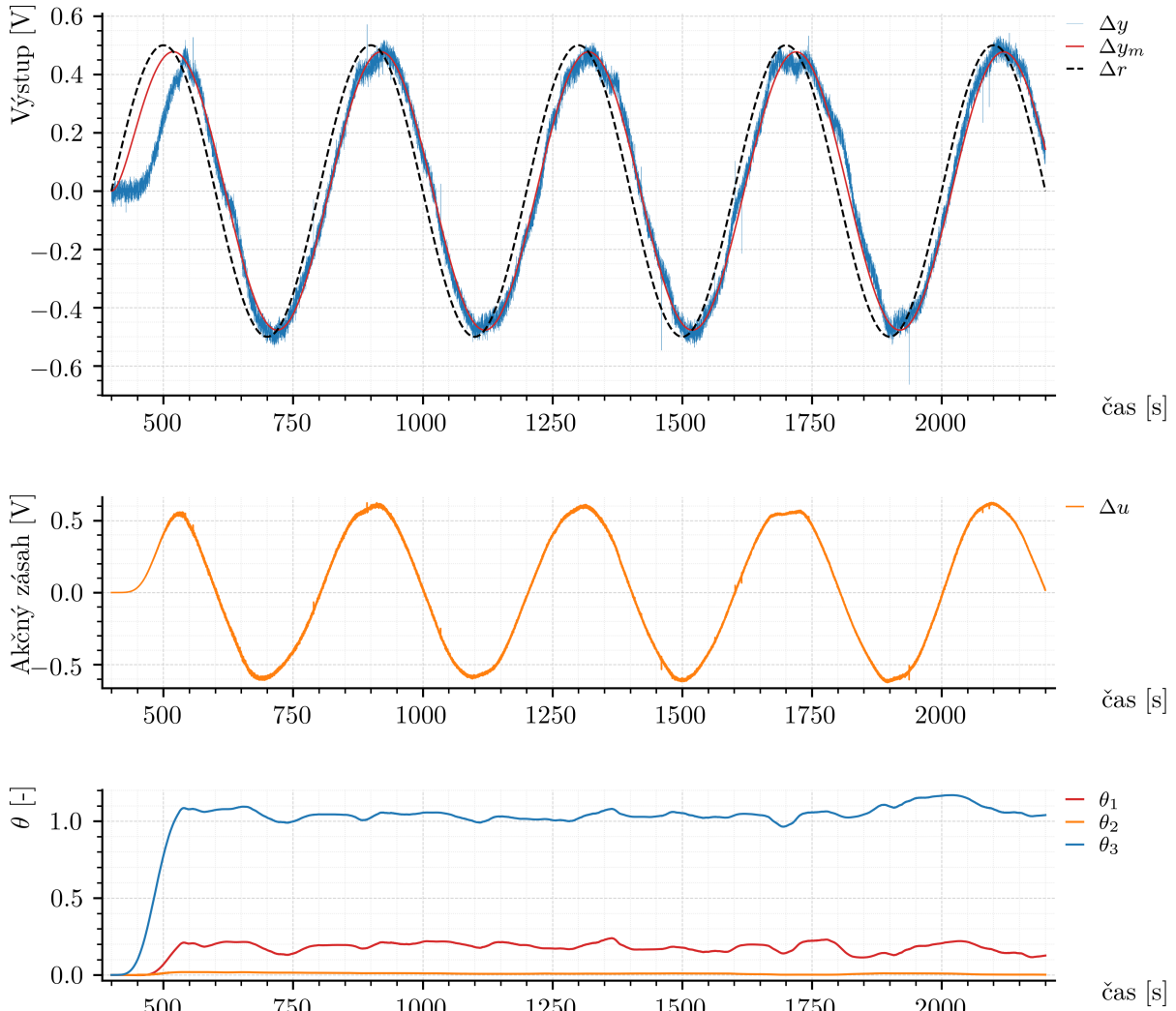
Obr. 40: Experimentálne výsledky gradientného MRAC regulátora v absolútnych veličinách

5.2.1 Overenie na reálnom systéme

Navrhnutý gradientný MRAC regulátor bol najskôr overený na modeli získanom identifikáciou (2.12) a následne aplikovaný aj na reálnu tepelnú sústavu TS05. Cieľom bolo overiť, či implementovaný adaptačný mechanizmus zabezpečuje sledovanie výstupu referenčného modelu pri súčasnej adaptácii parametrov regulátora.

Na obr. 40 sú priebehy v absolútnych veličinách. V hornom grafe sú zobrazené priebehy $y(t)$, $y_m(t)$ a $r(t)$. Počas prvých 400 s prebieha fáza ustálenia pri konštantnom riadiacom zásahu, pričom systém nereaguje na zmeny referencie a veličiny sa ustália v okolí pracovného bodu. V druhom grafe je riadiaci zásah $u(t)$, ktorý je počas tejto fázy konštantný. Po uplynutí času T_{settle} sa aktivuje adaptačný zákon a riadiaci zásah sa začne meniť. V treťom grafe sú priebehy parametrov Θ_1 , Θ_2 a Θ_3 . Počas ustálenia sú konštantné, po prepnutí dochádza k ich adaptácii, čo potvrdzuje činnosť adaptačného mechanizmu.

Detail adaptačnej fázy je uvedený na obr. 41, kde sú priebehy vyjadrené ako odchýlky od pracovného bodu a fáza ustálenia nie je zobrazená. V hornom grafe sú veličiny $\Delta y(t)$, $\Delta y_m(t)$ a $\Delta r(t)$, v druhom $\Delta u(t)$ a v treťom priebehy parametrov Θ . Z priebehov je zrejmé, že výstup systému sleduje výstup referenčného modelu bez výrazných oscilácií a parametre regulátora sa počas deja prispôsobujú. Adaptačné koeficienty boli zvolené experimentálne na základe správania reálnej sústavy (5.77), čo umožnilo dosiahnuť s uspokojivou presnosťou sledovanie výstupu referenčného modelu.



Obr. 41: Experimentálne výsledky gradientného MRAC regulátora po prechode do súradníc pracovného bodu

Na základe experimentu možno konštatovať, že regulátor zabezpečuje sledovanie referenčného modelu pri súčasnej adaptácii parametrov a zachováva adaptačný charakter riadenia aj pri aplikácii na reálnu sústavu.

5.3 Implementácia: mikrokontrolér a jazyk C++

Po overení funkčnosti gradientného MRAC regulátora v prostredí MATLAB/Simulink bola následne realizovaná aj jeho implementácia v jazyku C++ na platforme Arduino UNO. Všeobecný princíp komunikácie so systémom TS05, spôsob časovania hlavného cyklu programu, kalibrácia analógových vstupov a výstupov a záznam dát cez sériový port boli už opísané v podkapitole 2.3.2. V tejto časti sa preto pozornosť sústreďuje už iba na samotnú realizáciu adaptačného regulačného algoritmu.

Rovnako ako pri implementácii PI regulátora a prepínacej logiky pracovných bodov, aj v tomto prípade bola regulačná časť vložená priamo do hlavného riadiaceho cyklu programu. Na rozdiel od prostredia MATLAB Function, kde bolo potrebné všetky vnútorné stavy prenášať medzi jednotlivými krokmi výpočtu cez vstupy a výstupy bloku, v jazyku C++ sú tieto veličiny uchovávané priamo vo vnútorných premenných programu. Týka sa to najmä vektora parametrov Θ , stavov referenčného modelu a filtračných blokov, predchádzajúcej hodnoty výstupu, ako aj uložených hodnôt pracovného bodu.

Referenčný signál bol v experimente generovaný harmonickým priebehom v tvare

$$r(t) = 0.5 \sin\left(\frac{2\pi t}{400}\right), \quad (5.79)$$

pričom počiatočná fáza ustálenia systému zostala rovnaká ako pri implementácii v prostredí MATLAB. Počas intervalu $T_{\text{settle}} = 400$ s je na vstup systému privádzaný konštantný zásah $u_{pb} = 3$ V, čím sa sústava uvedie do okolia zvoleného pracovného bodu. Až po uplynutí tohto času sa aktivuje samotný adaptačný mechanizmus.

Z hľadiska implementácie bolo najprv potrebné doplniť niekoľko jednoduchých pomocných funkcií pre výpočty so stavovým modelom a s regresným vektorom. Keďže všetky dynamické bloky MRAC algoritmu sú druhého rádu, postačuje základná implementácia násobenia matice a vektora a skalárnych súčinov. Použité pomocné funkcie sú uvedené vo výpise 23.

```

1 // 2x2 matrix times 2x1 vector
2 void mat2x2Vec2Mul(const float A[2][2], const float x[2], float result[2]) {
3     result[0] = A[0][0] * x[0] + A[0][1] * x[1];
4     result[1] = A[1][0] * x[0] + A[1][1] * x[1];
5 }
6
7 // Dot product of 1x2 and 2x1
8 float rowVec2ColVec2Mul(const float c[2], const float x[2]) {
9     return c[0] * x[0] + c[1] * x[1];
10 }
11
12 // Dot product of 1x3 and 3x1
13 float rowVec3ColVec3Mul(const float a[3], const float b[3]) {
14     return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
15 }

```

Výpis kódu 23: Pomocné funkcie použité pri implementácii gradientného MRAC regulátora v jazyku C++

Po definovaní pomocných funkcií boli inicializované všetky parametre regulátora, stavové matice referenčného modelu a filtračných blokov, ako aj vnútorné stavové premenné algoritmu. Na rozdiel od PI regulátora ide v tomto prípade o väčší počet premenných, pretože implementácia musí uchovávať tri adaptačné parametre, tri dynamické bloky druhého rádu a viacero pomocných signálov používaných pri výpočte regulačného zásahu a adaptačného zákona. Táto inicializácia je uvedená vo výpise 24.

```

1 constexpr float PI_F = 3.14159265358979323846f;
2 constexpr float R_PERIOD = 400.0f;
3 constexpr float R_AMPLITUDE = 0.5f;
4

```

```

5 float vent = 6.0f;
6 float spirala = 0.0f;
7
8 // MRAC parameters
9 constexpr float AOM = 1.0f;
10 constexpr float AIM = 20.0f;
11 constexpr float BOM = 1.0f;
12
13 constexpr float ALPHA1 = 14.0f * 0.01f;
14 constexpr float ALPHA2 = 30.0f * 0.01f;
15 constexpr float ALPHA3 = 14.0f * 0.01f;
16
17 constexpr float SIGN_BO = 1.0f;
18 constexpr float TS = 0.1f;
19
20 constexpr float U_pb = 3.0f;
21 constexpr float T_SETTLE = 400.0f;
22
23 constexpr float U_MIN = 0.0f;
24 constexpr float U_MAX = 10.0f;
25
26 // State space matrices
27 const float A_MRAC[2][2] = {
28     {0.0f, 1.0f},
29     {-AOM, -AIM}
30 };
31
32 const float B_MRAC[2] = {
33     0.0f,
34     1.0f
35 };
36
37 const float C_RF[2] = {
38     1.0f,
39     0.0f
40 };
41
42 const float C_YM[2] = {
43     BOM,
44     0.0f
45 };
46
47 // MRAC states
48 float Theta[3] = {0.0f, 0.0f, 0.0f};
49 float Theta_used[3] = {0.0f, 0.0f, 0.0f};
50 float Theta_next_arr[3] = {0.0f, 0.0f, 0.0f};
51
52 float x_ym[2] = {0.0f, 0.0f};
53 float x_yf[2] = {0.0f, 0.0f};
54 float x_rf[2] = {0.0f, 0.0f};
55
56 float x_ym_next_arr[2] = {0.0f, 0.0f};
57 float x_yf_next_arr[2] = {0.0f, 0.0f};
58 float x_rf_next_arr[2] = {0.0f, 0.0f};
59
60 float y_prev = 0.0f;
61 float y_prev_next = 0.0f;
62
63 float y_pb = 0.0f;
64 float r_pb = 0.0f;
65 float y_pb_next = 0.0f;
66 float r_pb_next = 0.0f;
67
68 float sw = 0.0f;
69 float sw_next = 0.0f;
70
71 // MRAC signals for logging
72 float r = 0.0f;
73 float y = 0.0f;
74 float ym = 0.0f;
75 float yf = 0.0f;
76 float ydot_f = 0.0f;
77 float rf = 0.0f;
78 float e = 0.0f;
79
80 float dy = 0.0f;
81 float dr = 0.0f;
82 float dy_prev = 0.0f;

```



```

83 float dy_dot = 0.0f;
84
85 float omega[3] = {0.0f, 0.0f, 0.0f};
86
87 float u = U_pb;
88 float du = 0.0f;
89 float u_unsat = U_pb;
90 float dTheta[3] = {0.0f, 0.0f, 0.0f};

```

Výpis kódu 24: Inicializácia stavových premenných gradientného MRAC regulátora v jazyku C++

Samotná implementácia riadiaceho algoritmu bola následne vložená do hlavnej časti programu. Na začiatku každého vzorkovacieho kroku sa najprv načíta aktuálny výstup systému y , vygeneruje sa referencia $r(t)$ a pripraví sa predvolené hodnoty výstupov a vnútorných stavov. Počas úvodnej fázy ustálenia zostáva na výstupe hodnota u_{pb} a adaptačný zákon nie je aktívny. Po ukončení ustálenia sa pri prvom prechode do adaptívneho režimu uložia hodnoty pracovného bodu y_{pb} a r_{pb} , nulovo sa inicializujú dynamické bloky a aktivuje sa prepínací príznak sw .

V ďalších krokoch sa už počítajú odchýlkové veličiny Δy a Δr , numerická derivácia výstupu, výstupy referenčného modelu a filtračných blokov, regresný vektor ω , samotný akčný zásah a následne aj prírastky parametrov adaptácie. Stavové rovnice všetkých troch dynamických blokov sú integrované explicitnou Eulerovou metódou. Implementovaný kód hlavnej regulačnej logiky je uvedený vo výpise 25.

```

1  // MAIN CONTROL LOGIC
2  ///////////////////////////////////////////////////
3  // Measured output
4  y = snimac1;
5
6  // Reference generation
7  r = R_AMPLITUDE * sinf(2.0f * PI_F * globalTime / R_PERIOD);
8
9  // Default outputs
10 u = U_pb;
11 ym = 0.0f;
12
13 Theta_used[0] = Theta[0];
14 Theta_used[1] = Theta[1];
15 Theta_used[2] = Theta[2];
16
17 Theta_next_arr[0] = Theta[0];
18 Theta_next_arr[1] = Theta[1];
19 Theta_next_arr[2] = Theta[2];
20
21 x_ym_next_arr[0] = x_ym[0];
22 x_ym_next_arr[1] = x_ym[1];
23
24 x_yf_next_arr[0] = x_yf[0];
25 x_yf_next_arr[1] = x_yf[1];
26
27 x_rf_next_arr[0] = x_rf[0];
28 x_rf_next_arr[1] = x_rf[1];
29
30 y_prev_next = y;
31 y_pb_next = y_pb;
32 r_pb_next = r_pb;
33 sw_next = sw;
34
35 // Plant settle
36 if (globalTime < T_SETTLE) {
37     u = U_pb;
38     ym = 0.0f;
39
40     y_prev_next = y;
41     y_pb_next = y_pb;
42     r_pb_next = r_pb;
43     sw_next = sw;
44 } else {
45     float x_ym_work[2];
46     float x_yf_work[2];
47     float x_rf_work[2];
48     float Theta_work[3];
49

```

```

50 // Transition
51 if (sw < 0.5f) {
52     y_pb_next = y;
53     r_pb_next = r;
54
55     x_ym_work[0] = 0.0f;
56     x_ym_work[1] = 0.0f;
57
58     x_yf_work[0] = 0.0f;
59     x_yf_work[1] = 0.0f;
60
61     x_rf_work[0] = 0.0f;
62     x_rf_work[1] = 0.0f;
63
64     Theta_work[0] = Theta[0];
65     Theta_work[1] = Theta[1];
66     Theta_work[2] = Theta[2];
67
68     dy_prev = 0.0f;
69     sw_next = 1.0f;
70 } else {
71     x_ym_work[0] = x_ym[0];
72     x_ym_work[1] = x_ym[1];
73
74     x_yf_work[0] = x_yf[0];
75     x_yf_work[1] = x_yf[1];
76
77     x_rf_work[0] = x_rf[0];
78     x_rf_work[1] = x_rf[1];
79
80     Theta_work[0] = Theta[0];
81     Theta_work[1] = Theta[1];
82     Theta_work[2] = Theta[2];
83
84     dy_prev = y_prev - y_pb;
85 }
86
87 // Normalize
88 dy = y - y_pb_next;
89 dr = r - r_pb_next;
90
91 // Derivative
92 dy_dot = (dy - dy_prev) / timeTickPeriod;
93
94 // Outputs of dynamic blocks
95 ym = rowVec2ColVec2Mul(C_YM, x_ym_work);
96 yf = x_yf_work[0];
97 ydot_f = x_yf_work[1];
98 rf = rowVec2ColVec2Mul(C_RF, x_rf_work);
99
100 // Error
101 e = dy - ym;
102
103 // Regressor
104 omega[0] = dy;
105 omega[1] = dy_dot;
106 omega[2] = dr;
107
108 // Control law
109 du = rowVec3ColVec3Mul(Theta_work, omega);
110 u_unsat = U_pb + du;
111 u = clampFloat(u_unsat, U_MIN, U_MAX);
112
113 // Adaptation law
114 dTheta[0] = -SIGN_B0 * e * (ALPHA1 * yf);
115 dTheta[1] = -SIGN_B0 * e * (ALPHA2 * ydot_f);
116 dTheta[2] = -SIGN_B0 * e * (ALPHA3 * rf);
117
118 Theta_next_arr[0] = Theta_work[0] + dTheta[0] * timeTickPeriod;
119 Theta_next_arr[1] = Theta_work[1] + dTheta[1] * timeTickPeriod;
120 Theta_next_arr[2] = Theta_work[2] + dTheta[2] * timeTickPeriod;
121
122 Theta_used[0] = Theta_work[0];
123 Theta_used[1] = Theta_work[1];
124 Theta_used[2] = Theta_work[2];
125
126 // State updates
127 float Ax[2];

```

```

128     float bu[2];
129
130     mat2x2Vec2Mul(A_MRAC, x_ym_work, Ax);
131     bu[0] = B_MRAC[0] * dr;
132     bu[1] = B_MRAC[1] * dr;
133     x_ym_next_arr[0] = x_ym_work[0] + timeTickPeriod * (Ax[0] + bu[0]);
134     x_ym_next_arr[1] = x_ym_work[1] + timeTickPeriod * (Ax[1] + bu[1]);
135
136     mat2x2Vec2Mul(A_MRAC, x_yf_work, Ax);
137     bu[0] = B_MRAC[0] * dy;
138     bu[1] = B_MRAC[1] * dy;
139     x_yf_next_arr[0] = x_yf_work[0] + timeTickPeriod * (Ax[0] + bu[0]);
140     x_yf_next_arr[1] = x_yf_work[1] + timeTickPeriod * (Ax[1] + bu[1]);
141
142     mat2x2Vec2Mul(A_MRAC, x_rf_work, Ax);
143     bu[0] = B_MRAC[0] * dr;
144     bu[1] = B_MRAC[1] * dr;
145     x_rf_next_arr[0] = x_rf_work[0] + timeTickPeriod * (Ax[0] + bu[0]);
146     x_rf_next_arr[1] = x_rf_work[1] + timeTickPeriod * (Ax[1] + bu[1]);
147
148     y_prev_next = y;
149 }
150
151 // Commit next states after MRAC step
152 Theta[0] = Theta_next_arr[0];
153 Theta[1] = Theta_next_arr[1];
154 Theta[2] = Theta_next_arr[2];
155
156 x_ym[0] = x_ym_next_arr[0];
157 x_ym[1] = x_ym_next_arr[1];
158
159 x_yf[0] = x_yf_next_arr[0];
160 x_yf[1] = x_yf_next_arr[1];
161
162 x_rf[0] = x_rf_next_arr[0];
163 x_rf[1] = x_rf_next_arr[1];
164
165 y_prev = y_prev_next;
166 y_pb = y_pb_next;
167 r_pb = r_pb_next;
168 sw = sw_next;
169
170 // Final output
171 spirala = u;

```

Výpis kódu 25: Hlavná regulačná logika gradientného MRAC regulátora v jazyku C++

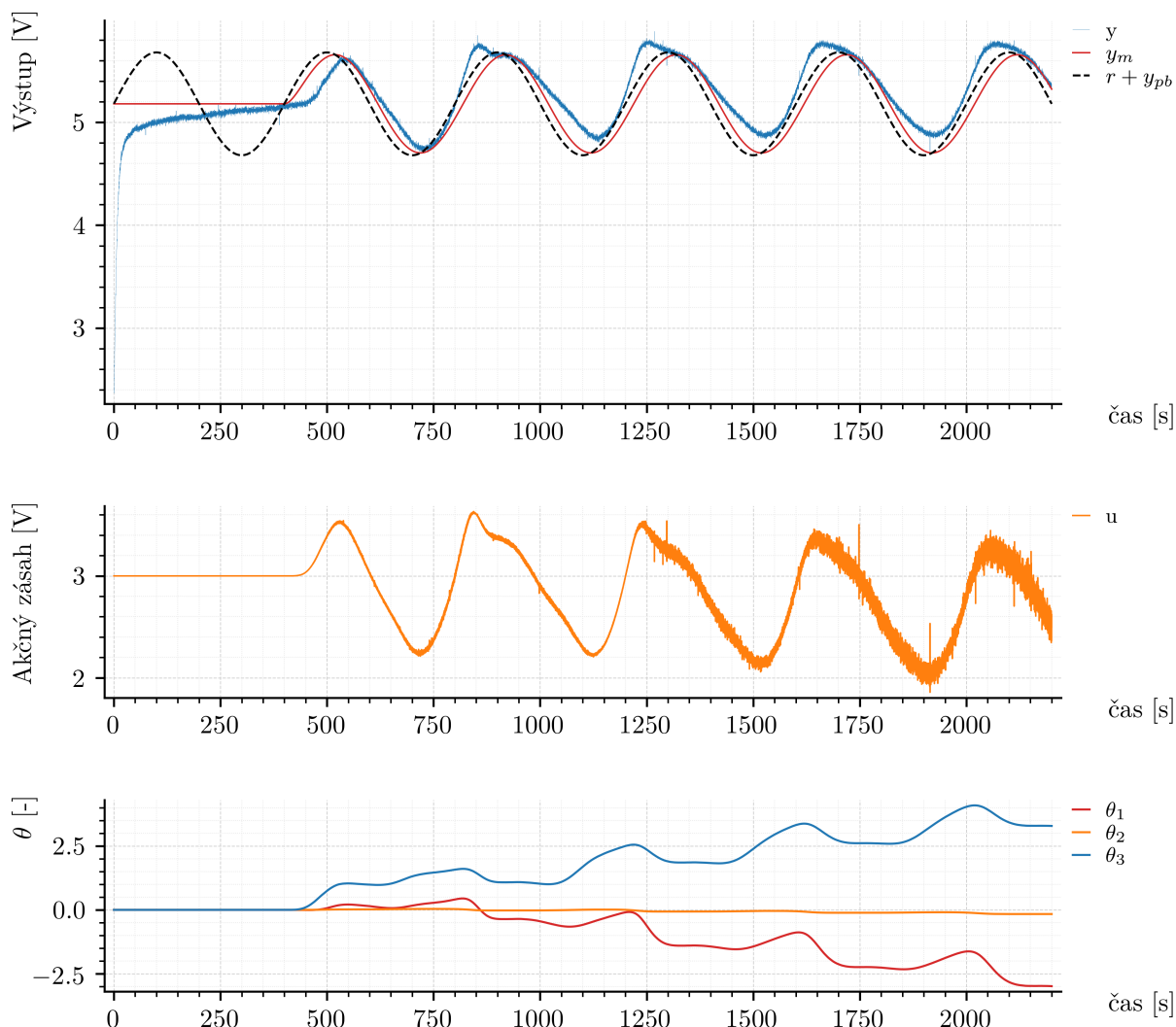
Z hľadiska numerickej realizácie zostáva princíp výpočtu rovnaký ako v prostredí MATLAB, avšak implementácia v jazyku C++ využíva pri aktualizácii stavových premenných reálne nameranú dĺžku vykonaného kroku `timeTickPeriod` namiesto konštantnej hodnoty T_s . Tým sa čiastočne zohľadňujú malé odchýlky skutočnej periódy vykonávania programu od nominálnej hodnoty 0.1 s. Súčasne zostáva zachovaná rovnaká logika prechodu do súradníc pracovného bodu, rovnaký referenčný model aj rovnaký adaptačný zákon ako pri implementácii v prostredí MATLAB/Simulink.

5.3.1 Overenie na reálnom systéme

Na základe uvedenej implementácie bol vykonaný experiment na reálnom systéme TS05. Celkový priebeh experimentu v absolútnych veličinách je zobrazený na obr. 42. V hornom grafe sú znázornené priebehy výstupu systému $y(t)$, výstupu referenčného modelu $y_m(t)$ a referencie $r(t)$. V druhom grafe je zobrazený akčný zásah $u(t)$ a v treťom priebehy parametrov Θ_1 , Θ_2 a Θ_3 .

Z obr. 42 je zrejmé, že počas prvých 400 s prebieha ustálenie systému pri konštantnom zásahu u_{pb} . Po ukončení tejto fázy sa aktivuje adaptačný mechanizmus, začne sa meniť akčný zásah a parametre regulátora sa prispôbujú aktuálnemu správaniu sústavy. Tým sa potvrdzuje správna funkcia implementovaného prepnutia medzi fázou ustálenia a vlastnou adaptačnou reguláciou.

Pre detailnejšie posúdenie správania počas samotnej adaptívnej fázy je na obr. 43 zobrazený priebeh po prechode do súradníc pracovného bodu. V hornom grafe sú



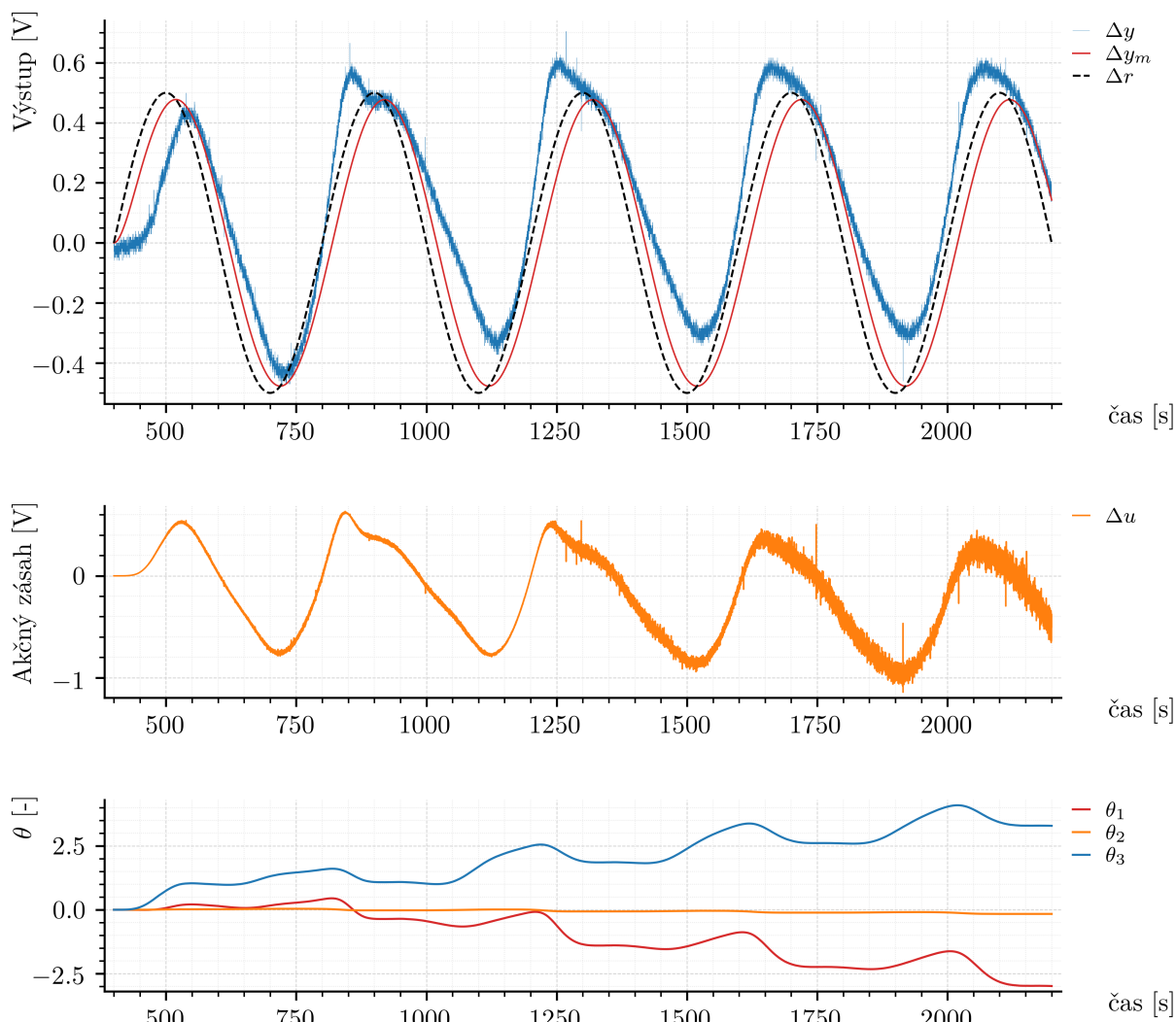
Obr. 42: Experimentálne výsledky gradientného MRAC regulátora implementovaného v jazyku C++ v absolútnych veličinách

znázornené odchýlkové veličiny $\Delta y(t)$, $\Delta y_m(t)$ a $\Delta r(t)$, v druhom priebehu $\Delta u(t)$ a v treťom priebehu adaptačných parametrov.

Z priebehov možno pozorovať, že bezprostredne po aktivácii adaptácie regulátor spočiatkovo sleduje referenčný model pomerne uspokojivo. Výstup systému sa v úvodnej časti experimentu viaže na priebeh $\Delta y_m(t)$ bez výrazných prechodových javov a adaptačné parametre sa menia spojitاً. V neskoršej fáze experimentu však možno pozorovať, že parametre Θ sa neustávajú na približne konštantných hodnotách, ale začínajú postupne driftovať. Tento jav sa následne prejavuje aj na zhoršení kvality sledovania referenčného modelu.

Presná príčina tohto správania nie je z jediného experimentu jednoznačne určiteľná, pričom pravdepodobne ide o kombináciu viacerých vplyvov. Jednou z možností je citlivosť gradientného adaptačného zákona na merací šum, ktorý sa výraznejšie prejavuje najmä v signáli numerickej derivácie $\Delta \dot{y}$. Ďalším faktorom môže byť pomalý drift pracovného bodu tepelnej sústavy, spôsobený zmenami teploty okolia a dlhodobým tepelným nahrievaním systému. Svoju úlohu môže zohrávať aj nepresná zhoda medzi reálnou sústavou a zvoleným referenčným modelom, ako aj skutočnosť, že v implementovanom adaptačnom zákone nebola použitá žiadna dodatočná stabilizačná úprava, napríklad normalizácia regresora, projekčný algoritmus alebo σ -modifikácia.

Na základe experimentu teda možno konštatovať, že implementácia gradientného MRAC regulátora v jazyku C++ potvrdila správnosť samotnej programovej realizácie



Obr. 43: Experimentálne výsledky gradientného MRAC regulátora implementovaného v jazyku C++ po vynechaní fázy ustálenia

algoritmu a jeho počiatočnú schopnosť sledovať referenčný model aj mimo prostredia MATLAB/Simulink. Súčasne sa však ukázalo, že pri dlhšej prevádzke na reálnom tepelnom systéme sa prejavuje drift adaptačných parametrov, čo poukazuje na praktické obmedzenia použitého gradientného adaptačného zákona v tejto jednoduchšej forme.

5.3.2 Heuristická modifikácia na potlačenie driftu adaptovaných parametrov

V predchádzajúcej časti bolo ukázané, že pri dlhšej prevádzke gradientného MRAC regulátora na reálnom systéme TS05 dochádza k driftu adaptačných parametrov Θ . Z tohto dôvodu bola následne overená jednoduchá heuristická modifikácia implementácie, ktorej cieľom bolo tento jav prakticky potlačiť.

Nejde teda o zmenu samotného princípu MRAC regulácie, ale o doplnkovú inžiniersku modifikáciu existujúcej implementácie. Základný zákon riadenia zostal nezmenený, rovnako ako referenčný model, štruktúra regresného vektora aj adaptačné koeficienty $\alpha_1, \alpha_2, \alpha_3$ uvedené v (5.77). Modifikácia sa týkala iba dvoch praktických oblastí:

- spracovania meraného výstupného signálu,
- potlačenia driftu v adaptačnom mechanizme.

Filtrácia šumu spätnoväzbového signálu

Prvá úprava spočívala v tom, že namiesto použitia jedinej okamžitej hodnoty meraného signálu zo snímača sa začalo používať priemerovanie posledných n_y vzoriek uložených v kruhovom buffri. Ak označíme surový meraný výstup ako $y_{\text{raw}}(k)$ a počet použitých vzoriek v danom okamihu ako $M(k)$, potom vyhladený výstup možno zapísať v tvare

$$y_{\text{avg}}(k) = \frac{1}{M(k)} \sum_{i=0}^{M(k)-1} y_{\text{raw}}(k-i), \quad M(k) \leq n_y. \quad (5.80)$$

V samotnom algoritme sa potom ako aktuálny výstup použije

$$y(k) = y_{\text{avg}}(k). \quad (5.81)$$

V implementácii bola zvolená veľkosť okna

$$n_y = 8. \quad (5.82)$$

Cieľom tejto úpravy bolo zmenšiť vplyv meracieho šumu a kvantovania A/D prevodníka, ktoré sa inak priamo prenášajú do výpočtu regulačnej chyby a najmä do diskretnej aproximácie derivácie výstupu.

Pásmo necitlivosti v zákone adaptácie

Druhá úprava sa týkala adaptačného zákona. Samotná regulačná chyba zostala definovaná v pôvodnom tvare

$$e(k) = \Delta y(k) - y_m(k), \quad (5.83)$$

pričom ani zákon riadenia sa nezmenil:

$$\Delta u(k) = \Theta^T(k) \omega(k), \quad u(k) = \text{sat}(u_{pb} + \Delta u(k)). \quad (5.84)$$

Zmena bola vykonaná iba v adaptačnom mechanizme. Namiesto priameho použitia chyby $e(k)$ sa zaviedla modifikovaná chyba $e_{\text{ad}}(k)$ s dead-zone nonlinearitou:

$$e_{\text{ad}}(k) = \begin{cases} e(k) - E_{\text{dead}}, & \text{ak } e(k) > E_{\text{dead}}, \\ e(k) + E_{\text{dead}}, & \text{ak } e(k) < -E_{\text{dead}}, \\ 0, & \text{ak } |e(k)| \leq E_{\text{dead}}. \end{cases} \quad (5.85)$$

V implementácii bola zvolená hodnota prahu

$$E_{\text{dead}} = 0.02. \quad (5.86)$$

Adaptácia parametrov potom prebieha podľa vzťahu

$$\dot{\Theta}(k) = -\text{sign}(b_0) e_{\text{ad}}(k) \begin{bmatrix} \alpha_1 y_f(k) \\ \alpha_2 \dot{y}_f(k) \\ \alpha_3 r_f(k) \end{bmatrix}. \quad (5.87)$$

Zmyslom tejto úpravy je, že pri veľmi malej chybe sa adaptácia úplne vypne a pri väčšej chybe sa jej účinok zmenší o pevnú prahovú hodnotu. Takto sa obmedzí postupné hromadenie parametrov spôsobené malou, ale dlhodobo prítomnou chybou, ktorá môže mať pôvod v šume, numerickej derivácii alebo v pomalom drifte pracovného bodu.

Doplnenie implementácie

Na realizáciu uvedených úprav bolo potrebné doplniť dve pomocné funkcie pre obsluhu kruhového buffra a niekoľko nových globálnych premenných. Tieto doplnenia sú uvedené vo výpise 26.

```

1 // Circular buffer helpers for y averaging
2 void pushToRing(float* buffer, int size, int& index, int& count, float value) {
3     buffer[index] = value;
4     index = (index + 1) % size;
5     if (count < size) {
6         count++;
7     }
8 }
9
10 float meanRing(const float* buffer, int count) {
11     if (count <= 0) {
12         return 0.0f;
13     }
14
15     float sum = 0.0f;
16     for (int i = 0; i < count; ++i) {
17         sum += buffer[i];
18     }
19     return sum / static_cast<float>(count);
20 }
21
22 // Y averaging
23 constexpr int Y_AVG_WINDOW = 8;
24 float yHist[Y_AVG_WINDOW] = {0.0f};
25 int yHistIndex = 0;
26 int yHistCount = 0;
27
28 float y_raw = 0.0f;
29 float y_avg = 0.0f;
30
31 // Adaptation dead zone
32 constexpr float E_DEAD = 0.02f;
33 float e_ad = 0.0f;

```

Výpis kódu 26: Pomocné funkcie a nové premenné pre heuristickú modifikáciu MRAC regulátora

Po načítaní analógových vstupov sa do programu doplnilo priebežné ukladanie aktuálnej surovej hodnoty výstupu do kruhového buffra. Táto úprava je uvedená vo výpise 27.

```

1 float snimac2 = cubic(
2     static_cast<float>(a2),
3     -9.09109415e-10f,
4     8.99002333e-07f,
5     9.78867803e-03f,
6     -9.65193864e-05f
7 );
8
9 // Save latest raw output sample every real Arduino loop
10 y_raw = snimac1;
11 pushToRing(yHist, Y_AVG_WINDOW, yHistIndex, yHistCount, y_raw);

```

Výpis kódu 27: Doplnenie ukladania surového meraného výstupu do kruhového buffra

V hlavnej časti regulačného algoritmu sa potom namiesto okamžitej hodnoty snímača začal používať jej vyhladený priemer a súčasne bol adaptačný zákon upravený o dead-zone nelinearitu. Modifikovaný úsek regulačnej logiky je uvedený vo výpise 28.

```

1 // Measured output
2 y_avg = meanRing(yHist, yHistCount);
3 y = y_avg;
4
5 // Error
6 e = dy - ym;
7
8 // Regressor
9 omega[0] = dy;
10 omega[1] = dy_dot;
11 omega[2] = dr;
12
13 // Control law
14 du = rowVec3ColVec3Mul(Theta_work, omega);
15 u_unsat = U_pb + du;
16 u = clampFloat(u_unsat, U_MIN, U_MAX);
17
18 // Adaptation dead zone

```

```

19 e_ad = e;
20
21 if (e_ad > E_DEAD) {
22     e_ad -= E_DEAD;
23 } else if (e_ad < -E_DEAD) {
24     e_ad += E_DEAD;
25 } else {
26     e_ad = 0.0f;
27 }
28
29 // Adaptation law
30 dTheta[0] = -SIGN_B0 * e_ad * (ALPHA1 * yf);
31 dTheta[1] = -SIGN_B0 * e_ad * (ALPHA2 * ydot_f);
32 dTheta[2] = -SIGN_B0 * e_ad * (ALPHA3 * rf);

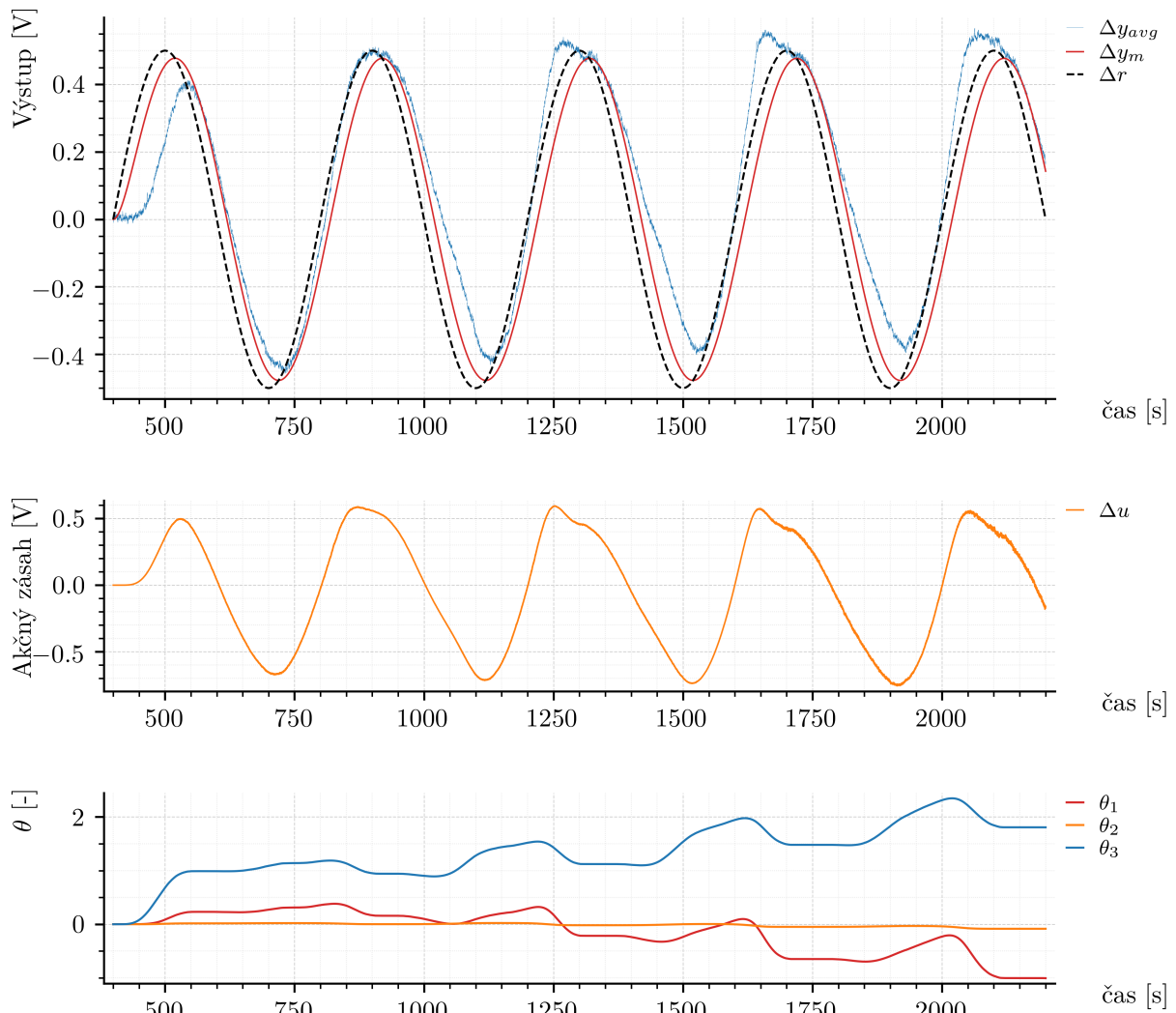
```

Výpis kódu 28: Modifikovaný úsek MRAC algoritmu s priemerovaním výstupu a dead-zone v adaptácii

Overenie modifikovaného algoritmu riadenia na reálnom systéme

Na základe tejto úpravy bol následne vykonaný nový experiment na reálnom systéme TS05. Je tu zobrazená už iba samotná regulačná fáza po ukončení úvodného ustálenia, teda bez prvých 400 s. Výsledok experimentu je uvedený na obr. 44.

Adaptačné koeficienty zostali pri tomto experimente rovnaké ako v pôvodnej implementácii, teda nebola menená ich hodnota ani samotná štruktúra adaptácie.



Obr. 44: Experimentálne výsledky heuristicky upravenej implementácie gradientného MRAC regulátora v jazyku C++

Z obr. 44 je zrejmé, že drift parametrov Θ sa po tejto úprave úplne neodstránil, avšak jeho intenzita sa zmenšila. Na priebehoch parametrov možno pozorovať miernejší drift ako v predchádzajúcom experimente. Zároveň je viditeľné, že akčný zásah u má pokojnejší priebeh a menšiu úroveň šumu.

Možno teda konštatovať, že navrhnutá heuristická modifikácia viedla k praktickému zlepšeniu správania implementovaného regulátora, avšak problém driftu parametrov úplne neodstránila.

5.4 Implementácia: programovateľný logický automat (PLC)

Po overení funkčnosti gradientného MRAC regulátora v prostredí MATLAB/Simulink a po jeho implementácii v jazyku C++ bola následne realizovaná aj jeho implementácia na platforme Siemens SIMATIC S7-1200 G2.

Samotná regulačná logika bola realizovaná vo funkčnom bloku FB_MRAC_TS05. Blok obsahuje všetky vnútorné stavové premenné algoritmu, teda adaptačné parametre Θ_1 , Θ_2 , Θ_3 , stavy referenčného modelu, stavy filtra výstupu a referenčného filtra, predchádzajúcu hodnotu výstupu, uloženú hodnotu pracovného bodu a prepínací príznak Sw. Okrem samotného riadiaceho bloku bol v programe použitý aj blok FB_MRAC_SETPPOINT na generovanie žiadanej hodnoty v čase a blok FB_MRAC_LOGGER_DB na záznam experimentálnych dát.

Podobne ako v predchádzajúcich implementáciách aj tu prechádza algoritmus najprv fázou ustálenia. Táto fáza bola realizovaná pomocou časovača TON, ktorý po uplynutí zvoleného času aktivuje samotnú adaptívnu reguláciu. Počas neaktívnej fázy zostáva na výstupe hodnota pracovného bodu u_{pb} a blok si zároveň uloží aktuálnu hodnotu výstupu pre korektný výpočet diskretné derivácie po prechode do aktívneho režimu.

Po aktivácii regulátora sa pri prvom kroku vykoná prechod do súradníc pracovného bodu. Uložia sa hodnoty y_{pb} a r_{pb} , nulovo sa inicializujú stavy dynamických blokov a nastaví sa prepínací príznak. V ďalších krokoch už algoritmus pracuje s odchýlkovými veličinami Δy a Δr , počíta ich diskretnú deriváciu, výstup referenčného modelu, výstupy filtračných blokov, samotný akčný zásah a následne aj prírastky adaptačných parametrov.

Implementácia funkčného bloku FB_MRAC_TS05 je uvedená v nasledujúcej ukážke.

```

1 // Default outputs
2 #U := #U_pb;
3 #Ym := 0.0;
4 #Vent_out := #Vent_in;
5
6 #Theta1_out := #Theta1;
7 #Theta2_out := #Theta2;
8 #Theta3_out := #Theta3;
9 #Y_pb_out := #Y_pb;
10
11 // Passive mode
12 IF NOT #Enable THEN
13     #U := #U_pb;
14     #Ym := 0.0;
15     // Store current output for future derivative consistency
16     #Y_prev := #Y;
17     RETURN;
18 END_IF;
19
20 // Default state propagation
21 #Theta1_work := #Theta1;
22 #Theta2_work := #Theta2;
23 #Theta3_work := #Theta3;
24
25 #Theta1_next := #Theta1;
26 #Theta2_next := #Theta2;
27 #Theta3_next := #Theta3;
28
29 #X_ym1_next := #X_ym1;
30 #X_ym2_next := #X_ym2;
31
32 #X_yf1_next := #X_yf1;
33 #X_yf2_next := #X_yf2;
```

```

34
35 #X_rf1_next := #X_rf1;
36 #X_rf2_next := #X_rf2;
37
38 #Y_prev_next := #Y;
39 #Y_pb_next := #Y_pb;
40 #R_pb_next := #R_pb;
41 #Sw_next := #Sw;
42
43 // Transition to operating-point coordinates
44 IF #Sw < 0.5 THEN
45     #Y_pb_next := #Y;
46     #R_pb_next := #R;
47     #X_ym1_work := 0.0;
48     #X_ym2_work := 0.0;
49     #X_yf1_work := 0.0;
50     #X_yf2_work := 0.0;
51     #X_rf1_work := 0.0;
52     #X_rf2_work := 0.0;
53     #Theta1_work := #Theta1;
54     #Theta2_work := #Theta2;
55     #Theta3_work := #Theta3;
56     #Dy_prev := 0.0;
57     #Sw_next := 1.0;
58 ELSE
59     #X_ym1_work := #X_ym1;
60     #X_ym2_work := #X_ym2;
61     #X_yf1_work := #X_yf1;
62     #X_yf2_work := #X_yf2;
63     #X_rf1_work := #X_rf1;
64     #X_rf2_work := #X_rf2;
65     #Theta1_work := #Theta1;
66     #Theta2_work := #Theta2;
67     #Theta3_work := #Theta3;
68     #Dy_prev := #Y_prev - #Y_pb;
69 END_IF;
70
71 // Normalize signals
72 #Dy := #Y - #Y_pb_next;
73 #Dr := #R - #R_pb_next;
74
75 // Backward-difference derivative
76 #Dy_dot := (#Dy - #Dy_prev) / #Ts;
77
78 // Dynamic block outputs
79 #Ym := #BOM * #X_ym1_work;
80 #Yf := #X_yf1_work;
81 #Ydot_f := #X_yf2_work;
82 #Rf := #X_rf1_work;
83
84 // Tracking error
85 #E := #Dy - #Ym;
86
87 // Control law
88 #Du := #Theta1_work * #Dy + #Theta2_work * #Dy_dot + #Theta3_work * #Dr;
89 #U_unsat := #U_pb + #Du;
90
91 // Saturation
92 IF #U_unsat > #U_max THEN
93     #U := #U_max;
94 ELSIF #U_unsat < #U_min THEN
95     #U := #U_min;
96 ELSE
97     #U := #U_unsat;
98 END_IF;
99
100 // Adaptation law
101 #DTheta1 := - #Sign_b0 * #E * (#Alpha1 * #Yf);
102 #DTheta2 := - #Sign_b0 * #E * (#Alpha2 * #Ydot_f);
103 #DTheta3 := - #Sign_b0 * #E * (#Alpha3 * #Rf);
104
105 #Theta1_next := #Theta1_work + #DTheta1 * #Ts;
106 #Theta2_next := #Theta2_work + #DTheta2 * #Ts;
107 #Theta3_next := #Theta3_work + #DTheta3 * #Ts;
108
109 // Reference model state update
110 #X_ym1_next := #X_ym1_work + #Ts * #X_ym2_work;
111 #X_ym2_next := #X_ym2_work + #Ts * (- #A0M * #X_ym1_work - #A1M * #X_ym2_work +

```

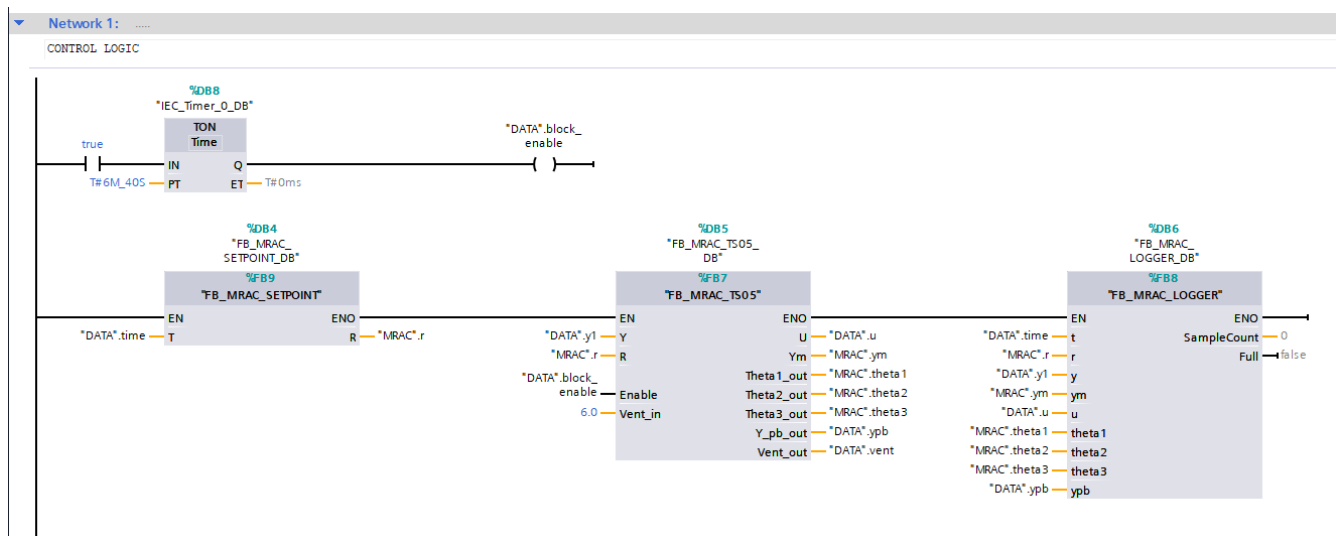
```

112     #Dr);
113 // Output filter state update
114 #X_yf1_next := #X_yf1_work + #Ts * #X_yf2_work;
115 #X_yf2_next := #X_yf2_work + #Ts * (- #A0M * #X_yf1_work - #A1M * #X_yf2_work +
116     #Dy);
117 // Reference filter state update
118 #X_rf1_next := #X_rf1_work + #Ts * #X_rf2_work;
119 #X_rf2_next := #X_rf2_work + #Ts * (- #A0M * #X_rf1_work - #A1M * #X_rf2_work +
120     #Dr);
121 // Commit next states
122 #Theta1 := #Theta1_next;
123 #Theta2 := #Theta2_next;
124 #Theta3 := #Theta3_next;
125
126 #X_ym1 := #X_ym1_next;
127 #X_ym2 := #X_ym2_next;
128
129 #X_yf1 := #X_yf1_next;
130 #X_yf2 := #X_yf2_next;
131
132 #X_rf1 := #X_rf1_next;
133 #X_rf2 := #X_rf2_next;
134
135 #Y_prev := #Y_prev_next;
136 #Y_pb := #Y_pb_next;
137 #R_pb := #R_pb_next;
138 #Sw := #Sw_next;
139
140 // Outputs for logging
141 #Theta1_out := #Theta1_work;
142 #Theta2_out := #Theta2_work;
143 #Theta3_out := #Theta3_work;
144 #Y_pb_out := #Y_pb;

```

Výpis kódu 29: Implementácia funkčného bloku FB_MRAC_TS05 v PLC Siemens

Schéma zapojenia algoritmu v hlavnom organizačnom bloku, vrátane bloku pre generovanie referencie, samotného MRAC regulátora, logovania a časovača, je uvedená na obr. 45. Zoznam premenných funkčného bloku je uvedený v tab. 14.



Obr. 45: Zapojenie gradientného MRAC regulátora v prostredí TIA Portal

Tabuľka 14: Premenné funkčného bloku FB_MRAC_TS05

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Input	Y	Real	0.0	Non-retain
Input	R	Real	0.0	Non-retain
Input	Enable	Bool	false	Non-retain
Input	Vent_in	Real	0.0	Non-retain
Output	U	Real	0.0	Non-retain
Output	Ym	Real	0.0	Non-retain
Output	Theta1_out	Real	0.0	Non-retain
Output	Theta2_out	Real	0.0	Non-retain
Output	Theta3_out	Real	0.0	Non-retain
Output	Y_pb_out	Real	0.0	Non-retain
Output	Vent_out	Real	0.0	Non-retain
Static	Theta1	Real	0.0	Non-retain
Static	Theta2	Real	0.0	Non-retain
Static	Theta3	Real	0.0	Non-retain
Static	X_ym1	Real	0.0	Non-retain
Static	X_ym2	Real	0.0	Non-retain
Static	X_yf1	Real	0.0	Non-retain
Static	X_yf2	Real	0.0	Non-retain
Static	X_rf1	Real	0.0	Non-retain
Static	X_rf2	Real	0.0	Non-retain
Static	Y_prev	Real	0.0	Non-retain
Static	Y_pb	Real	0.0	Non-retain
Static	R_pb	Real	0.0	Non-retain
Static	Sw	Real	0.0	Non-retain
Temp	Theta1_work	Real	—	—
Temp	Theta2_work	Real	—	—
Temp	Theta3_work	Real	—	—
Temp	X_ym1_work	Real	—	—
Temp	X_ym2_work	Real	—	—
Temp	X_yf1_work	Real	—	—
Temp	X_yf2_work	Real	—	—
Temp	X_rf1_work	Real	—	—
Temp	X_rf2_work	Real	—	—
Temp	Theta1_next	Real	—	—
Temp	Theta2_next	Real	—	—
Temp	Theta3_next	Real	—	—
Temp	X_ym1_next	Real	—	—
Temp	X_ym2_next	Real	—	—
Temp	X_yf1_next	Real	—	—
Temp	X_yf2_next	Real	—	—
Temp	X_rf1_next	Real	—	—
Temp	X_rf2_next	Real	—	—
Temp	Y_prev_next	Real	—	—
Temp	Y_pb_next	Real	—	—
Temp	R_pb_next	Real	—	—
Temp	Sw_next	Real	—	—
Temp	Dy	Real	—	—
Temp	Dr	Real	—	—
Temp	Dy_prev	Real	—	—
Temp	Dy_dot	Real	—	—
Temp	Yf	Real	—	—
Temp	Ydot_f	Real	—	—
Temp	Rf	Real	—	—
Temp	E	Real	—	—
Temp	Du	Real	—	—
Temp	U_unsat	Real	—	—
Temp	DTheta1	Real	—	—

Sekcia	Premenná	Dátový typ	Predvolená hodnota	Retain
Temp	DTheta2	Real	–	–
Temp	DTheta3	Real	–	–
Constant	A0M	Real	1.0	–
Constant	A1M	Real	20.0	–
Constant	B0M	Real	1.0	–
Constant	Alpha1	Real	0.14	–
Constant	Alpha2	Real	0.3	–
Constant	Alpha3	Real	0.14	–
Constant	Sign_b0	Real	1.0	–
Constant	Ts	Real	0.1	–
Constant	U_pb	Real	3.0	–
Constant	U_min	Real	0.0	–
Constant	U_max	Real	10.0	–

5.4.1 Overenie na reálnom systéme

Na základe popísanej implementácie bol následne vykonaný experiment na reálnom systéme TS05. Celkový priebeh experimentu v absolútnych veličinách je zobrazený na obr. 46.

Po vynechaní úvodnej fázy ustálenia sú výsledky zobrazené aj v odchýlkových veličinách na obr. 47.

Z priebehov je zrejmé, že regulátor bol implementovaný korektne a jeho správanie je kvalitatívne podobné výsledkom získaným v prostredí MATLAB/Simulink a v jazyku C++. Výstup systému sleduje referenčný model pomerne uspokojivo a adaptačné parametre sa počas deja menia v súlade s činnosťou adaptačného zákona.

Na rozdiel od implementácie v jazyku C++ tu nepozorujeme tak výrazný drift parametrov, aký bol viditeľný v predchádzajúcom experimente. Zároveň však výsledky nedosahujú takú kvalitu sledovania ako v prípade implementácie v prostredí MATLAB. Z priebehov parametrov Θ možno pozorovať, že parametre sa adaptujú správnym smerom, avšak zároveň je na nich stále viditeľný mierny drift.

Možných príčin tohto správania môže byť viac. Stručne možno uvažovať najmä o tom, že adaptačné koeficienty prevzaté z predchádzajúcich implementácií nemusia byť úplne vhodné práve pre túto hardvérovú platformu a jej úroveň meracieho šumu. Určitú úlohu môžu zohrávať aj odlišné vonkajšie podmienky počas experimentu. Tieto faktory však z uvedených meraní nemožno jednoznačne potvrdiť.

Pokojnejší priebeh signálov môže súvisieť aj so spracovaním analógových hodnôt v PLC module. Ako bolo uvedené v podkapitole 2.3.3, aj pri vypnutom digitálnom vyhladzovaní zostáva aktívne potlačenie rušenia (*noise rejection*).

Napriek tomu možno konštatovať, že implementácia gradientného MRAC regulátora v PLC Siemens je správna a výsledné správanie systému zodpovedá charakteru výsledkov, ktoré boli pozorované pri realizácii v jazyku C++ aj v prostredí MATLAB/Simulink.

5.5 Porovnanie výsledkov z jednotlivých implementácií adaptívneho radiaceho systému

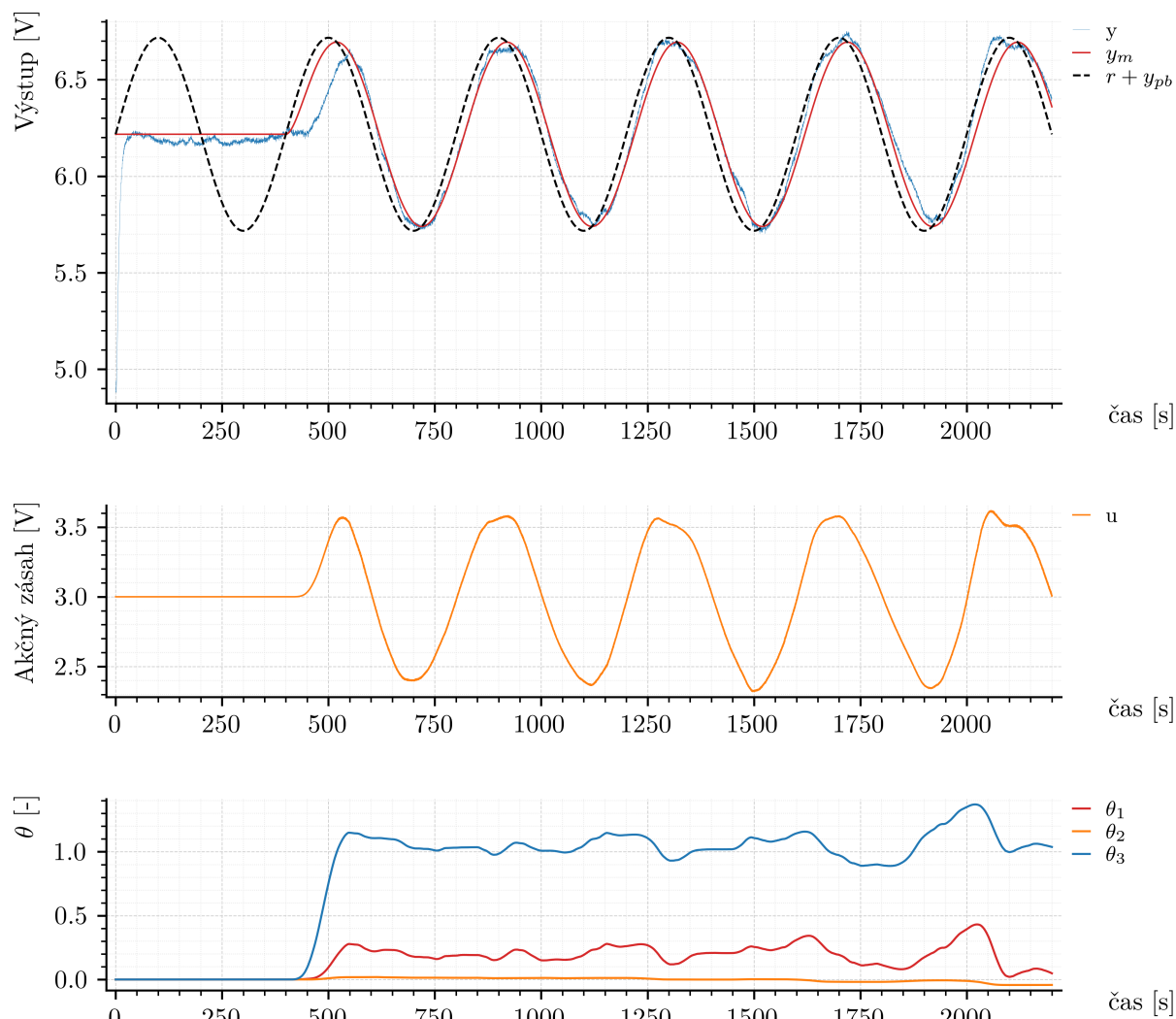
Porovnanie experimentálnych výsledkov jednotlivých implementácií MRAC regulátora je uvedené na obr. 48. Pre kvantitatívne porovnanie bola vypočítaná hodnota RMSE medzi priebehmi Δy a Δy_m :

$$\text{RMSE}_{\text{MATLAB}} = 0.062104 \text{ V}, \quad (5.88)$$

$$\text{RMSE}_{\text{C++}} = 0.176280 \text{ V}, \quad (5.89)$$

$$\text{RMSE}_{\text{PLC}} = 0.074174 \text{ V}, \quad (5.90)$$

$$\text{RMSE}_{\text{C++ ex.}} = 0.127432 \text{ V}. \quad (5.91)$$

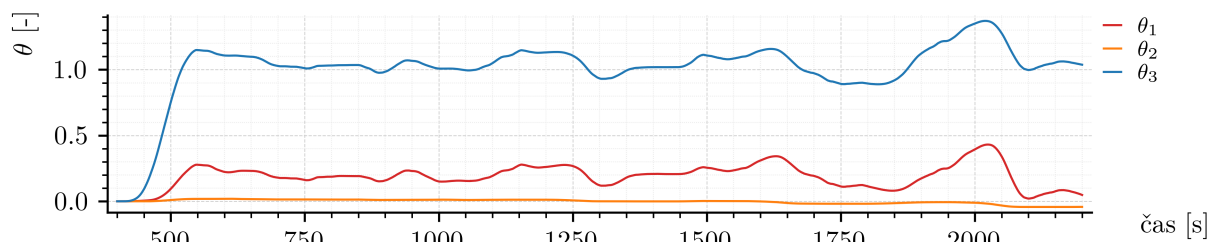
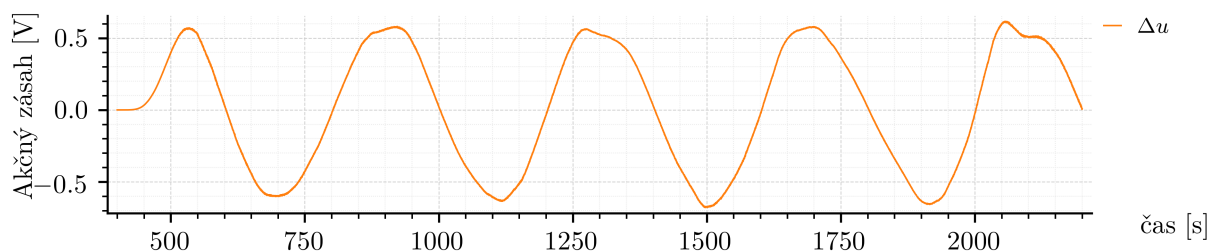
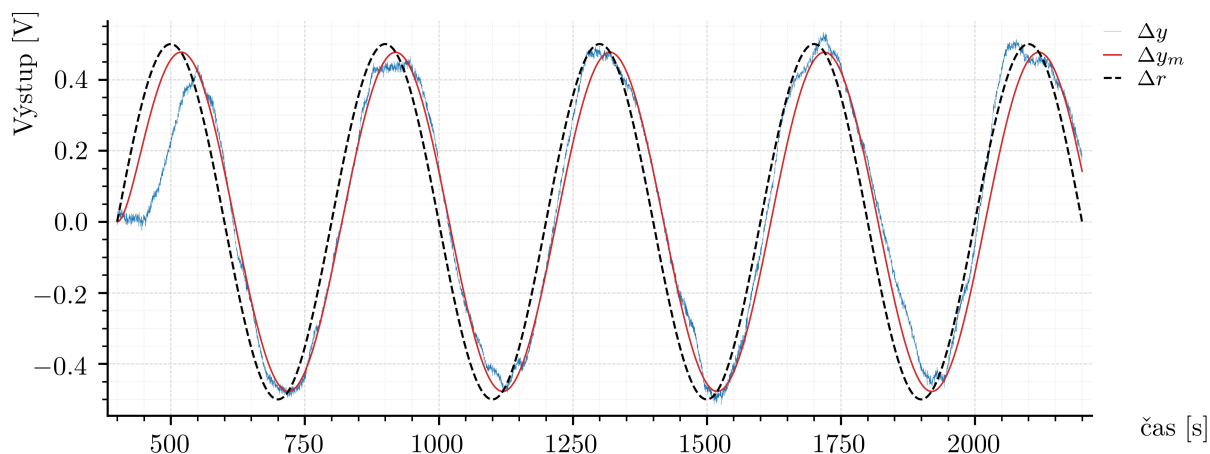


Obr. 46: Experimentálne výsledky gradientného MRAC regulátora implementovaného v PLC Siemens v absolútnych veličinách

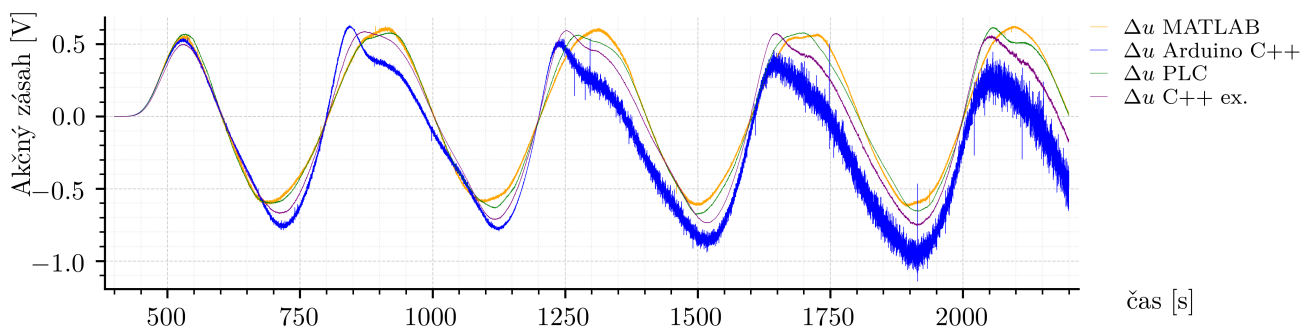
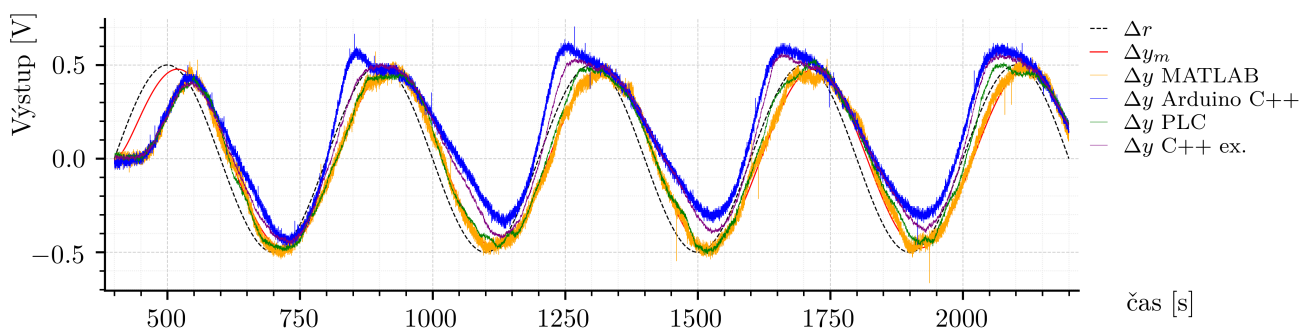
Z porovnania je zrejmé, že výsledky jednotlivých implementácií nie sú také podobné ako pri PI regulátore zo sekcie 3 alebo pri prepínaní pracovných bodov zo sekcie 4. To súvisí s tým, že gradientný MRAC regulátor je citlivejší na praktické podmienky experimentu. Na výsledné správanie môže mať vplyv najmä úroveň a charakter meracieho šumu, diskrétno rozlíšenie vstupov a výstupov, vonkajšie podmienky počas merania a tiež samotná voľba adaptačných koeficientov.

Adaptačné koeficienty boli v tejto práci zvolené empiricky na základe správania systému v prostredí MATLAB/Simulink a následne boli použité aj na ostatných platformách. Z uvedených meraní však nemožno jednoznačne určiť, ktorý z faktorov mal na rozdiely medzi experimentmi rozhodujúci vplyv.

Napriek týmto rozdielom výsledky ukazujú, že základná implementácia MRAC algoritmu bola realizovaná na všetkých uvažovaných platformách. Detailná optimalizácia adaptačných parametrov pre každú platformu samostatne však nebola cieľom tejto práce.



Obr. 47: Experimentálne výsledky gradientného MRAC regulátora implementovaného v PLC Siemens po prechode do súradníc pracovného bodu



Obr. 48: Porovnanie experimentálnych výsledkov gradientného MRAC regulátora na jednotlivých platformách

Záver

Cieľom tejto diplomovej práce bolo navrhnúť implementáciu riadiaceho systému pre vybrané laboratórne zariadenie predstavujúce riadený systém. Hlavný dôraz bol kladený na praktickú realizáciu riadiaceho systému v rámci dostupných implementačných platforiem, konkrétne v prostredí MATLAB/Simulink, v jazyku C++ na vývojovej doske Arduino UNO a na priemyselnom PLC systéme Siemens SIMATIC S7-1200 G2.

V rámci práce bol najskôr spracovaný opis použitého laboratórneho zariadenia TS05, spôsob jeho pripojenia k jednotlivým platformám a základné postupy potrebné pre prácu s jeho vstupnými a výstupnými signálmi. Pre potreby návrhu regulátorov bola vykonaná identifikácia systému na základe experimentálne nameraných údajov. Získaný model bol použitý pri návrhu regulačných algoritmov a pri ich počiatočnom simulačnom overení.

Hlavným výsledkom práce je realizácia troch algoritmov riadenia na reálnom laboratórnom zariadení. Prvým z nich je PI regulátor, ktorý predstavuje základné riešenie pre riadenie v okolí jedného pracovného bodu. Druhým je riadiaci systém, ktorý je implementáciou vlastného návrhu PI regulácie s prepínaním pracovných bodov, ktorý rozšíril použiteľný pracovný rozsah riadenia kombináciou globálnej a lokálnej regulačnej vetvy. Tretím bol algoritmus z oblasti adaptívneho riadenia s referenčným modelom (MRAC) založený na gradientnej metóde, ktorý umožnil overiť implementáciu riadiaceho systému z kategórie pokročilých metód automatického riadenia na reálnom hardvéri.

Navrhnuté algoritmy boli implementované na všetkých uvažovaných platformách. Tým sa podarilo overiť nielen ich regulačný princíp, ale aj praktické aspekty spojené s realizáciou riadiaceho systému, ako sú časovanie výpočtu, uchovávanie stavových premenných, škálovanie analógových signálov, saturácia akčného zásahu a časový záznam signálov. Porovnanie výsledkov medzi platformami slúžilo ako doplnkové vyhodnotenie a ukázalo, že pri jednoduchších regulačných štruktúrach možno dosiahnuť veľmi podobné správanie, zatiaľ čo adaptívny algoritmus je výraznejšie ovplyvnený vlastnosťami konkrétnej implementácie.

Z experimentálnych výsledkov vyplynulo, že PI regulátor je vhodný najmä pre lokálne riadenie v okolí pracovného bodu. Algoritmus s prepínaním pracovných bodov umožnil realizovať prechody medzi rôznymi pracovnými oblasťami systému a následne pokračovať v lokálnej regulácii. Pri MRAC regulátore sa potvrdila možnosť implementácie adaptačného mechanizmu, zároveň sa však prejavila jeho citlivosť na prítomnosť šumu v spätnoväzbovom signále, numerickú deriváciu, drift pracovného bodu a voľbu adaptačných koeficientov. Heuristická úprava použitá pri C++ implementácii znížila drift parametrov, ale úplne ho neodstránila.

Možno konštatovať, že stanovené ciele práce boli splnené. Bol zostavený riadiaci systém pre reálny laboratórny dynamický systém, pričom bola navrhnutá implementácia niekoľkých algoritmov riadenia a ich funkčnosť bola overená experimentálne. Práca zároveň ukázala, že prenos algoritmu zo simulačného prostredia na reálny hardvér nie je iba formálnym prepísaním kódu, ale samostatnou technickou úlohou, ktorá vyžaduje zohľadnenie vlastností merania, akčných členov, výpočtovej platformy a fyzikálneho správania riadeného systému.

Práca tiež vytvára základ pre modifikáciu a nadviazanie na prezentované výsledky napríklad z hľadiska zlepšenia prezentovaného adaptívneho algoritmu riadenia v zmysle jeho robustnosti, teda overiť normalizáciu regresora, projekčný algoritmus, σ -modifikáciu alebo iné stabilizačné úpravy zákona adaptácie. Pri algoritme s prepínaním pracovných bodov by bolo možné rozšíriť počet pracovných bodov, doplniť interpoláciu parametrov lokálnych regulátorov alebo upraviť podmienku detekcie ustálenia. Samostatným smerom ďalšej práce by mohlo byť rozšírenie návrhu zo SISO prístupu na využitie MIMO charakteru systému TS05.

Literatúra

- [1] M. Fikar, J. Mikleš, *Identifikácia systémov*. Slovenská technická univerzita v Bratislave, 1998. ISBN 80-227-1177-2.
- [2] Ústav robotiky a kybernetiky FEI STU v Bratislave, Učebný text KUT015: *Laboratórne zariadenie TS: orientačný prehľad*. Slovenská technická univerzita v Bratislave, 2025. Dostupné online: <https://github.com/OkoliePracovnehoBodu/KUT/>
- [3] D. C. Montgomery, E. A. Peck, G. G. Vining, *Introduction to Linear Regression Analysis, Fifth Edition*. John Wiley & Sons, Inc., Hoboken, New Jersey, 2012. ISBN 978-0-470-54281-1.
- [4] L. Wang, *PID Control System Design and Automatic Tuning using MATLAB/Simulink*. Wiley-IEEE Press, 2020. ISBN 978-1-119-46934-6.
- [5] F. E. Cellier, *Continuous System Modeling*. Springer-Verlag, New York, 1991. ISBN 978-1-4757-3924-4.
- [6] K. J. Åström, T. Hägglund, *PID Controllers: Theory, Design, and Tuning*. ISA – The Instrumentation, Systems and Automation Society, Research Triangle Park, North Carolina, 1995. ISBN 1-55617-516-7.
- [7] K. Ogata, *Modern Control Engineering*. 5th ed. Prentice Hall, Upper Saddle River, 2010. ISBN 0-13-615673-8.
- [8] N. S. Nise, *Control Systems Engineering*. 7th ed. John Wiley & Sons, Inc., Hoboken, New Jersey, 2015. ISBN 978-1-118-80082-9.
- [9] P. Hudzovič, *Optimalizácia*. Vydavateľstvo STU, Bratislava, 2000.
- [10] G. Tao, *Adaptive Control Design and Analysis*. John Wiley & Sons, Inc., Hoboken, New Jersey, 2003. ISBN 0-471-27452-6.
- [11] J. Murgaš, I. Hejda, *Adaptívne riadenie technologických procesov*. Slovenská technická univerzita v Bratislave, Bratislava, 1993. 176 s. ISBN 80-227-0529-2.
- [12] Siemens AG, *SIMATIC S7-1200 G2 Programmable Logic Controller: System Manual*. Siemens Industry Online Support, 2024. Entry ID: 109972011. Dostupné online: <https://support.industry.siemens.com/cs/document/109972011/s7-1200-g2-programmable-controller>

Prílohy

P.1 Hlavný program pre mikrokontrolér platformy Arduino UNO

```
1 #include <Arduino.h>
2
3 // a1 -> snimac1
4 // a2 -> snimac2
5 // pum5 -> vent
6 // pum6 -> spirala
7
8 // Pin configuration
9 constexpr uint8_t PWM_OUT_VENT = 5;
10 constexpr uint8_t PWM_OUT_SPIR = 6;
11 constexpr uint8_t ANALOG_IN_1 = A1;
12 constexpr uint8_t ANALOG_IN_2 = A2;
13
14 // Experiment timing
15 constexpr uint32_t TS_US = 100000UL;
16 constexpr float TS_WARNING_S = 0.11f;
17 constexpr float EXPERIMENT_DURATION_S = 30.0f;
18
19 // Experiment state
20 bool experimentRunning = false;
21
22 // Global experiment timing
23 uint32_t experimentStartUs = 0;
24 uint32_t lastControlTickUs = 0;
25
26 // HELPER FUNCTIONS
27 ///////////////////////////////////////////////////////////////////
28 // Cubic polynomial evaluation
29 float cubic(float x, float a, float b, float c, float d) {
30     return a * x * x * x + b * x * x + c * x + d;
31 }
32
33 // Clamp float value into range
34 float clampFloat(float value, float minValue, float maxValue) {
35     if (value < minValue) {
36         return minValue;
37     }
38     if (value > maxValue) {
39         return maxValue;
40     }
41     return value;
42 }
43
44 // Clamp integer value into range
45 int clampInt(int value, int minValue, int maxValue) {
46     if (value < minValue) {
47         return minValue;
48     }
49     if (value > maxValue) {
50         return maxValue;
51     }
52     return value;
53 }
54
55 // SERIAL COMMANDS
56 ///////////////////////////////////////////////////////////////////
57 // Stop all outputs
58 void stopOutputs() {
59     analogWrite(PWM_OUT_VENT, 0);
60     analogWrite(PWM_OUT_SPIR, 0);
61 }
62
63 // Start experiment and reset timers
64 void startExperiment() {
65     experimentStartUs = micros();
66     lastControlTickUs = experimentStartUs;
67     experimentRunning = true;
68
69     Serial.println("t,Ts,snimac1");
70 }
71
```

```

72 // Handle serial commands
73 void handleSerialCommands() {
74     if (Serial.available() <= 0) {
75         return;
76     }
77
78     String command = Serial.readStringUntil('\n');
79     command.trim();
80
81     if (command == "START") {
82         if (!experimentRunning) {
83             startExperiment();
84         }
85     } else if (command == "STOP") {
86         experimentRunning = false;
87         stopOutputs();
88         Serial.println("STOPPED");
89     }
90 }
91
92 void setup() {
93     pinMode(PWM_OUT_VENT, OUTPUT);
94     pinMode(PWM_OUT_SPIR, OUTPUT);
95     pinMode(ANALOG_IN_1, INPUT);
96     pinMode(ANALOG_IN_2, INPUT);
97
98     Serial.begin(115200);
99
100     stopOutputs();
101
102     delay(300);
103     Serial.println("READY");
104 }
105
106 // INIT control variables //////////////////////////////////////
107
108 float vent = 0.0f;
109 float spirala = 0.0f;
110
111 // MAIN CONTROL LOOP //////////////////////////////////////
112
113 void loop() {
114     handleSerialCommands();
115
116     if (!experimentRunning) {
117         return;
118     }
119
120     uint32_t nowUs = micros();
121     uint32_t elapsedSinceLastTickUs = nowUs - lastControlTickUs;
122
123     // Loop executes only when real elapsed time reached Ts
124     if (elapsedSinceLastTickUs < TS_US) {
125         return;
126     }
127
128     float timeTickPeriod = static_cast<float>(elapsedSinceLastTickUs) /
129         1000000.0f;
130     float globalTime = static_cast<float>(nowUs - experimentStartUs) / 1000000.0f;
131
132     // Update tick timestamp only when control loop actually runs
133     lastControlTickUs = nowUs;
134
135     // OUTPUT control variables (vent, spirala) //////////////////////////////////
136
137     // Limit control variables to 0..10
138     vent = clampFloat(vent, 0.0f, 10.0f);
139     spirala = clampFloat(spirala, 0.0f, 10.0f);
140
141     // Convert vent -> PWM5 count using cubic calibration
142     float pwm5Float = cubic(
143         vent,
144         0.05938291f,
145         -0.27988543f,
146         24.20478979f,
147         -0.52522611f
148     );

```

```

149 // Convert spirala -> PWM6 count using cubic calibration
150 float pwm6Float = cubic(
151     spirala,
152     0.08732906f,
153     -0.39322033f,
154     22.8496962f,
155     -0.61564759f
156 );
157
158 // Round to nearest integer and clamp to valid PWM range
159 int pwm5 = clampInt(static_cast<int>(pwm5Float + 0.5f), 0, 255);
160 int pwm6 = clampInt(static_cast<int>(pwm6Float + 0.5f), 0, 255);
161
162 // Output PWM
163 analogWrite(PWM_OUT_VENT, pwm5);
164 analogWrite(PWM_OUT_SPIR, pwm6);
165
166 // INPUT processing //////////////////////////////////////
167
168 // Read raw ADC values
169 int a1 = analogRead(ANALOG_IN_1);
170 int a2 = analogRead(ANALOG_IN_2);
171
172 // Convert ADC -> snimac values using cubic calibration
173 float snimac1 = cubic(
174     static_cast<float>(a1),
175     4.64044633e-10f,
176     -8.44254606e-07f,
177     1.02946107e-02f,
178     5.87859135e-05f
179 );
180
181 float snimac2 = cubic(
182     static_cast<float>(a2),
183     -9.09109415e-10f,
184     8.99002333e-07f,
185     9.78867803e-03f,
186     -9.65193864e-05f
187 );
188
189 //////////////////////////////////////
190 // Control law area - update control variables (vent, spirala) based on
191 // snimac1, snimac2, globalTime, etc.
192
193 vent = ...;
194 spirala = ...;
195
196 //////////////////////////////////////
197
198 // End experiment after duration expires //////////////////////////////////
199
200 // After experiment duration expires, force control inputs to zero
201 bool experimentFinished = false;
202 if (globalTime >= EXPERIMENT_DURATION_S) {
203     vent = 0.0f;
204     spirala = 0.0f;
205     experimentFinished = true;
206 }
207
208 // Warning flag for too-large real period
209 int warnTs = (timeTickPeriod > TS_WARNING_S) ? 1 : 0;
210
211 // Print CSV line with all relevant data for logging and analysis ///
212
213 // CSV output to Serial
214 Serial.print(globalTime, 6);
215 Serial.print(",");
216
217 Serial.print(timeTickPeriod, 6);
218 Serial.print(",");
219
220 Serial.print(snimac1, 6);
221 Serial.print(",");
222
223 // Finish experiment and return to idle state
224 if (experimentFinished) {
225     stopOutputs();

```

```

226         experimentRunning = false;
227         Serial.println("DONE");
228     }
229 }

```

Výpis kódu 30: Hlavný program pre mikrokontrolér platformy Arduino UNO

P.2 Externý skript pre logovanie dát

```

1  import csv
2  import os
3  import sys
4  import time
5  from datetime import datetime
6
7  import serial
8
9
10 PORT = "COM3"
11 BAUDRATE = 115200
12 TIMEOUT_S = 1.0
13 OUTPUT_DIR = "logs"
14
15 CSV_HEADER = "t,Ts,snimac1"
16
17
18 def make_output_path() -> str:
19     os.makedirs(OUTPUT_DIR, exist_ok=True)
20     timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
21     return os.path.join(OUTPUT_DIR, f"experiment_{timestamp}.csv")
22
23
24 def print_red(text: str) -> None:
25     print(f"\033[31m{text}\033[0m")
26
27
28 def main() -> None:
29     output_path = make_output_path()
30
31     print(f"Opening serial port: {PORT} @ {BAUDRATE}")
32     print(f"Output file: {output_path}")
33
34     try:
35         with serial.Serial(PORT, BAUDRATE, timeout=TIMEOUT_S) as ser:
36             # Let Arduino reset after opening the port
37             time.sleep(2.0)
38
39             print("Waiting for READY...")
40
41             while True:
42                 raw_line = ser.readline()
43                 if not raw_line:
44                     continue
45
46                 line = raw_line.decode("utf-8", errors="replace").strip()
47                 if not line:
48                     continue
49
50                 print(f"RX: {line}")
51                 if line == "READY":
52                     break
53
54             with open(output_path, "w", newline="", encoding="utf-8") as csv_file:
55                 writer = csv.writer(csv_file)
56
57                 print("Sending START...")
58                 ser.write(b"START\n")
59                 ser.flush()
60
61                 header_received = False
62
63                 while True:
64                     raw_line = ser.readline()
65                     if not raw_line:
66                         continue
67

```

```

68         line = raw_line.decode("utf-8", errors="replace").strip()
69         if not line:
70             continue
71
72         if line == "DONE":
73             print("Experiment finished.")
74             break
75
76         if line == "STOPPED":
77             print("Experiment stopped.")
78             break
79
80         if not header_received:
81             if line == CSV_HEADER:
82                 writer.writerow(line.split(","))
83                 csv_file.flush()
84                 header_received = True
85                 print("CSV header received. Logging started.")
86             else:
87                 print(f"Skipping: {line}")
88                 continue
89
90         parts = [part.strip() for part in line.split(",")]
91         writer.writerow(parts)
92         csv_file.flush()
93
94         warn_ts = len(parts) >= 11 and parts[10] == "1"
95         if warn_ts:
96             print_red(line)
97         else:
98             print(line)
99
100     except serial.SerialException as exc:
101         print(f"Serial error: {exc}")
102         sys.exit(1)
103     except KeyboardInterrupt:
104         print("\nInterrupted by user.")
105
106
107 if __name__ == "__main__":
108     main()

```

Výpis kódu 31: Externý skript pre logovanie dát